



iOS SDK 1.11.0.0 Additional Works User Guide



This document is intended only for authorized individuals of AppSealing customer.
Unauthorized distribution of any parts of this document to other parties is prohibited.

The software referenced herein, this User Guide, and any associated documentation is provided to you pursuant to the agreement between your company, governmental body or other entity (“you”) and Doverrunner Corporation (“AppSealing”) under which you have received a copy of AppSealing Licensed Technology and this User Guide (such agreement, the “Agreement”). Defined terms not defined herein shall have the meanings ascribed to them in the Agreement. In the event of conflict between the terms of this User Guide and the terms of the Agreement, the terms of the Agreement shall prevail. Without limiting the generality of the remainder of this paragraph, (a) this User Guide is provided to you for informational purposes only, (b) your right to access, view, use, and copy this User Guide is limited to the rights and subject to the applicable requirements and limitations set forth in the Agreement, and (c) all of the content of this User Guide constitutes “Confidential Information” of AppSealing (or the equivalent term used in the Agreement) and is subject to all of the limitations and requirements pertinent to the use, disclosure and safeguarding of such information. Permitting anyone who is not directly involved in the authorized use of AppSealing Licensed Technology by your company or other entity to gain any access to this User Guide shall violate the Agreement and subject your company or other entity to liability therefore.

Copyright Information

Copyright © 2000-2025 Doverrunner Corporation. All rights reserved.

AppSealing® is a trademark of Doverrunner Corporation in the South Korea and/or other countries.

macOS™ and Xcode® are trademarks of Apple Inc., registered in the United States and other countries.

All other trademarks are the property of their respective owners.

Disclaimer

The remainder of this User Guide notwithstanding, this User Guide is provided “as is”, without any warranty whatsoever (including that it is error-free or complete). This User Guide contains no express or implied warranties, covenants or grants of rights or licenses, and does not modify or supplement any express warranty, covenant or grant of rights or licenses that is set forth in the Agreement. This User Guide is current as of the date set forth in the header that appears above on this page, but may be modified at any time without prior notice. Future revisions and updates of this User Guide shall be distributed as part of AppSealing SDK new releases. Doverrunner shall under no circumstances bear any responsibility for your failure to operate AppSealing Licensed Technology in compliance with the then-current version of this User Guide. Your remedies with respect to your use of this User Guide, and Doverrunner's liability for your use of this User Guide (including for any errors or inaccuracies that appear in this User Guide) are limited to those remedies expressly authorized by the Agreement (if any).

Contact Information

Address: [06109] 1F 608, Nonhyeon-ro, Gangnam-gu, Seoul, South Korea

Technical support: <https://helpcenter.appsealing.com>

Website: www.doverrunner.com

목차

요구 사항	4
Part 1. 앱 서버를 이용한 강화된 탈옥 감지 적용 방법	
1-1 강화된 탈옥 감지 방안의 개요 및 필요성	5
1-2 iOS 앱 코드 작업	6
1-3 서버 측 검증 작업	8
Part 2. Xcode Cloud 적용	
2-1 Xcode Cloud 설정	12
2-2 ci_scripts 준비 사항	14
2-3 빌드 및 업로드 과정 확인	17
Part 3. Fastlane 적용	
3-1 프로젝트 Fastfile 설정	22
3-2 Github Action과 Fastlane 동시 적용	23
Part 4. Azure 파이프라인 적용	
4-1 인증서 및 프로파일 준비	27
4-2 Azure 프로젝트 설정	28
4-3 Azure 빌드 스크립트 수정	32
Part 5. Bitrise 파이프라인 적용	
5-1 인증서 및 프로파일 준비	34
5-2 Bitrise CI 워크스페이스 설정	35
5-3 Bitrise CI 프로젝트 설정	39
Part 6. Circle CI 파이프라인 적용	
6-1 인증서 및 프로파일 준비	50
6-2 Circle CI 프로젝트 설정	51
6-3 Circle CI 프로젝트 빌드	55

Part 7. 기타 / 공통 작업

7-1 배포용 인증서 PKCS#12 파일로 내보내기.	57
7-2 앱 전용 비밀번호 생성하기	60

Prerequisite

- Xcode 15.0 이상 (macOS 14 이상)
- iOS 12.0 이상 기기

Part 1. 앱 서버를 이용한 강화된 탈옥 감지 적용 방법

1-1 강화된 탈옥 감지 방안의 개요 및 필요성

AppSealing SDK의 주요 기능 중 탈옥된 단말의 환경을 감지하여 강제로 앱을 종료시키는 기능이 포함되어 있습니다. 그러나 더욱 고도화된 공격 방법에 의해 이러한 탐지 기능이 우회될 가능성이 존재합니다. 이는 iOS의 운영체제 특성 상 앱 실행 시 로드된 동적 라이브러리(dylib)의 코드가 먼저 실행되기 때문인데 공격자는 이러한 동적 라이브러리에 실행 파일의 특정 영역을 패치하는 코드를 담아 배포하는 경우가 있기 때문입니다.

AppSealing의 탐지 로직이 실행되기 이전 시점에 이러한 코드 패치가 일어날 경우 앱을 종료시키는 코드가 제거된 상태가 때문에 탈옥이 감지되더라도 앱은 계속 실행 중인 상태가 됩니다.

물론 이런 유형의 공격을 누구나 쉽게 할 수 있는 것은 아니지만 전문적인 해킹 지식을 가진 해커 그룹에 의해 이런 방식의 공격이 실제 이루어진 상황이 확인되었으므로 AppSealing은 이런 공격 상황에도 대비할 수 있는 추가적인 탈옥 감지 방법을 제공합니다.

이 공격 방식의 특징은 실행 중인 앱의 코드를 사전에 변경시켜 버리는 것이기 때문에 AppSealing 라이브러리 자체에 아무리 강력한 탐지 로직을 추가하더라도 동적 라이브러리에 의해 코드가 패치 되는 상황은 피할 수 없게 됩니다. 따라서 신규로 제공하는 탈옥 감지 기능은 앱에서 감지를 하는 것이 아니라 앱과 연동되는 서버에서 탈옥이 의심되는 단말일 경우 로그인이나 API 호출 수행 등의 모든 서비스와 작업을 거부하는 방식으로 이루어집니다.

기본적인 방식은 앱에서 AppSealing 인터페이스를 통해 서버 자격 증명(credential)을 가져오고 이를 기존의 인증 정보에 추가하여 서버로 전송하면 서버가 이 값의 진위 여부를 검증하여 서비스를 지속하거나 거부하는 등의 동작을 하도록 하는 것입니다.

서버와 연동되지 않는 클라이언트 단독 앱일 경우 이 방법을 적용할 수 없습니다.

다음 절에서 서버 자격 증명을 얻고 이를 검증하기 위한 방법을 예제 코드와 함께 설명합니다.

1-2 iOS 앱 코드 작업

서버 자격 증명 검증을 위해 앱에서 추가해야 할 작업은 AppSealing SDK의 함수를 호출하여 서버 자격 증명 문자열을 가져온 뒤 이를 기존의 인증 정보와 함께 서버로 전송하는 것입니다.

서버와 연동되는 앱의 경우 대부분 사용자 인증이나 로그인 과정을 거칠 것이고 이 과정에서 사용자가 입력한 계정 정보를 서버로 전송할 것입니다. 이때 서버로 전송하는 파라미터에 서버 자격 증명 문자열을 추가하면 됩니다.

서버 자격 증명 문자열은 다음과 같은 방법으로 가져옵니다.

Simple UI code into 'ViewController.swift' for **Swift** project

```
let _instAppSealing_auto_generated3: AppSealingInterface = AppSealingInterface()
_instAppSealing_auto_generated3._GetEncryptedCredentialAsync { credentialPointer in
    if let ptr = credentialPointer {
        let credential = String( cString: ptr )
        NSLog( "[AppSealing] Credential: \(credential)" )
        // credential 값을 인증 정보와 함께 서버로 올린다
        ...
    }
    else {
        NSLog( "[AppSealing] Failed to get credential" )
    }
}
```

Simple UI code into 'ViewController.mm' for **Objective-C** project

```
- (BOOL)userLogin:(NSString*)userID withPassword:(NSString*)password
[inst _GetEncryptedCredentialAsync:^(const char *credential) {
    NSString *credentialStr = credential ? [NSString stringWithUTF8String:credential] : nil;
    if ( credentialStr )
    {
        [msg appendString:credentialStr];
        NSLog(@"[AppSealing] Credential: %@", msg);
        // use this credential value in your authentication function
        // BOOL loginResult = processLogin(userID, password, _appSealingCredential);
    }
    else
        NSLog(@"[AppSealing] Failed to get credential");
}];
```

- 위 코드는 SDK의 Code Samples.txt에 포함되어 있습니다. 반드시 이 파일에서 코드를 복사하여 사용하세요.
- 기존에 사용되던 `_GetEncryptedCredential`, `ObjC_GetEncryptedCredential` 함수는 더 이상 사용되지 않습니다. 이 함수들을 사용할 경우 시스템에 의해 강제로 앱이 중지될 가능성이 있습니다. 반드시 `GetEncryptedCredentialAsync` 함수를 사용하셔야 합니다.

서버에서 자격 증명 검증에 실패할 경우 로그인도 실패 처리하고 이후 앱이 더 이상 진행되지 않도록 구성해야 합니다. 그러나 앱에서 결과를 확인하고 앱을 종료하는 등의 코드는 공격자에 의해 변조될 가능성이 있기 때문에 가장 좋은 방법은 서버에서 자격 증명 검증에 실패한 이후 해당 클라이언트의 어떤 요청에 대해서도 서비스나 응답을 거부하도록 서버를 구성하는 것입니다.

이에 대해서는 다음 절에서 다시 설명됩니다.

1-3 서버 측 검증 작업

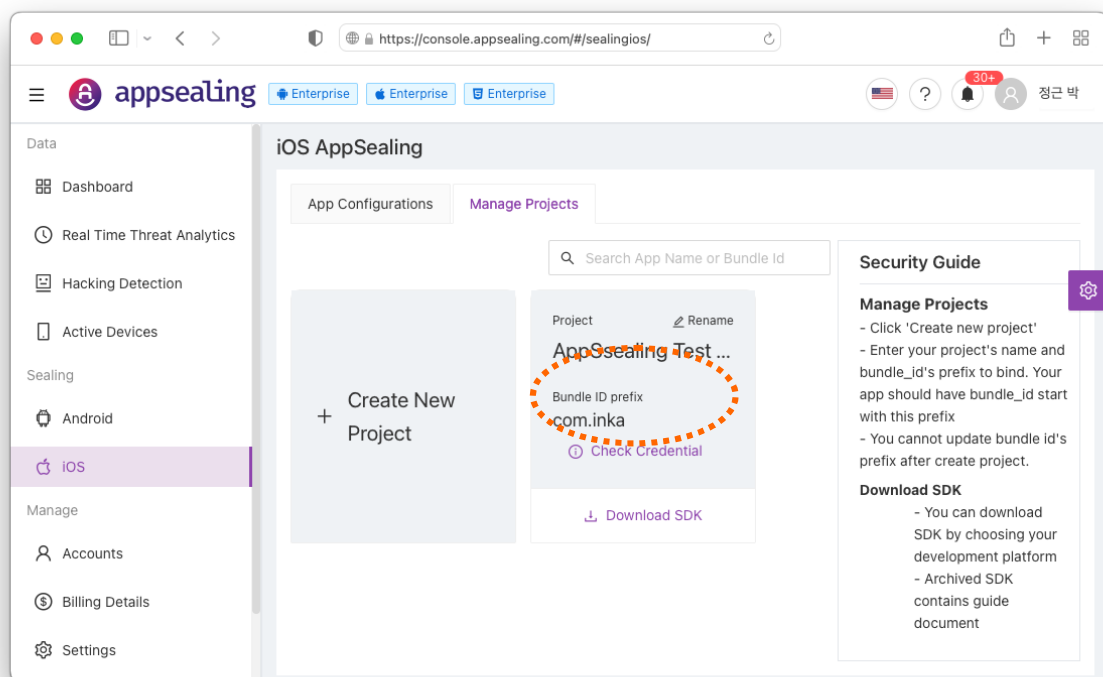
AppSealing 모듈에서 인터페이스 호출을 통해 반환되는 자격증명 데이터(credential)는 AppSealing 내부의 보안 로직이 정상적으로 수행되고 단말에서 위험 상황이 감지되지 않았을 경우만 정상적인 값이 리턴 됩니다.

만약 동적 라이브러리를 통한 코드 패치 공격이 이루어졌거나 기타 다른 방법으로 보안 로직을 우회했다면 정상적인 자격증명 데이터가 생성되지 않을 것이기 때문에 서버에서 이를 검증하여 단말의 공격 상황을 차단할 수 있게 됩니다.

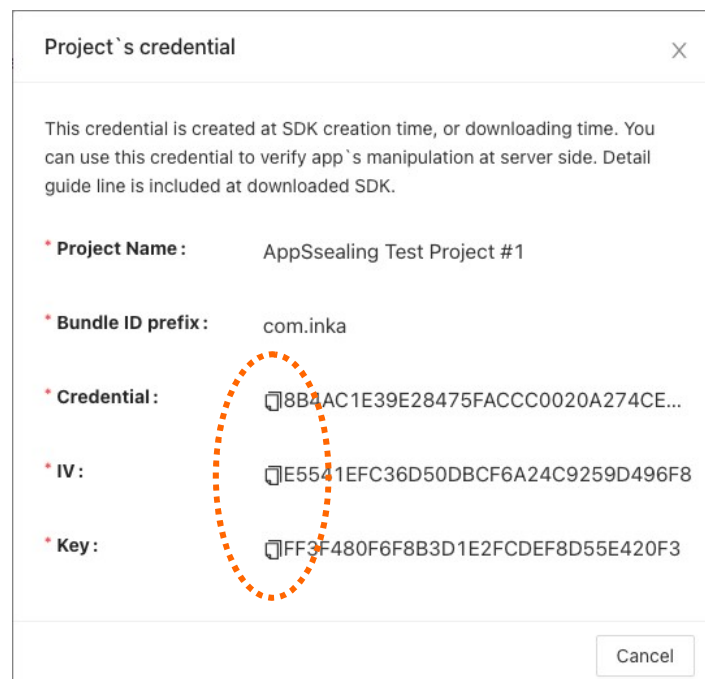
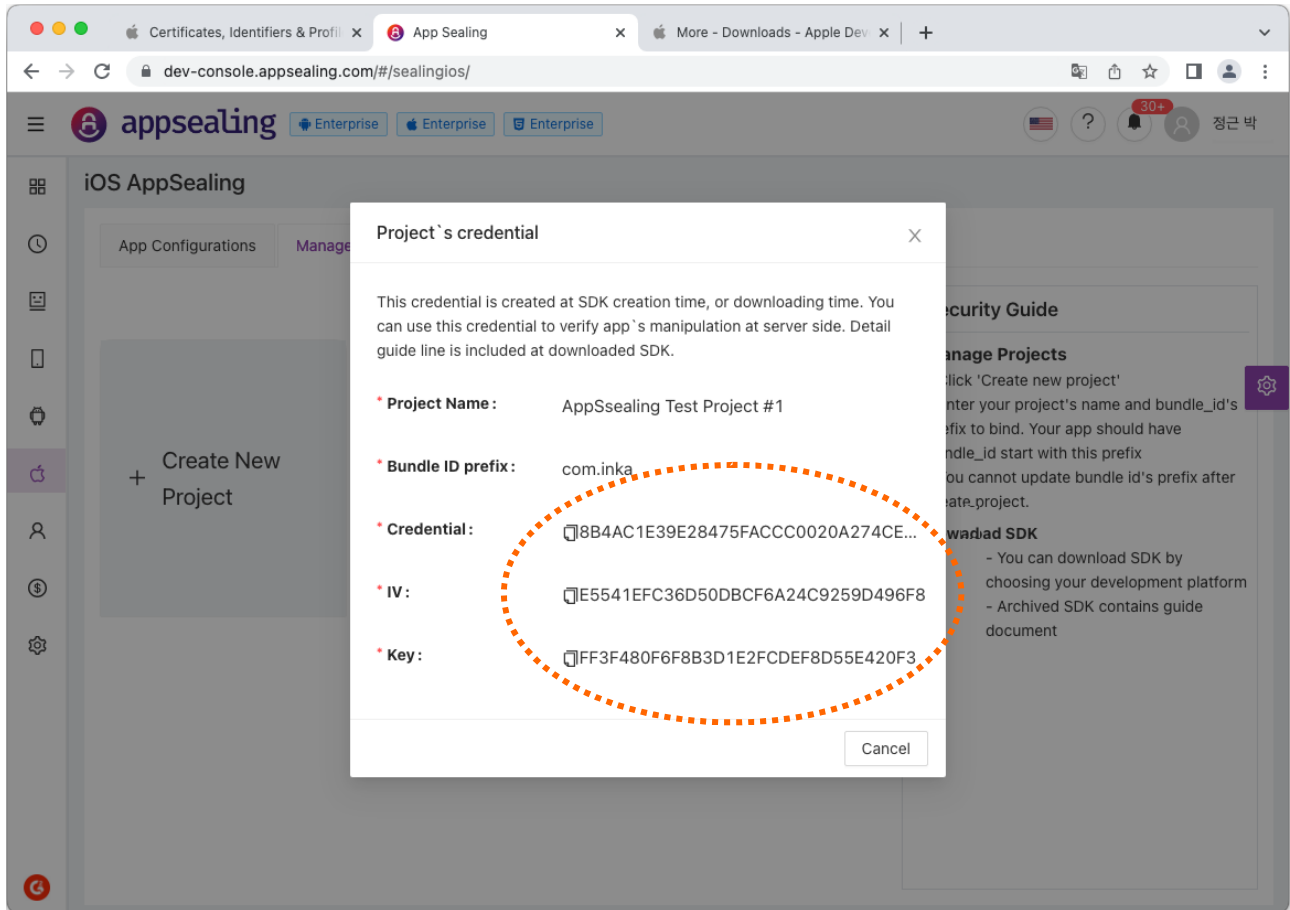
앱 서버는 클라이언트(앱)가 전송한 자격 증명 값이 올바른 것인지 확인하고 만약 올바르지 않다면 인증(로그인)을 거부한 후 이후에 해당 클라이언트가 요청하는 모든 서비스(API 호출)에 대해서도 작업을 거부해야 합니다.

서버에서 자격증명 데이터를 검증하려면 클라이언트에서 전송된 데이터를 복호화 하기 위한 AES Key와 IV, 비교 검증할 원본 자격증명 데이터가 필요합니다.

이 값들은 모두 ADC에서 해당 프로젝트의 "Check Credential" 버튼을 통해 확인할 수 있습니다. 여기 표시되는 Hex 문자열을 그대로 복사하여 예제 코드에 붙여 놓고 사용하면 됩니다. 먼저 아래 화면처럼 ADC로 접속하여 프로젝트 박스에서 "Check Credential" 버튼을 클릭합니다.

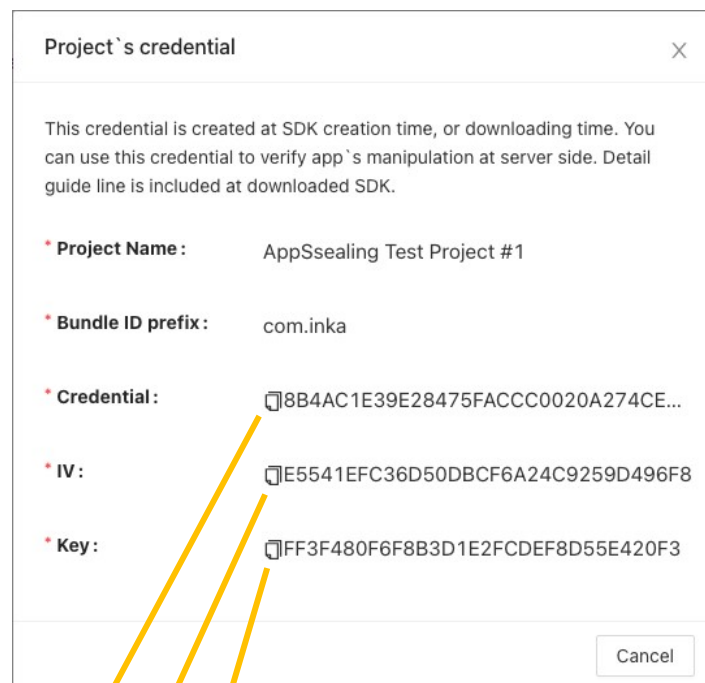


버튼을 클릭하면 아래와 같은 창이 표시되는데 여기에서 Credential 값과 복호화에 사용될 IV, key를 확인할 있습니다. 문자열 왼쪽의 복사 버튼을 이용해 이 값을 그대로 복사하여 서버 쪽 검증 코드에 붙여 넣고 사용하면 됩니다.



서버 쪽 코드가 node.js 또는 javascript로 구성되어 있는 경우 SDK의 자바스크립트 샘플 코드를 기존 코드에 추가하고 수정하여 자격증명 데이터 검증을 위해 사용하십시오. 기존 서버 코드의 로그인 또는 인증 함수에서 파라미터로 함께 올라온 credential 값을 샘플 코드에서 제공되는 verifyAppSealingCredential 함수에 넘겨 진위 여부를 판단할 수 있습니다. (샘플 코드는 SDK 내에 Server Credential/appsealing_credential_codepart.js 이름의 파일로 포함되어 있습니다)

**** 주의: 샘플 코드 중 ORG_CREDENTIAL과 AES_IV, AES_KEY는 반드시 ADC의 해당 프로젝트의 "Check Credential" 기능을 통해 가져온 값으로 대체해야 합니다. 예제 코드를 수정하지 않고 바로 사용할 경우 제대로 검증이 되지 않습니다.**



```
var crypto = require('crypto');

function verifyAppSealingCredential( credential )
{
    // Need to Change : Get From ADC (via 'Check Credential') -----
    const ORG_CREDENTIAL = "572E0E1459453F2078D6576FF71ECD0DBCA0484430C7FA7FE45B788A37DE3A04204F5A55FEA83AC9AFBA2C688594F75A3828B23972DB34858EC4F6CC3202533E44121E5F2614B227E18B6419A83810F7511D5E51FCACD5175A1CC550F83CB874A7378ACDAFE78EB2E329CD5D3C384061C4669674F1EE6B1B59FB7D91835DB7EE";

    const AES_IV = "055772B7434A4174749A009B1413472";
    const AES_KEY = "71CA94A64A4DEBF5566495AB03F6798F";
    //-----
    . . .
}
```

서버 코드가 자바스크립트가 아닌 다른 언어일 경우 해당 언어의 샘플 코드를 동일한 방식으로 사용합니다. SDK의 "Server Credential" 폴더에 있는 샘플 코드 중 서버 언어와 일치하는 코드를 기존 코드에 추가하고 수정하여 자격증명 데이터 검증을 위해 사용하십시오.

샘플 코드를 반영할 때 `ORG_CREDENTIAL`과 `AES_IV`, `AES_KEY`는 반드시 ADC의 해당 프로젝트의 "Check Credential" 기능을 통해 가져온 값으로 대체해야 합니다. 예제 코드를 그냥 가져다 바로 사용할 경우 제대로 검증이 되지 않습니다.

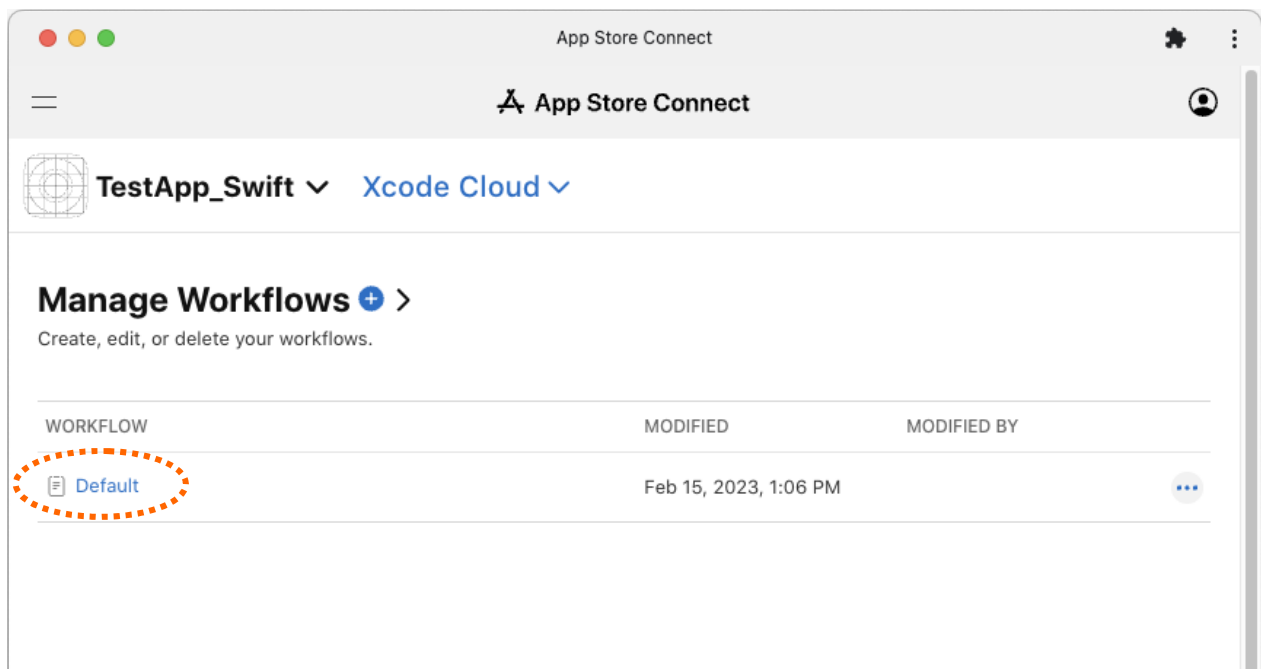
Part 2. Xcode Cloud 적용

AppSealing SDK는 Xcode Cloud를 사용하는 프로젝트에 적용할 수 있습니다. SDK에 필요한 파일들을 추가하고 Git 저장소에 푸시하면 Xcode Cloud의 동작 방식으로 앱을 보관(archive)하고 IPA로 export 한 후 재서명 스크립트를 수행하여 보안 관련 기능이 추가된 IPA를 App Store Connect로 업로드 하게 됩니다.

다만 이 과정에서 원래의 Cloud 동작과 다른 몇 가지 액션들을 수행하게 되므로 추가적인 설정과 준비 과정이 필요합니다. 다음에 설명하는 단계대로 준비 과정을 수행하세요.

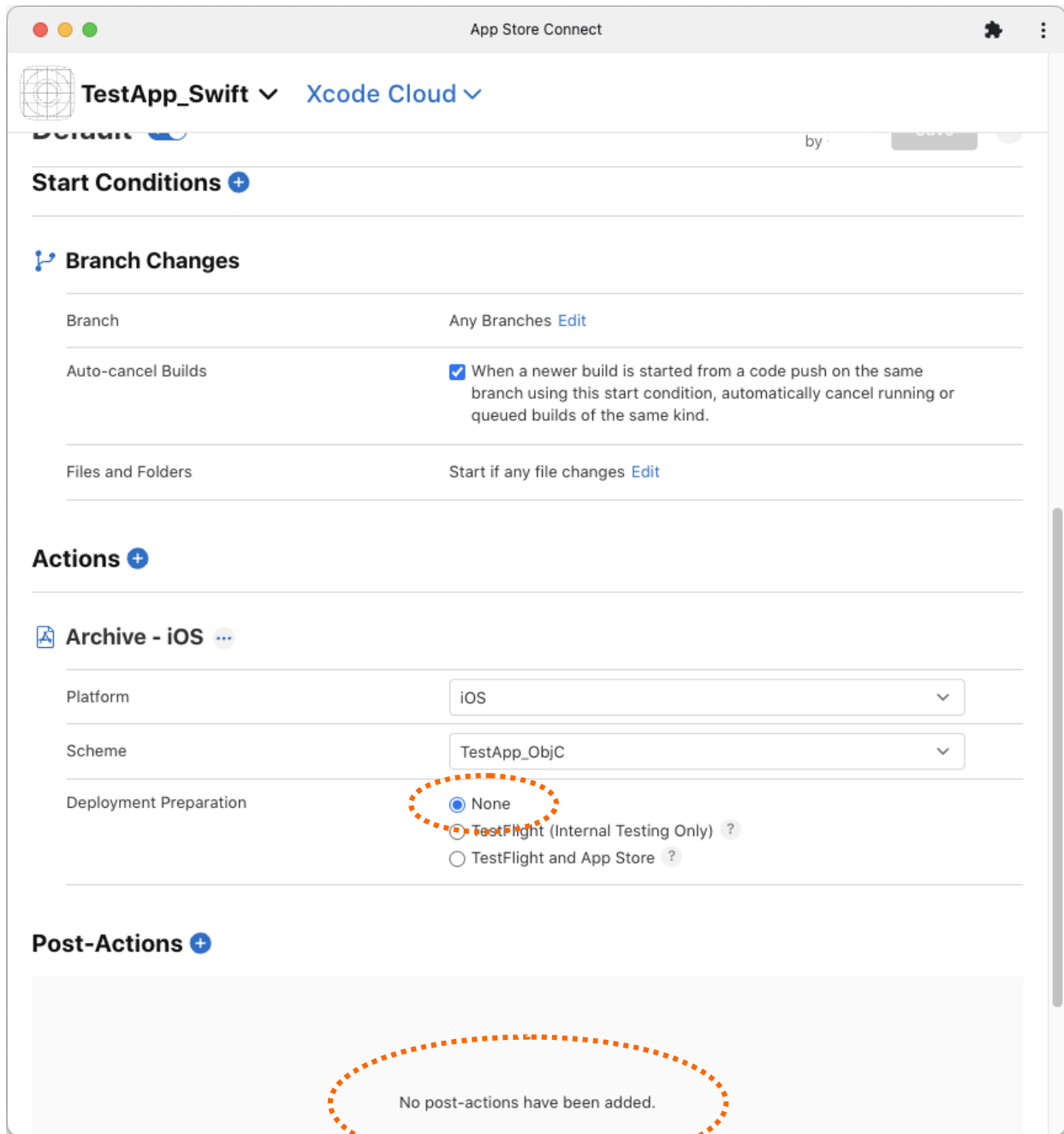
2-1 Xcode Cloud 설정

Xcode Cloud와 프로젝트 Git 저장소의 연동은 Xcode Cloud 문서 설명대로 완료되어 있는 것으로 간주합니다. 먼저 Xcode Cloud의 설정을 변경해야 합니다. 이 문서에서 사용된 프로젝트명은 "TestApp_Swift", 워크플로우 이름은 "Default"입니다. App Store Connect에 접속해서 해당 프로젝트의 "Xcode Cloud" 페이지로 이동합니다.



워크플로우 이름을 클릭하여 설정 화면으로 이동합니다.

설정 화면이 표시되면 다른 항목은 그대로 두고 페이지를 맨 아래로 스크롤 시킨 후 "Actions – Deployment Preparation" 항목의 값을 "None"으로 선택하고 "Post-Actions" 항목은 아무 액션도 들어가지 않도록 비워 둡니다.



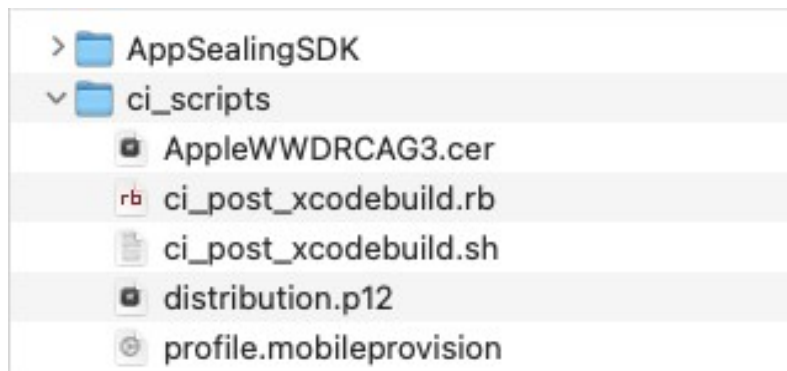
Deployment Preparation 항목의 값을 "None"으로 선택하지 않으면 빌드된 IPA가 재서명 과정을 거치지 않고 그대로 App Store Connect로 업로드 되기 때문에 앱이 정상적으로 실행되지 않습니다. Post-Actions도 마찬가지로 이 두 항목은 반드시 비워 두거나 "None"으로 설정해야 합니다.

2-2 ci_scripts 준비 사항

Xcode Cloud 환경에서 빌드된 IPA에 보안 설정 파일을 추가하고 재서명 한 뒤, 수정된 IPA를 App Store Connect에 업로드 하기 위해서는 커스텀 스크립트 파일과 서명용 인증서 및 개인키, 배포용 provisioning profile이 추가로 필요합니다.

이 파일들은 모두 프로젝트 폴더의 ci_scripts 폴더 안에 포함되어야 합니다. 이를 위해 먼저 프로젝트 폴더에 ci_scripts 폴더를 생성하고 AppSealing SDK에서 제공하는 "ci_scripts/Xcode Cloud" 폴더 안에 있는 세 개의 파일을 복사합니다. 여기에 더해 다음의 항목들이 추가로 필요합니다.

- App Store 배포용 인증서 : distribution.p12 (개인키 포함, PKCS#12 형식)
- App Store 배포용 프로비저닝 프로파일 : profile.mobileprovision
- Apple ID 및 App-Specific password (스크립트에 기록됨)



AppSealing SDK의 ci_scripts 구성에는 애플 루트 인증서인 "AppleWWDRCA3.cer" 인증서 파일과 "ci_post_xcodebuild.sh", "ci_post_xcodebuild.rb" 스크립트 파일이 기본으로 포함되어 있는데 이 파일들도 이름을 변경하면 안 됩니다.

이 기본 파일에 앞에서 언급된 파일(distribution.p12, profile.mobileprovision) 이 추가되어 위 그림과 같이 모두 다섯 개의 파일이 포함되게 됩니다.

App Store 배포용 인증서

ci_scripts 폴더에 반드시 배포용 인증서가 포함되어야 합니다. 이 인증서는 맥 키체인에서 PKCS#12 형식으로 내보내기 한 P12 파일이어야 하고 파일명은 "distribution.p12"로 고정되어 있어야 합니다. P12 파일을 생성할 때 비밀번호는 빈 문자열로 설정해야 하며 만약 비밀번호를 변경한 경우 ci_post_xcodebuild.rb 파일에서 사용하는 비밀번호를 직접 수정해 주어야 합니다. App Store 배포용 인증서가 아닌 인증서를 추가하거나 파일 이름이 다르게 되어 있을 경우 재서명이 정상적으로 이루어지지 않습니다.

인증서를 P12 파일로 내보내는 방법은 7-1에 자세히 설명되어 있습니다.

만약 P12 파일에 패스워드를 지정할 경우는 ci_post_xcodebuild.sh 파일을 수정해 주어야 합니다. 에디터로 파일을 열고 445 라인으로 이동합니다. 이 줄의 명령어에 개인키 파일의 패스워드가 공백으로 되어 있는 것을 볼 수 있습니다.

```
441 system( 'security create-keychain -p 0000 ' + APPSEALING_KEYCHAIN )
442 system( 'security list-keychains -d user -s login.keychain ' + APPSEALING_KEYCHAIN )
443 system( 'security import ./AppleWDRCA3.cer -k ' + APPSEALING_KEYCHAIN + ' -t cert -A -P "" ' )
444 system( 'security import ./distribution.cer -k ' + APPSEALING_KEYCHAIN + ' -t cert -A -P "" ' )
445 system( 'security import ./private_key.p12 -k ' + APPSEALING_KEYCHAIN + ' -t priv -A -P "" ' )
446 system( 'security default-keychain -d user -s ' + APPSEALING_KEYCHAIN )
447 system( 'security unlock-keychain -p 0000 ' + APPSEALING_KEYCHAIN )
448 system( 'security set-keychain-settings ' + APPSEALING_KEYCHAIN )
449 system( 'security set-key-partition-list -S apple-tool:,apple:,codesign: -s -k 0000 ' + APPSEALING_KEYCHAIN + ' > /dev/null ' )
```

이 위치에 지정한 패스워드를 기록하고 파일을 저장합니다. 만약 패스워드를 "your_password"라는 문자열로 저장했다면 아래와 같이 수정을 해야 합니다.

```
441 system( 'security create-keychain -p 0000 ' + APPSEALING_KEYCHAIN )
442 system( 'security list-keychains -d user -s login.keychain ' + APPSEALING_KEYCHAIN )
443 system( 'security import ./AppleWDRCA3.cer -k ' + APPSEALING_KEYCHAIN + ' -t cert -A -P "" ' )
444 system( 'security import ./distribution.cer -k ' + APPSEALING_KEYCHAIN + ' -t cert -A -P "" ' )
445 system( 'security import ./private_key.p12 -k ' + APPSEALING_KEYCHAIN + ' -t priv -A -P "your_password" ' )
446 system( 'security default-keychain -d user -s ' + APPSEALING_KEYCHAIN )
447 system( 'security unlock-keychain -p 0000 ' + APPSEALING_KEYCHAIN )
448 system( 'security set-keychain-settings ' + APPSEALING_KEYCHAIN )
449 system( 'security set-key-partition-list -S apple-tool:,apple:,codesign: -s -k 0000 ' + APPSEALING_KEYCHAIN + ' > /dev/null ' )
```

App Store 배포용 프로비저닝 프로파일

ci_scripts 폴더에 App store 배포를 위한 프로비저닝 프로파일이 포함되어야 합니다. 이 파일은 Apple Developer 사이트에서 다운로드 한 프로파일을 "profile.mobileprovision" 이라는 이름으로 포함시켜야 합니다. 파일명이 맞지 않거나 올바른 프로파일이 아닐 경우 재서명에 실패하거나 앱이 정상적으로 설치/실행이 되지 않을 수 있습니다.

빌드 후처리 과정용 커스텀 스크립트

ci_scripts 폴더에는 기본으로 ci_post_xcodebuild.sh 스크립트 파일이 포함되어 있습니다. 스크립트의 내용 중 재서명한 IPA를 App Store Connect에 업로드 하기 위한 과정이 포함되어 있는데 이 동작을 수행하기 위해서 앱 개발자의 Apple 계정과 앱 전용 비밀번호가 필요합니다. (Apple 계정 패스워드가 아닙니다!) 앱 전용 비밀번호가 없을 경우 7-2의 내용을 참고하여 생성합니다.

Apple 계정과 앱 전용 비밀번호를 ci_post_xcodebuild.sh 파일의 20~21 라인위치에 기록하고 파일을 저장합니다. 만약 Apple 계정이 "myappleid@company.com" 이고 발급받은 앱 전용 비밀번호가 "aaaa-bbbb-cccc-dddd" 라면 스크립트 파일은 다음과 같이 변경되어야 합니다.

```

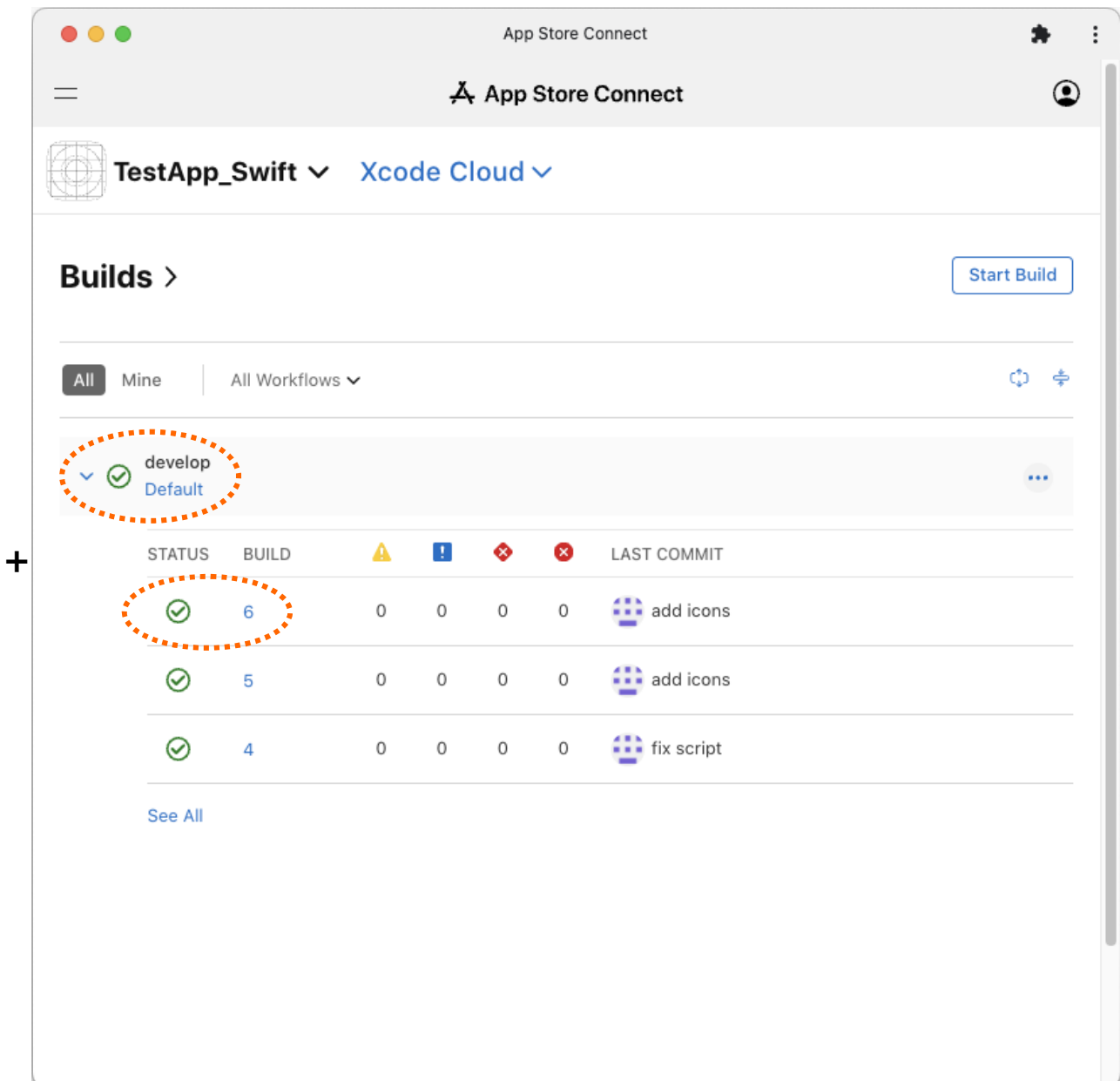
1  #!/usr/bin/env ruby
2
3  =begin
4  =====
5  |  AppSealing iOS SDK Hash Generator V1.2.2
6  |
7  |  * presented by Inka Entworks
8  |
9  =====
10 =end
11
12 require 'pathname'
13 require 'tmpdir'
14 require 'securerandom'
15 require 'net/https'
16 require 'json'
17 require 'io/console'
18 require 'open-uri'
19
20 #-----
21 APPLE_ID = "support@inka.co.kr" # replace with your apple developer ID
22 APPLE_APP_PASSWORD = "aaaa-bbbb-cccc-dddd" # replace with your apple application password (https://appleid.apple.com)
23 # NOT ACCOUNT PASSWORD !
24 #-----

```

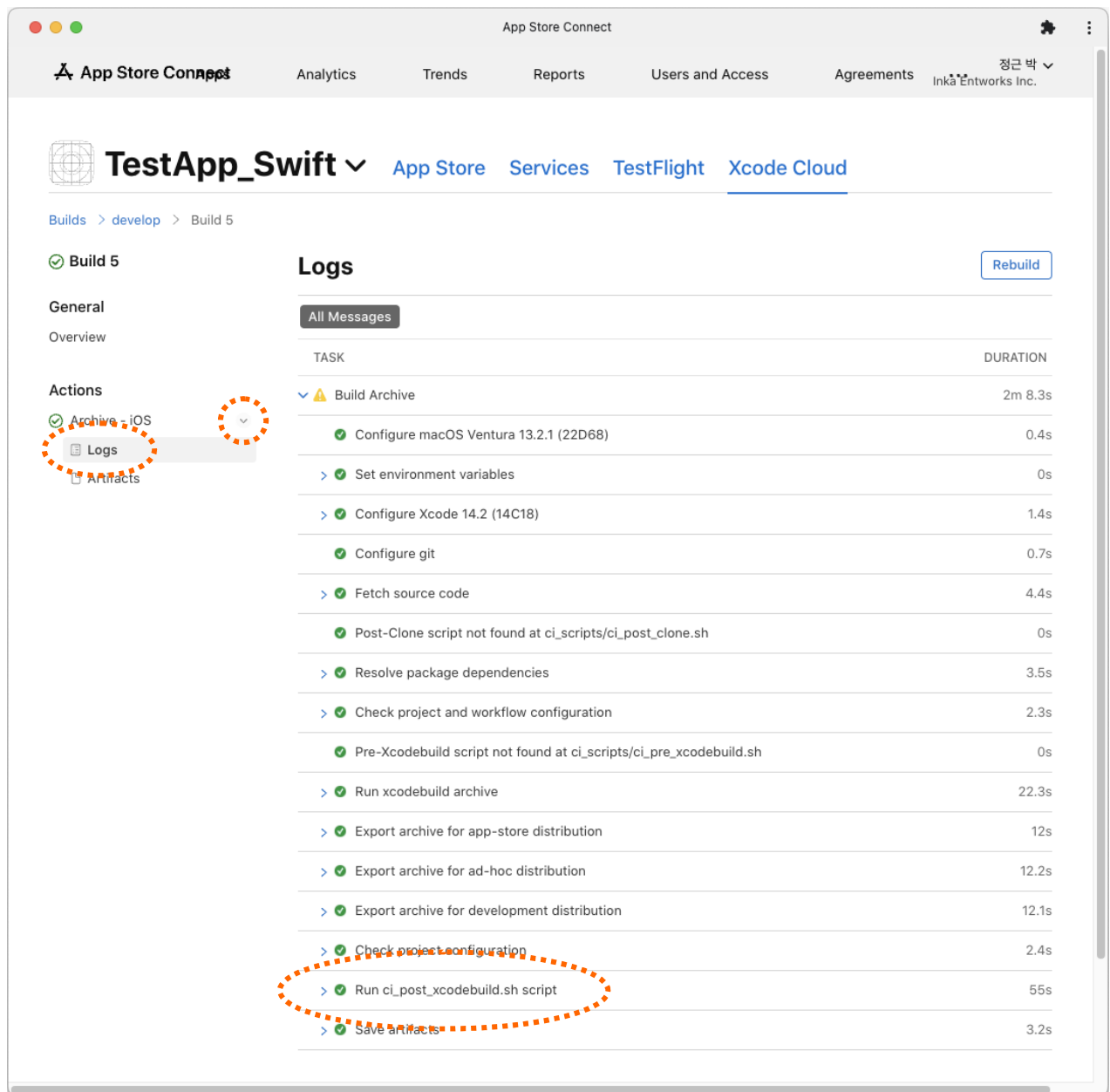

2-3 빌드 및 업로드 과정 확인

Xcode Cloud 환경 설정 및 ci_scripts 폴더에 필요한 파일 준비가 모두 완료되면 ci_scripts에 추가된 파일들을 git 저장소에 푸시합니다.

이제 App Store Connect에 접속하여 타겟 브랜치(이 문서에서는 develop)의 빌드가 완료된 것을 확인합니다.

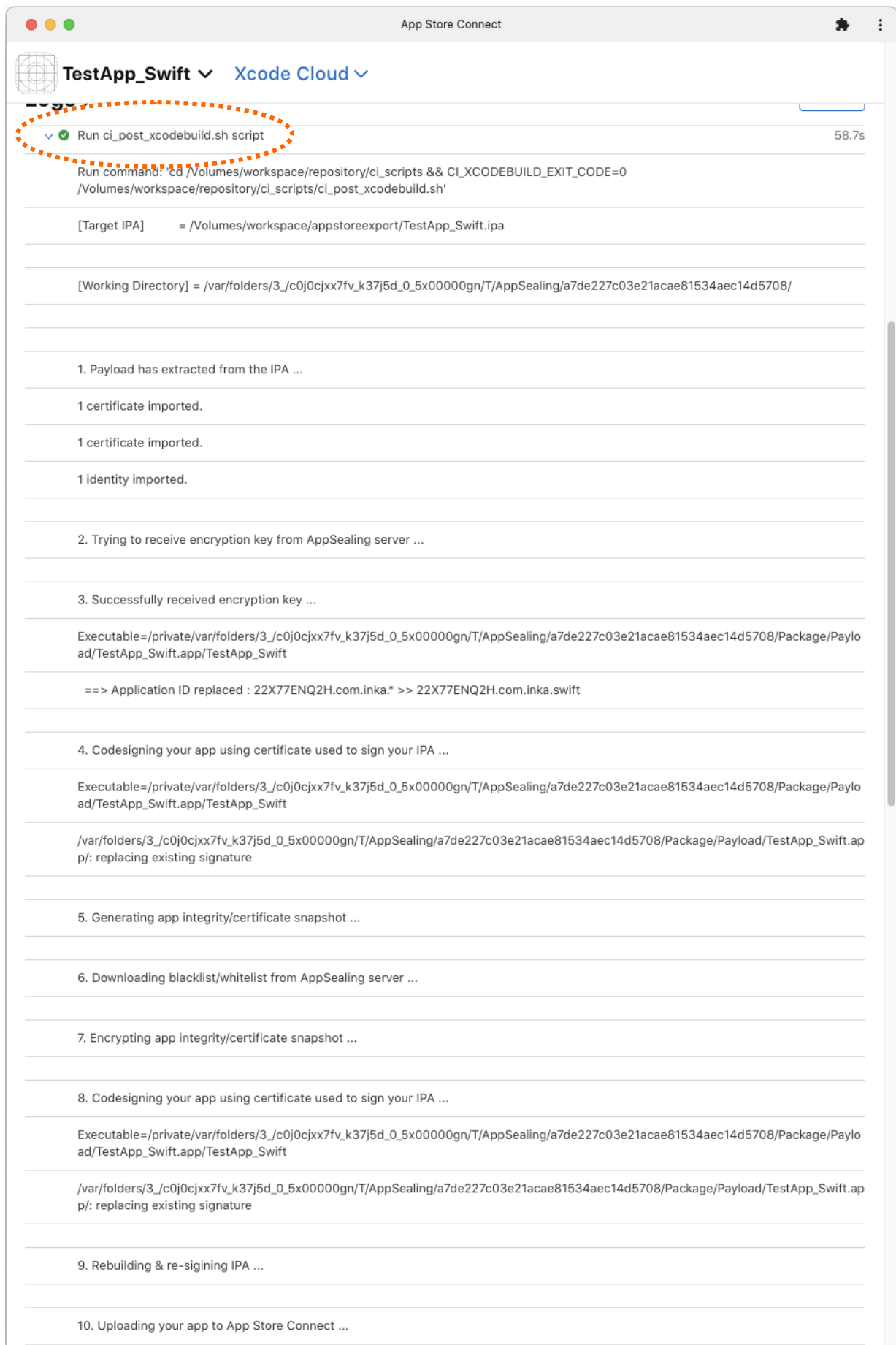


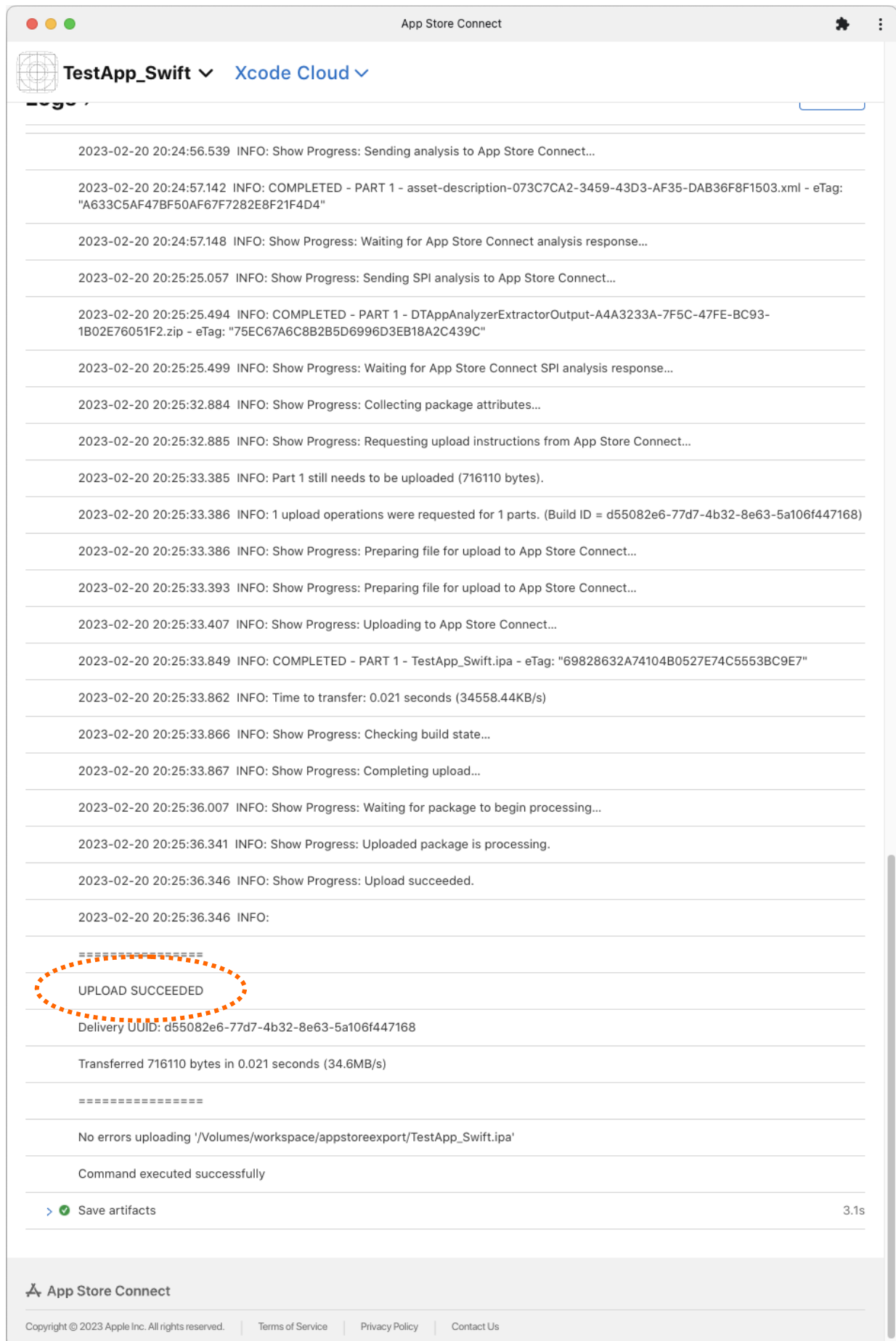
빌드 번호를 클릭하여 빌드 Overview 화면으로 이동합니다. 좌측의 "Actions" 아래 있는 "Archive – iOS" 항목의 우측 화살표를 클릭하여 메뉴를 확장하고 로그를 확인합니다.



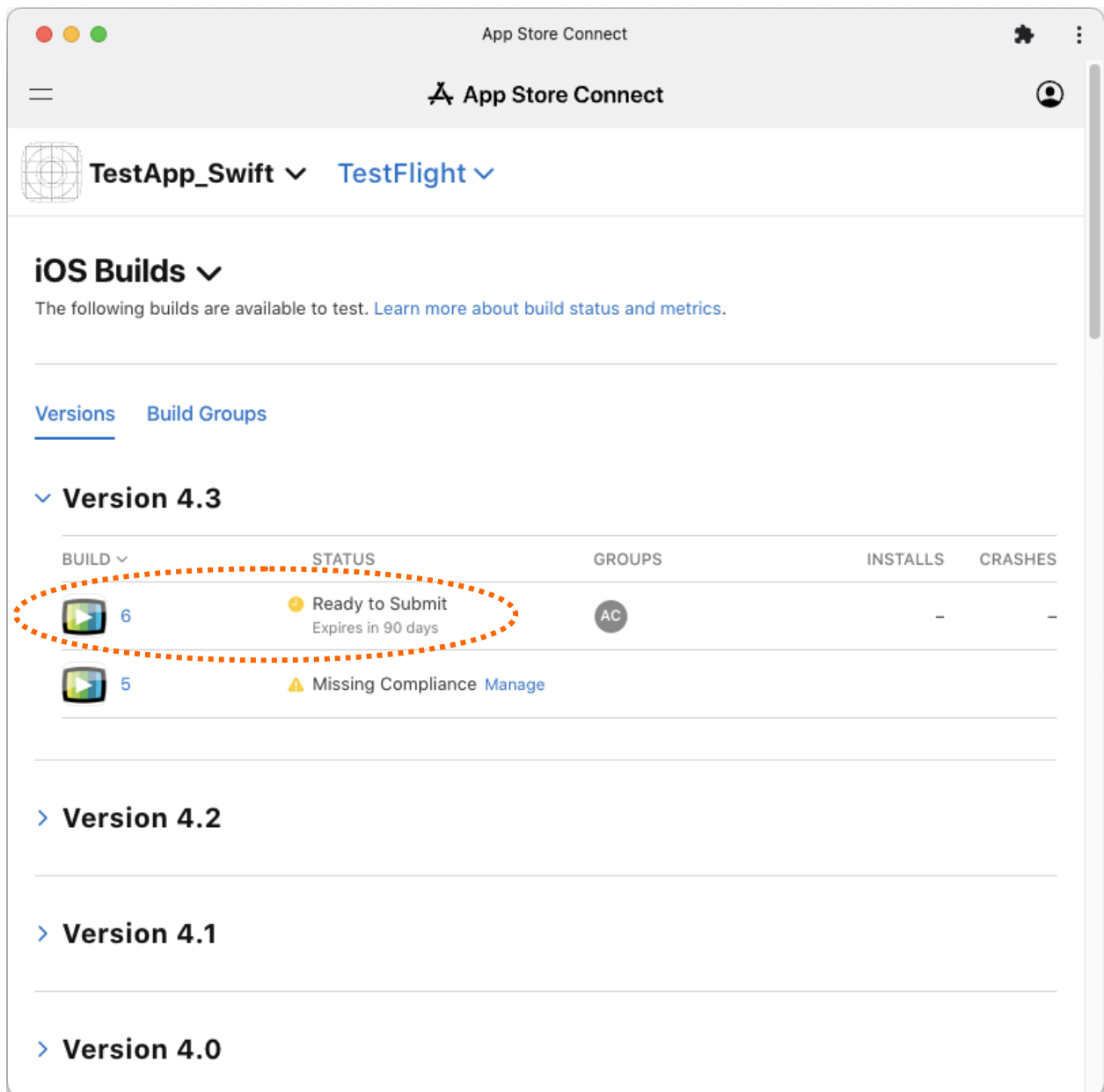
위 화면처럼 "Build Archive" 아래의 작업들이 모두 정상적으로 수행되어야 합니다. 특히 "Run ci_post_xcodebuild.sh script"와 "Save artifacts" 이후에 다른 작업이 더 수행되어서는 안 됩니다. 만약 다른 작업의 진행 로그가 있다면 7-1 섹션의 내용을 참고하여 워크플로우 설정을 수정한 후 다시 빌드를 진행해야 합니다.

이제 "Run ci_post_xcodebuild.sh script" 항목의 왼쪽 꺾쇠(>)를 클릭하여 스크립트 로그를 펼쳐 봅니다. Exported 된 IPA에 대해 ci_post_xcodebuild.sh 스크립트가 실행된 로그를 확인할 수 있습니다. 로그를 맨 아래쪽으로 스크롤 하여 IPA가 정상적으로 업로드 된 내용을 확인합니다.





이제 TestFlight로 이동해 보면 방금 빌드 된 버전(6)이 등록되어 있는 것을 확인할 수 있습니다.



Part 3. Fastlane 프로젝트에 적용

3-1 프로젝트 Fastfile 설정

AppSealing SDK는 Fastlane을 적용한 프로젝트에도 사용할 수 있습니다. 일반적으로 Fastlane 툴을 Xcode 프로젝트에 사용하는 이유는 앱 빌드 및 패키징 과정을 자동화하고 이후의 배포 과정도 자동화 하기 위한 것입니다. Xcode 프로젝트에 AppSealing SDK를 적용한 경우 앱 빌드 이후에 추가로 진행해야 되는 과정들이 있지만 Fastlane이 적용된 경우 Fastfile 스크립트에 AppSealing 적용 과정을 추가함으로써 기존의 자동화 과정을 그대로 유지할 수 있습니다.

이 장에서는 Fastlane의 빌드 및 배포 자동화를 그대로 유지하기 위해 스크립트를 어떻게 수정하고 반영해야 하는지를 설명합니다.

다음은 TestApp_Swift라는 프로젝트에 Fastlane을 적용한 상태의 Fastfile 코드입니다. (주석 제외). 앱을 빌드하고 서명해서 Testflight로 업로드하는 과정이 포함되어 있습니다. Apple Store에 업로드 하는 스크립트도 동일한 구조를 가지므로 Testflight에 대해서만 설명합니다.

프로젝트 기본 Fastfile

```
default_platform(:ios)

platform :ios do
  desc "Push a new beta build to TestFlight"
  lane :beta do
    increment_build_number(xcodeproj: "TestApp_Swift.xcodeproj")
    build_app(scheme: "TestApp_Swift")
    upload_to_testflight
  end
end
```

일반적인 경우는 이 파일에 크게 수정을 하지 않은 상태로 "fastlane beta" 명령어를 실행해서 빌드와 배포를 하게 됩니다. 여기에 AppSealing SDK를 적용한 경우는 빌드와 배포 사이에 추가 작업이 필요하기 때문에 fastfile 스크립트를 아래와 같이 수정해야 합니다. 원본 스크립트는 프로젝트 이름과 파일명이 문자열 값으로 직접 입력되어 있지만 수정할 스크립트는 모든 프로젝트에 적용할 수 있도록 직접 값을 지정하는 대신 프로젝트 폴더에서 필요한 값을 추출해서 사용하도록 되어 있습니다. 따라서 프로젝트 명이 TestApp_Swift가

아니더라도 다른 프로젝트에 모두 동일한 스크립트를 적용할 수 있습니다.

Fastfile 을 문서 편집기로 열고 모든 코드를 SDK 의 "Fastlane Scripts/Fastfile" 파일에 들어 있는 코드로 대체합니다. `FASTLANE_USER`, `FASTLANE_APPLE_APPLICATION_SPECIFIC_PASSWORD`, `PROFILE` 값은 실제 사용되는 값으로 바뀌어서 적용해야 합니다. `FASTLANE_USER` 은 Apple 계정, `FASTLANE_APPLE_APPLICATION_SPECIFIC_PASSWORD` 은 앱 전용 비밀번호를 입력해야 합니다. 앱 전용 비밀번호 발급 방법에 대해서는 64 페이지의 내용을 참고하면 됩니다.

`PROFILE` 값은 배포 단계에 사용하는 프로비저닝 프로파일의 이름을 입력해야 합니다.

코드를 수정하고 기존과 동일하게 "fastlane beta" 명령을 실행하면 앱을 빌드하고 IPA 로 export 한 후 generate_hash 스크립트를 자동으로 실행하고 AppSealing 이 적용된 IPA 를 testflight 로 업로드 하게 됩니다.

3-2 Github Action과 Fastlane 동시 적용

Xcode 프로젝트 코드를 Github에 올려 두고 Github Action을 사용하여 코드 푸시가 이루어진 경우 자동으로 빌드와 배포가 이루어지도록 배포 파이프라인을 구축할 수 있습니다. 이 경우 Action 정의를 위해 아래와 같은 yaml 형식의 빌드 스크립트 파일을 추가하게 됩니다

이 파일에는 소스코드 브랜치 이름이 develop으로 정의되어 있고 프로젝트에 포함된 fastfile을 실행하기 위해 Fastlane 설치 단계가 포함되어 있습니다. 또한 fastfile 실행에 필요한 환경 변수를 설정하기 위해 `APP_STORE_CONNECT_KEY_ID`, `APP_STORE_CONNECT_ISSUER_ID`, `APP_STORE_CONNECT_PRIVATE_KEY`값을 env에서 지정하고 있습니다. 로컬 프로젝트에 Fastlane을 적용하는 단계 혹은 Xcode Cloud를 사용하는 과정에서는 앱 전용 비밀번호를 사용 했으나 이 단계에서는 App Store Connect API를 사용합니다.

App Store Connect API를 사용하기 위해서는 Apple account 페이지에서 API Key를 발급 받아야 하며 여기서 발급된 개인키(`APP_STORE_CONNECT_PRIVATE_KEY`)와 키 ID(`APP_STORE_CONNE`

`CT_KEY_ID`), 발급자 ID(`APP_STORE_CONNECT_ISSUER_ID`)를 fastfile 스크립트로 전달해 주어야 합니다. 전달하는 실제 값들은 스크립트에 직접 입력하지 않고 Github Action의 secrets 변수로 저장한 다음 해당 변수를 참조해서 전달하는 방식으로 작성되어 있습니다.

프로젝트 기본 Github Action 빌드 스크립트 (ios_build.yml)

```
name: iOS Build & Deploy
on:
  push:
    branches:
      - develop

jobs:
  release-ios:
    name: Build and release iOS app
    runs-on: macos-latest
    steps:
      - uses: actions/checkout@v2

      - uses: actions/setup-ruby@v1
        with:
          ruby-version: '3.1.2'

      - name: Install Fastlane
        run: cd ios && bundle install

      - name: Install pods
        run: cd ios && pod install

      - name: Execute Fastlane
        env:
          APP_STORE_CONNECT_KEY_ID: ${ secrets.KEY_ID }
          APP_STORE_CONNECT_API_KEY: ${ secrets.API_KEY }
          APP_STORE_CONNECT_PRIVATE_KEY: ${ secrets.PRIVATE_KEY }
        run: cd ios && fastlane release
```

아래 코드는 이 환경 변수들을 넘겨 받아 실제 fastlane 작업을 수행하게 될 Fastfile 코드입니다. 여기에는 App Store Connect API를 사용하기 위한 설정 단계 및 앱 빌드, 업로드를 위한 과정이 포함되어 있습니다.

프로젝트 기본 Fastfile

```
default_platform(:ios)

platform :ios do
  desc "Release to App Store"
  lane :release do
    # 앱 스토어 커넥트 API 인증
    app_store_connect_api_key(
      key_id: ENV["APP_STORE_CONNECT_KEY_ID"],
```



```
    issuer_id: ENV["APP_STORE_CONNECT_ISSUER_ID"],
    key_content: ENV["APP_STORE_CONNECT_PRIVATE_KEY"]
  )

  # 앱 빌드
  build_app(
    scheme: "프로젝트스킴명",
    export_method: "app-store"
  )

  # 업로드
  upload_to_testflight(
    skip_waiting_for_build_processing: true,
    distribute_external: true
  )
end
end
```

기존에 App Store Connect API Key가 발급되어 있지 않은 상태라면 다음 링크의 내용을 참조하여 키를 발급받은 후 아래 과정을 진행합니다.

(<https://developer.apple.com/documentation/appstoreconnectapi/creating-api-keys-for-app-store-connect-api>)

이 상태에서 AppSealing SDK를 적용할 경우의 변경 사항에 대해 설명하겠습니다. AppSealing SDK를 적용할 경우 archive와 export 단계를 거쳐 생성된 IPA에 대해 스크립트 작업을 수행해야 하고 이 과정에서 IPA에 대한 재서명이 이루어지기 때문에 추가로 서명용 인증서와 프로비저닝 프로파일이 필요합니다. 인증서는 스토어 업로드를 위한 배포용 인증서여야 하고 프로파일 역시 배포용 프로파일이 있어야 합니다. 따라서 프로젝트에 인증서를 PKCS#12 형식으로 변환하여 포함시키고 프로비저닝 프로파일을 포함시킨 상태로 github에 푸시를 해야 합니다. 이 때 인증서의 파일명은 "certificate.p12"가 되어야 하고 프로비저닝 프로파일의 파일명은 "distribution.mobileprovision"이 되어야 합니다. 파일명을 다르게 할 경우 아래 스크립트에서 파일명을 그에 맞게 수정해야 합니다. 프로비저닝 프로파일의 이름은 **PROFILE** 환경 변수에 직접 값을 입력하여 수정합니다.

인증서를 PKCS#12 형식으로 저장할 때 비밀번호는 "123456"으로 설정한 것으로 간주합니다. 만약 비밀번호가 다르다면 아래 스크립트에서 역시 비밀번호 문자열을 올바르게 수정해야 합니다.

Github에 업로드 된 인증서를 키체인에 설치하고 프로비저닝 프로파일을 설치하기 위해 ios_build.yml 스크립트를 SDK의 "Fastlane Scripts/Fastfile_GithubAction" 파일의 내용으로 수정합니다.

이 스크립트에는 "runs-on: macos-15"와 "xcode-version: '16.0'" 항목이 포함되어 있는데 이는 프로비저닝 프로파일 저장 경로가 Xcode 16 버전부터 변경되었기 때문입니다. Xcode 버전을 16으로 지정하지 않으면 프로파일이 잘못된 경로로 복사되고 이로 인해 IPA 후처리 단계에서 재서명에 실패하게 됩니다.

이제 앱 빌드 및 재서명, 업로드를 위한 작업에 AppSealing SDK 관련 작업이 반영되도록 Fastfile도 SDK의 "Fastlane Scripts/Fastfile_GithubAction" 파일의 내용으로 수정합니다.

이 스크립트는 App Store Connect의 앱 정보를 가져와 자동으로 빌드 번호를 증가시킨 후 앱을 빌드하고 IPA로 내보낸 다음 generate_hash 스크립트를 적용하고 재서명하여 App Store Connect로 업로드 하는 전 과정을 포함하고 있습니다.

이 과정에 필요한 프로젝트 이름과 스킴, 타겟, 번들 ID 등의 모든 값은 프로젝트 파일에서 자동으로 추출하여 사용하도록 되어 있기 때문에 TestApp_Swift가 아닌 다른 프로젝트에 적용할 때도 동일한 스크립트를 그대로 사용할 수 있습니다.

Part 4. Azure 파이프라인 적용

이 장에서는 Azure CI/CD 툴에서 관리되는 프로젝트에 AppSealing SDK를 적용하여 빌드하기 위한 방법을 소개합니다. 적용 가능한 프로젝트의 종류는 Xcode 기본 프로젝트(Swift / Objective-C)와 Flutter 프로젝트, Ionic, Cordova, React Native 프로젝트 입니다. 모든 플랫폼의 프로젝트에 대해 동일한 준비 과정이 필요하며 빌드 스크립트는 각 플랫폼마다 다른 스크립트를 사용합니다.

4-1 인증서 및 프로파일 준비

AppSealing SDK를 적용한 프로젝트의 경우 IPA가 export 되고 난 후 generate_hash 스크립트를 실행하는 단계에서 재서명 과정을 거칩니다. 따라서 최초 IPA 서명에 사용된 배포용 인증서와 개인키, 배포용 프로비저닝 프로파일이 필요하며 이 파일들을 준비해서 소스 코드 저장소의 폴더에 추가해 주어야 합니다.

App Store 배포용 인증서

프로젝트 폴더에 반드시 배포용 인증서가 포함되어야 합니다. 이 인증서는 Apple Developer 사이트에서 다운로드 받은 배포용 인증서를 개인키와 함께 PKCS#12 형식으로 내보내기 한 P12 파일이어야 합니다. 파일명은 반드시 "distribution.p12"로 고정되어 있어야 하고 P12 파일을 생성할 때 설정한 비밀번호는 4-2의 Azure 프로젝트 설정 단계에서 variables 에 추가되어야 합니다. (4-2 참조)

App Store 배포용 프로비저닝 프로파일

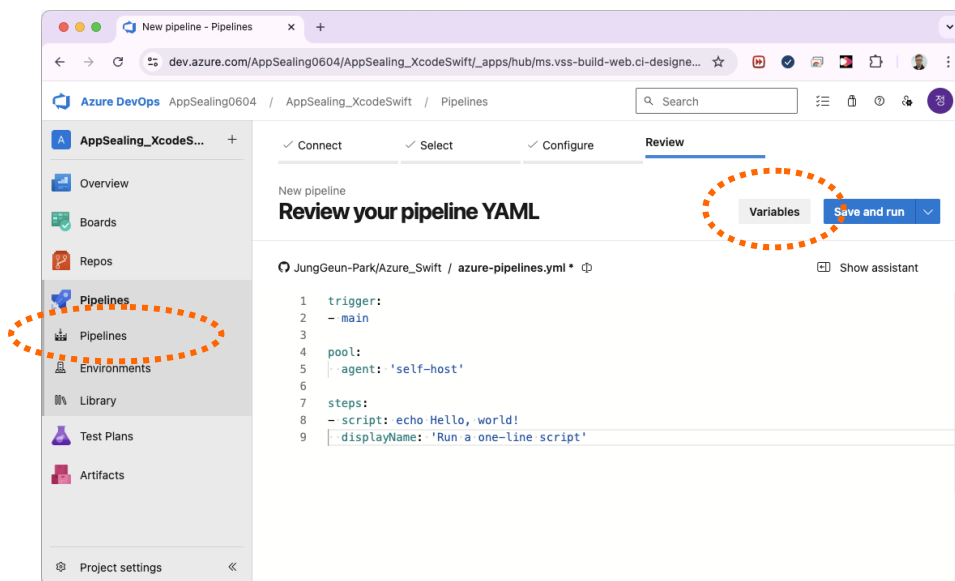
프로젝트 폴더에 App store 배포를 위한 프로비저닝 프로파일이 포함되어야 합니다. 이 파일은 Apple Developer 사이트에서 다운로드 한 프로파일을 "profile.mobileprovision"이라는 이름으로 포함시켜야 합니다. 파일명이 맞지 않거나 올바른 프로파일이 아닐 경우 재서명에 실패하거나 앱이 정상적으로 설치/실행이 되지 않을 수 있습니다.

인증서와 프로파일을 프로젝트에 추가하고 소스코드 저장소에 푸시하기 전에 다음 단계를 먼저 수행하십시오. 최종 단계까지 모두 완료한 후 저장소에 푸시해야 Azure 파이프라인에서 정상적으로 빌드가 수행됩니다.

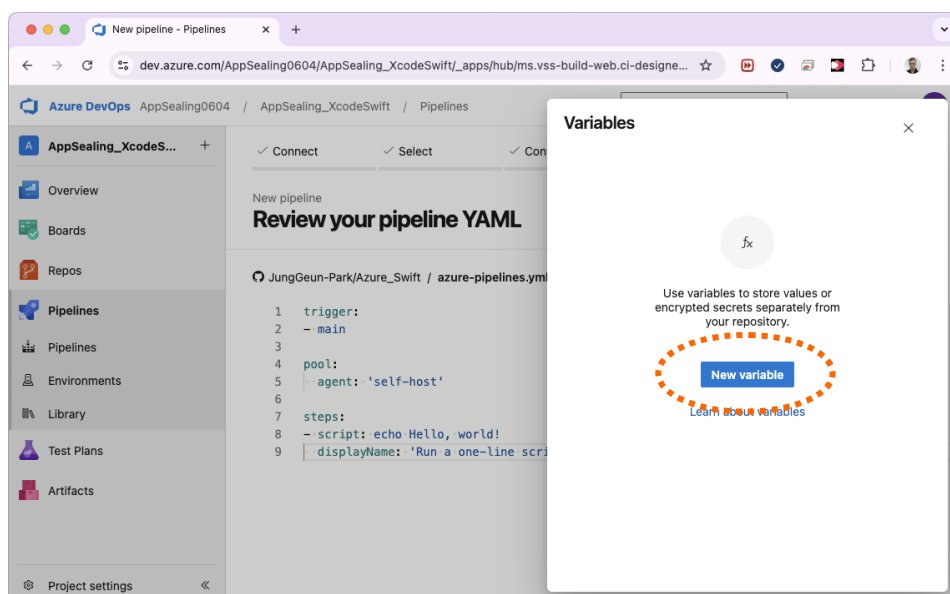
4-2 Azure 프로젝트 설정

이 단계에서는 Azure 프로젝트와 관련된 설정을 진행합니다. 프로젝트의 플랫폼과 무관하게 모든 프로젝트에 동일한 과정을 수행하면 됩니다.

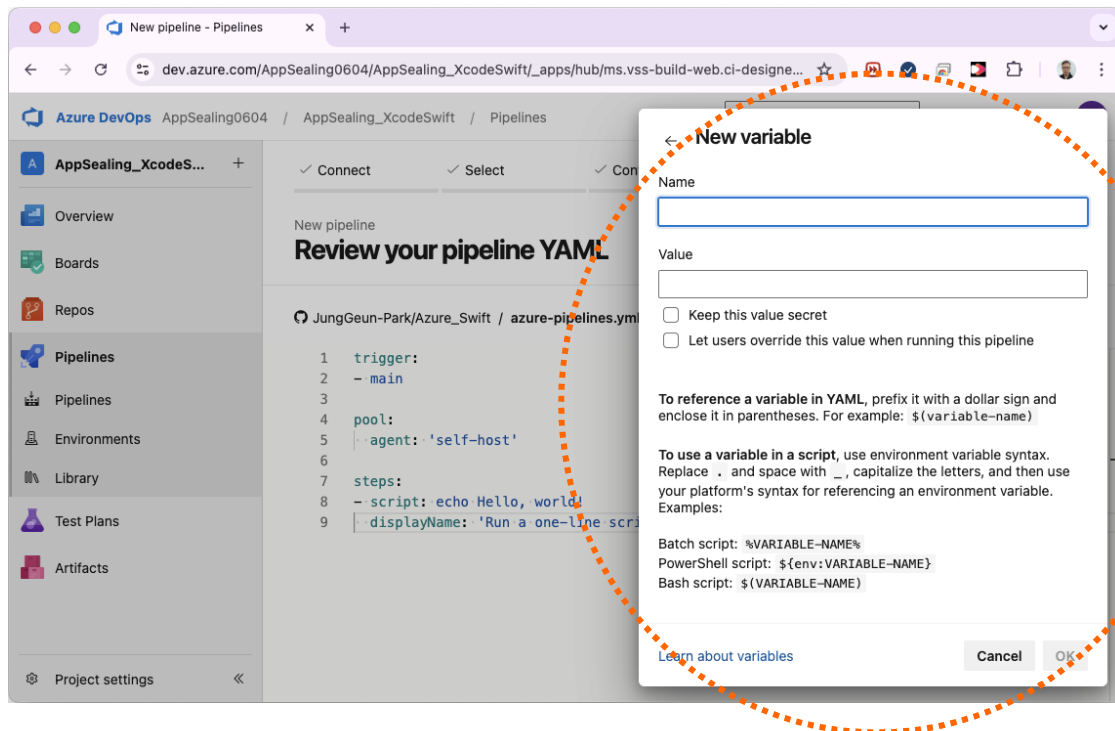
먼저 Azure DevOps 페이지에서 프로젝트 화면으로 이동한 뒤 현재 활성화 되어 있는 Pipelines을 선택하고 우측 상단의 "Variables" 버튼을 클릭합니다.



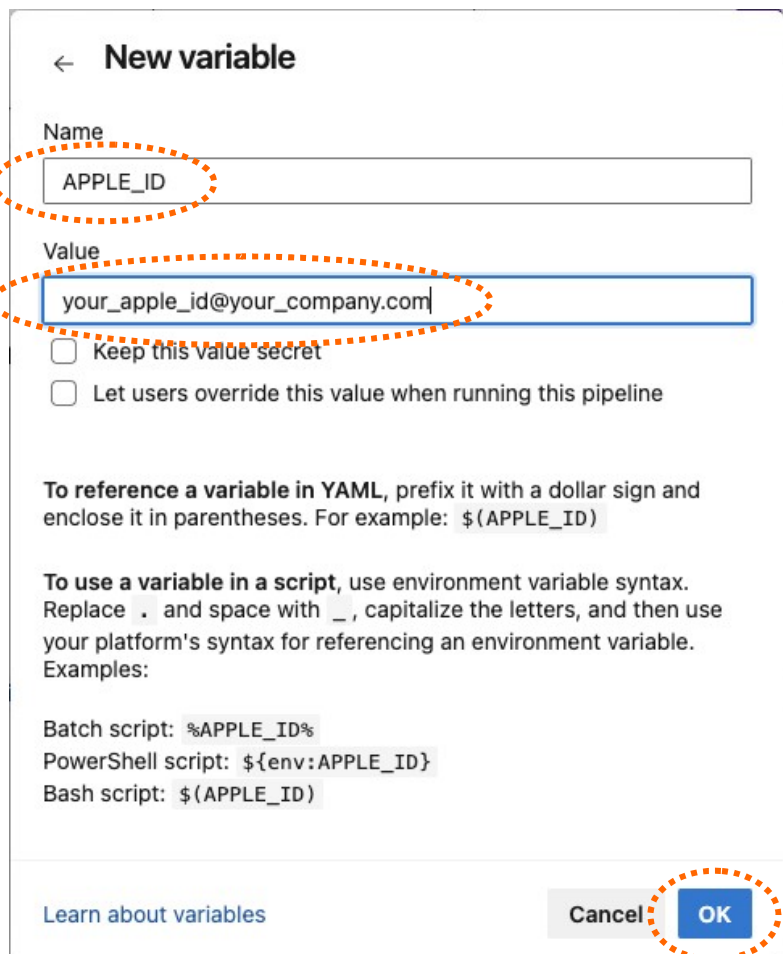
화면과 같이 Variables 편집 창이 나타납니다. 이미 등록하여 사용 중인 변수가 있을 경우 목록에 표시될 수 있습니다.



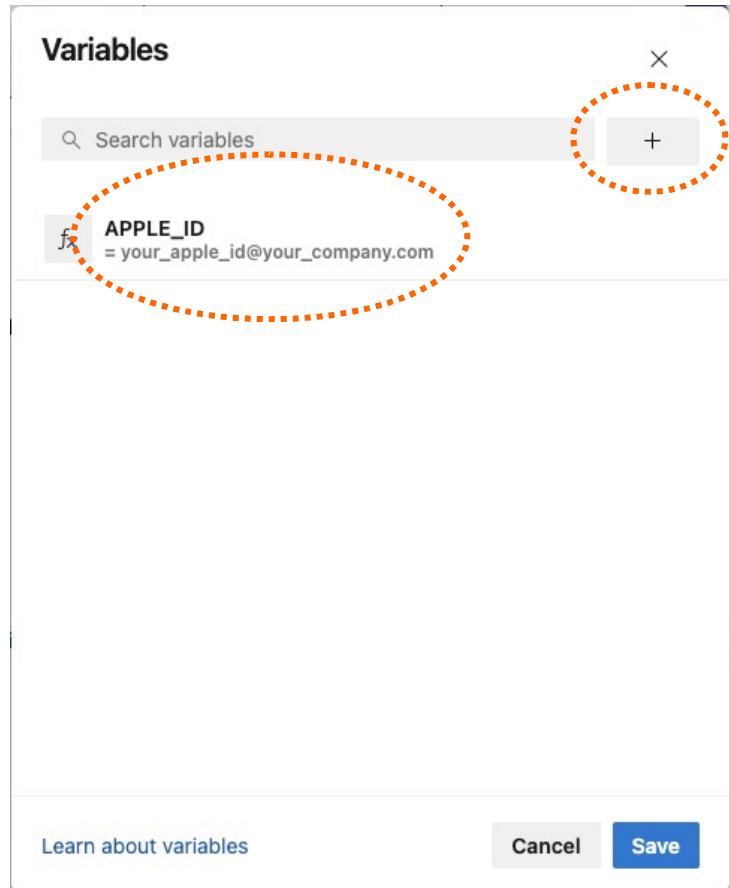
이제 Variables 창에서 "New Variable" 버튼을 클릭하여 새로운 변수를 추가합니다.



먼저 애플 ID를 생성합니다.
"Name" 칸에 "APPLE_ID" 문자열을 입력하고 "Value" 칸에는 실제 사용 중인 애플 계정의 ID 값을 입력합니다. 입력 후에는 "OK" 버튼을 클릭합니다.



오른쪽 화면과 같이 "APPLE_ID" 변수가 등록된 것을 확인할 수 있습니다. 이제 새로운 변수를 추가하기 위해 다시 "+" 버튼을 클릭합니다.



변수 등록 창이 나타나면 이번에는 앱 전용 비밀번호 항목을 등록합니다. "Name" 칸에 "APP_SPECIFIC_PASSWORD" 문자열을 입력하고 "Value" 칸에 실제 사용 중인 앱 전용 비밀번호 값을 입력합니다. 만약 앱 전용 비밀번호가 없다면 다음 페이지의 내용을 참고하여 새로 발급받아 사용하도록 합니다.

앱 전용 비밀번호와 같이 민감한 정보를 변수로 등록하여 사용할 경우 이 값이 보이지 않도록 숨김 처리할 수 있습니다.

우측 화면과 같이 “Keep this value secret” 항목을 체크하면 값이 숨겨지고 이후 값을 수정하거나 지울 수는 있지만 내용을 확인할 수는 없게 됩니다.

APP_SPECIFIC_PASSWORD와 이후 추가할 CERTIFICATE_PASSWORD의 값은 모두 숨김 처리하는 것이 권장됩니다.

New variable

Name
APP_SPECIFIC_PASSWORD

Value

☒ Keep this value secret
☐ Let users override this value when running this pipeline

To reference a variable in YAML, prefix it with a dollar sign and enclose it in parentheses. For example:
\$(APP_SPECIFIC_PASSWORD)

To use a variable in a script, use environment variable syntax. Replace . and space with _, capitalize the letters, and then use your platform's syntax for referencing an environment variable. Examples:
Batch script: %APP_SPECIFIC_PASSWORD%
PowerShell script: \${env:APP_SPECIFIC_PASSWORD}
Bash script: \$(APP_SPECIFIC_PASSWORD)

[Learn about variables](#) Cancel OK

위와 같은 방식으로 TEAM_ID와 PROVISIONING_PROFILE_NAME, CERTIFICATE_PASSWORD 세 개의 변수를 더 추가합니다.

PROVISIONING_PROFILE_NAME 값에는 앱 배포에 사용되는 프로비저닝 프로파일의 이름을 명시합니다.

모든 변수가 입력되면 최종적으로 우측과 같은 화면이 표시되어야 합니다.

변수 이름이 잘못되거나 값이 잘못될 경우 파이프라인의 빌드 단계에서 실패하기 때문에 정확하게 입력해야 하고 입력 후 한번 더 확인을 거치도록 합니다.

Variables

Search variables +

☐ CERTIFICATE_PASSWORD

☒ PROVISIONING_PROFILE_NAME
= YourCompanyDistributionProfile

☒ TEAM_ID
= 22X77ENQ2H

☐ APP_SPECIFIC_PASSWORD

☒ APPLE_ID
= your_apple_id@your_company.com

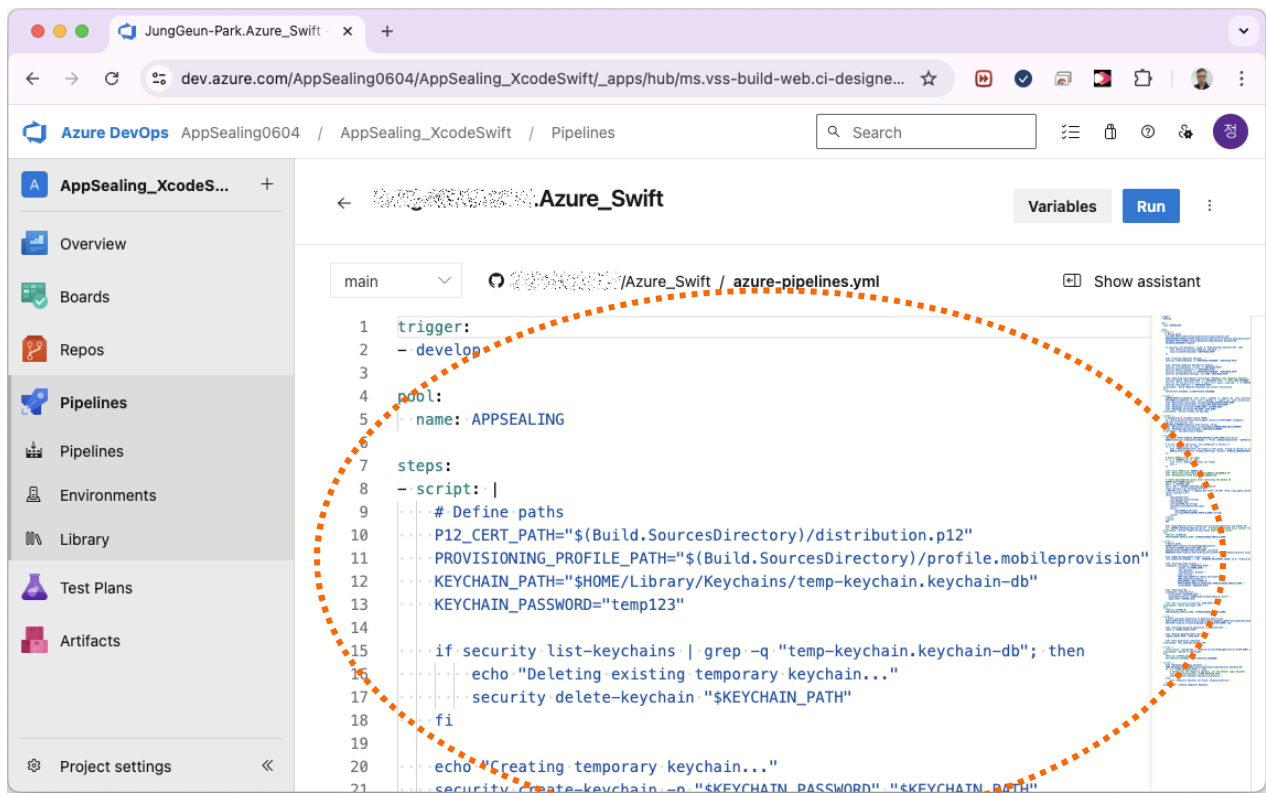
[Learn about variables](#) Cancel Save

4-3 Azure 빌드 스크립트 수정

이제 Azure 프로젝트의 Variables 설정은 모두 완료됐습니다.

다음 단계는 Azure의 빌드 스크립트인 YML 파일을 수정하는 것입니다.

아래 화면처럼 Azure의 파이프라인 편집 창에서 직접 스크립트를 수정해도 되고 소스 저장소에 수정된 스크립트를 푸시 해도 됩니다.



기존의 YML 스크립트 내용을 SDK의 "Azure Scripts/azure_swift.yml" 파일의 내용으로 대체하고 코드를 푸시합니다.

다만 이 코드 중 "trigger"에 해당하는 브랜치 이름과 파이프라인 빌드를 위한 에이전트 풀(pool) 이름은 실제 사용 중인 이름으로 바꾸어서 입력해야 합니다.

프로젝트가 Flutter나 Ionic, Cordova, React Native 플랫폼 기반의 프로젝트인 경우 이 스크립트 대신 SDK의 "Azure Scripts" 폴더에 포함된 각 플랫폼 별 스크립트 코드를 사용해야 합니다.

YML 스크립트를 수정하고 코드를 푸시하면 이제 AppSealing SDK가 적용된 상태로 아래 화면처럼 빌드가 진행이 됩니다.

The screenshot shows the Azure DevOps Pipelines interface for a build run. The browser address bar displays the URL: `dev.azure.com/AppSealing0604/AppSealing_XcodeSwift/_build/results?buildId=347&view=results`. The page title is "Pipelines - Run 20250424.1". The left sidebar contains navigation links: Overview, Boards, Repos, Pipelines (selected), Environments, Library, Test Plans, and Artifacts. The main content area shows the build summary for "#20250424.1 • Applied AppSealing SDK and script". The summary includes the repository and version (main, b236ea16), time started and elapsed (Just now), related work items (0), and artifacts (0). A "Jobs" table shows a single job named "Job" with a status of "Queued".

Azure DevOps / Pipelines / AppSealing_XcodeSwift / 20250424.1

#20250424.1 • Applied AppSealing SDK and script

Summary Code Coverage

Manually run by [User]

View change

Repository and version: main b236ea16

Time started and elapsed: Just now

Related: 0 work items, 0 artifacts

Tests and coverage: Get started

Name	Status	Duration
Job	Queued	

The screenshot shows the Azure DevOps Pipelines interface for a build run, displaying the logs. The browser address bar displays the URL: `dev.azure.com/AppSealing0604/AppSealing_XcodeSwift/_build/results?buildId=347&view=logs&j...`. The page title is "Pipelines - Run 20250424.1". The left sidebar contains navigation links: Overview, Boards, Repos, Pipelines (selected), Environments, Library, Test Plans, and Artifacts. The main content area shows the build logs for "Jobs in run #20250424.1". The logs include the following steps:

- Initialize job: <1s
- Checkout Ju...: 4s
- Setup Temp...: <1s
- Extract Sche...: 2s
- Increment B...: <1s
- Extract Bundl...: 1s
- Build and Ex...: 12s
- Run generate_h...
- Upload to TestF...
- Cleanup Tempo...

The "Build and Export IPA" step is expanded, showing the following logs:

```
43 { platform:iOS Simulator, id:133B2FA2-A855-450B-BCE8-D722939E9B5E, OS:18.0}
44 { platform:iOS Simulator, id:E868A41F-03AA-4388-A404-0466D4360127, OS:18.0}
45 { platform:iOS Simulator, id:1DF0324D-5408-4351-B1E9-F6D06825EB48, OS:18.0}
46 { platform:iOS Simulator, id:A6BF4FBC-9260-4A6A-AFE5-0A6625A0CFAB, OS:18.0}
47 { platform:iOS Simulator, id:007FD222-75D7-4FE9-BE4D-FB8D0F53D2FC, OS:18.0}
48 { platform:iOS Simulator, id:8AB0EF8B-72AD-4254-A549-96D56200F258, OS:18.0}
49 { platform:iOS Simulator, id:A61097E5-BD37-44FB-AFCF-3FBD0825EFFFFA, OS:18.0}
50 Prepare packages
51
52 note: Using codesigning identity override: iPhone Distribution
53 ComputeTargetDependencyGraph
54 note: Building targets in dependency order
55 note: Target dependency graph (1 target)
56   Target 'Azure_Swift' in project 'Azure_Swift' (no dependencies)
57
58 GatherProvisioningInputs
59
60 CreateBuildDescription
61
62 ExecuteExternalTool /Applications/Xcode.app/Contents/Developer/Toolchain:
63
64 ExecuteExternalTool /Applications/Xcode.app/Contents/Developer/Toolchain:
65
```

Part 5. Bitrise 파이프라인 적용

이 장에서는 Bitrise CI 툴에서 관리되는 프로젝트에 AppSealing SDK를 적용하여 빌드하기 위한 방법을 소개합니다. 적용 가능한 프로젝트의 종류는 Xcode 기본 프로젝트(Swift / Objective-C)와 Flutter 프로젝트, Ionic, Cordova, React Native 프로젝트 입니다. 모든 플랫폼의 프로젝트에 대해 동일한 준비 과정이 필요하며 빌드 스크립트는 각 플랫폼마다 다른 스크립트를 사용합니다.

5-1 인증서 및 프로파일 준비

AppSealing SDK를 적용한 프로젝트의 경우 IPA가 export 되고 난 후 generate_hash 스크립트를 실행하는 단계에서 재서명 과정을 거칩니다. 따라서 최초 IPA 서명에 사용된 배포용 인증서와 개인키, 배포용 프로비저닝 프로파일이 필요하며 이 파일들을 준비해서 소스 코드 저장소의 폴더에 추가해 주어야 합니다.

App Store 배포용 인증서

프로젝트 폴더에 반드시 배포용 인증서가 포함되어야 합니다. 이 인증서는 Apple Developer 사이트에서 다운로드 받은 배포용 인증서를 개인키와 함께 PKCS#12 형식으로 내보내기 한 P12 파일이어야 합니다. 파일명은 반드시 "distribution.p12"로 고정되어 있어야 하고 P12 파일을 생성할 때 설정한 비밀번호는 5-3의 Bitrise CI 프로젝트 설정 단계에서 variables 에 추가되어야 합니다.

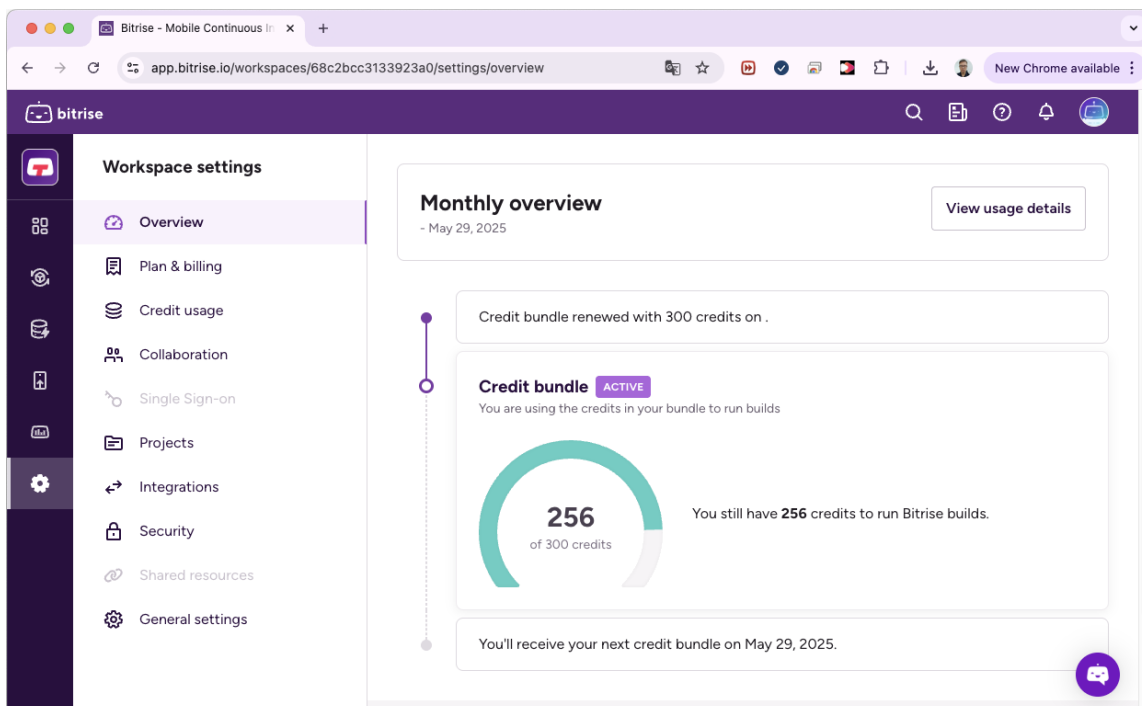
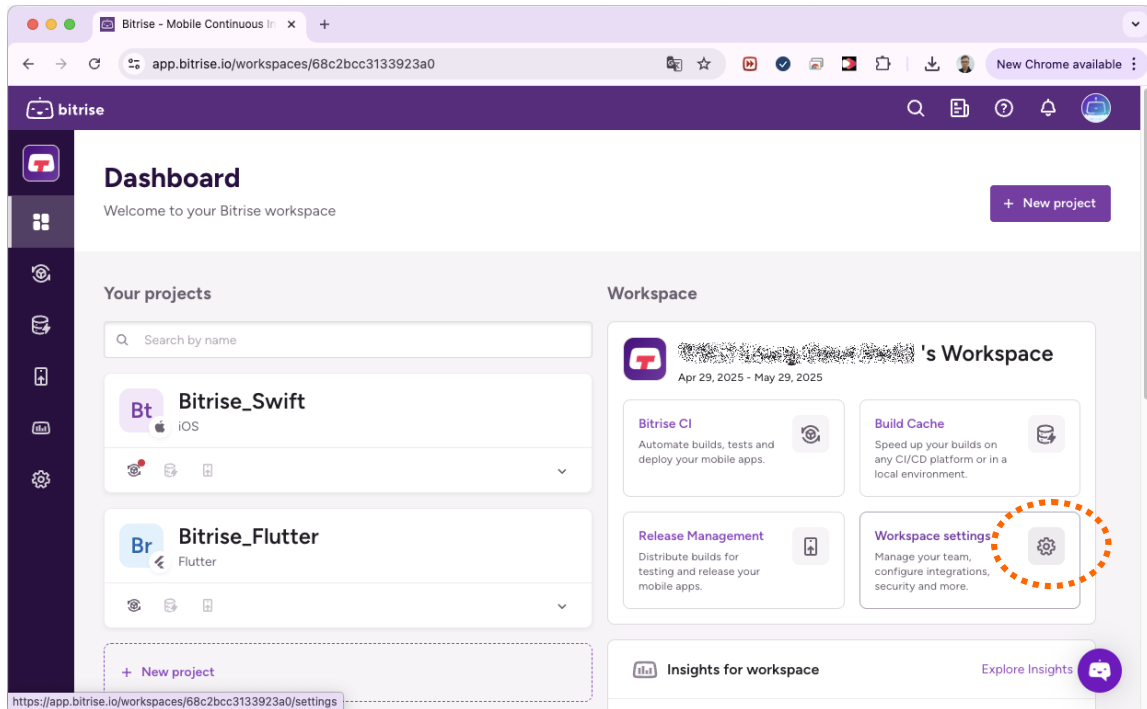
App Store 배포용 프로비저닝 프로파일

프로젝트 폴더에 App store 배포를 위한 프로비저닝 프로파일이 포함되어야 합니다. 이 파일은 Apple Developer 사이트에서 다운로드 한 프로파일을 "profile.mobileprovision"이라는 이름으로 포함시켜야 합니다. 파일명이 맞지 않거나 올바른 프로파일이 아닐 경우 재서명에 실패하거나 앱이 정상적으로 설치/실행이 되지 않을 수 있습니다.

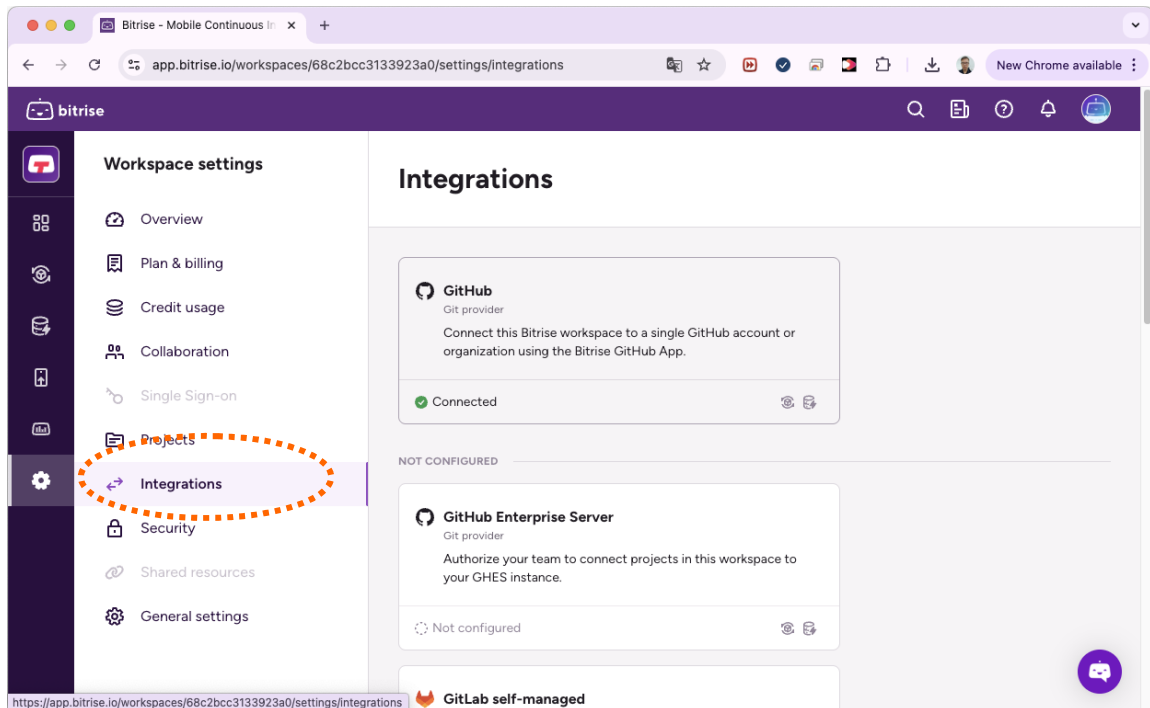
인증서와 프로파일을 프로젝트에 추가하고 소스코드 저장소에 푸시하기 전에 다음 단계를 먼저 수행하십시오. 최종 단계까지 모두 완료한 후 저장소에 푸시해야 Bitrise CI 프로젝트에서 정상적으로 빌드가 수행됩니다.

5-2 Bitrise CI 워크스페이스 설정

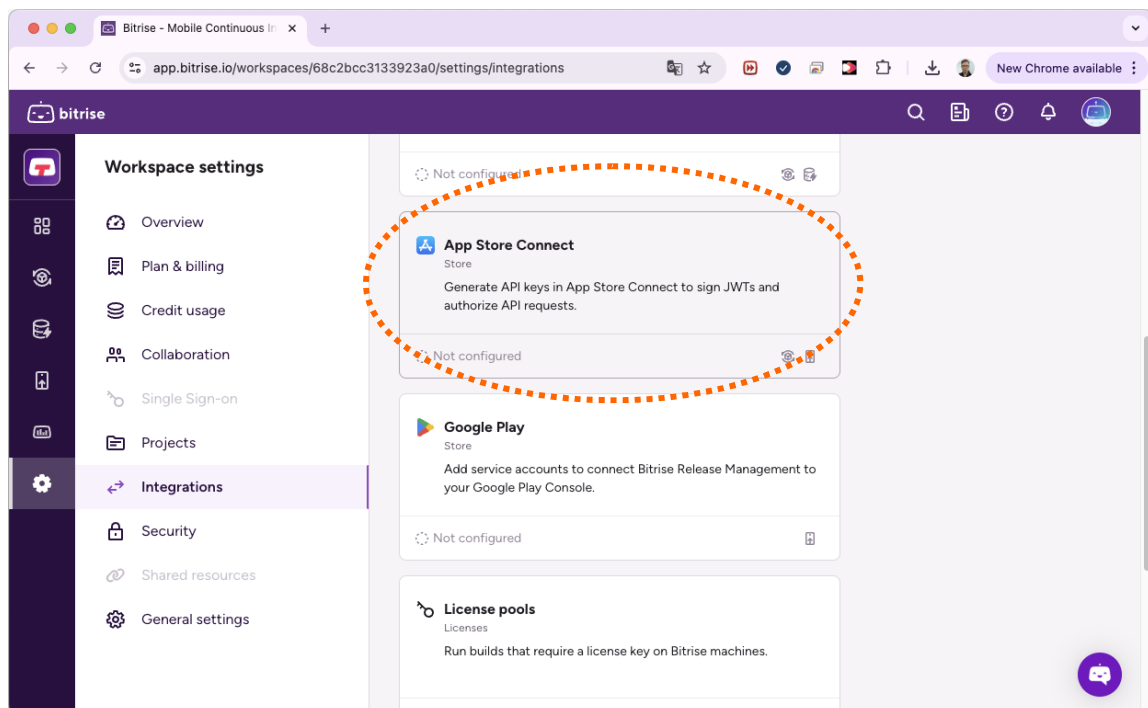
이 단계에서는 Bitrise CI 워크스페이스 설정을 진행합니다. 워크스페이스 설정은 프로젝트 수와 관계없이 1회만 진행하면 되고 각 프로젝트 설정은 플랫폼과 무관하게 모든 프로젝트에 동일한 과정을 적용하면 됩니다. 먼저 워크스페이스 설정을 위해 대시보드 화면에서 우측의 워크스페이스 설정 아이콘을 클릭합니다.



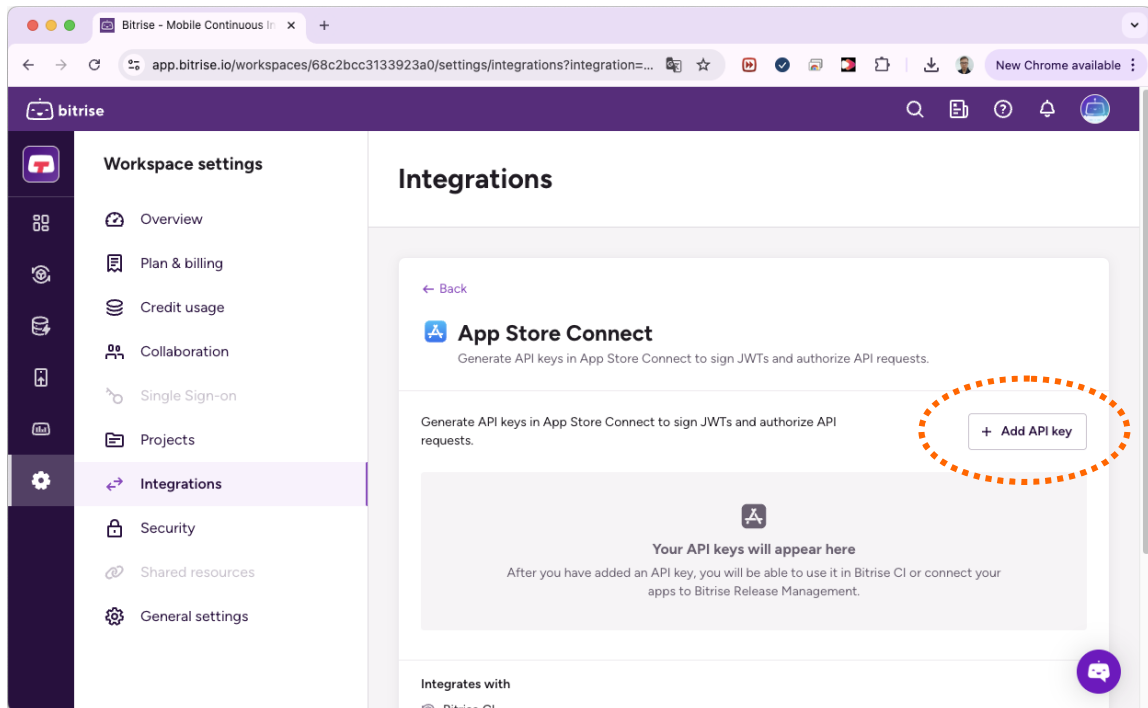
워크스페이스 설정 화면이 나타나면 좌측의 "Integrations" 항목을 선택합니다.



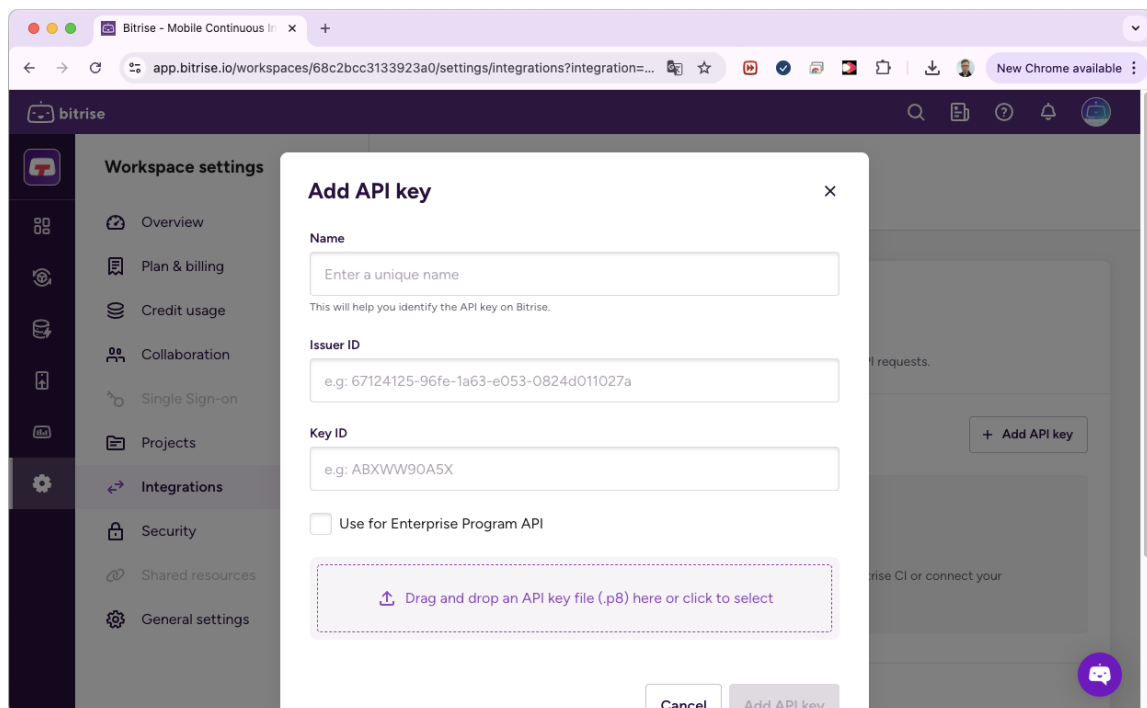
우측의 Integrations 패널을 아래로 스크롤 하면 "App Store Connect" 항목이 나타납니다. 해당 항목을 클릭합니다.



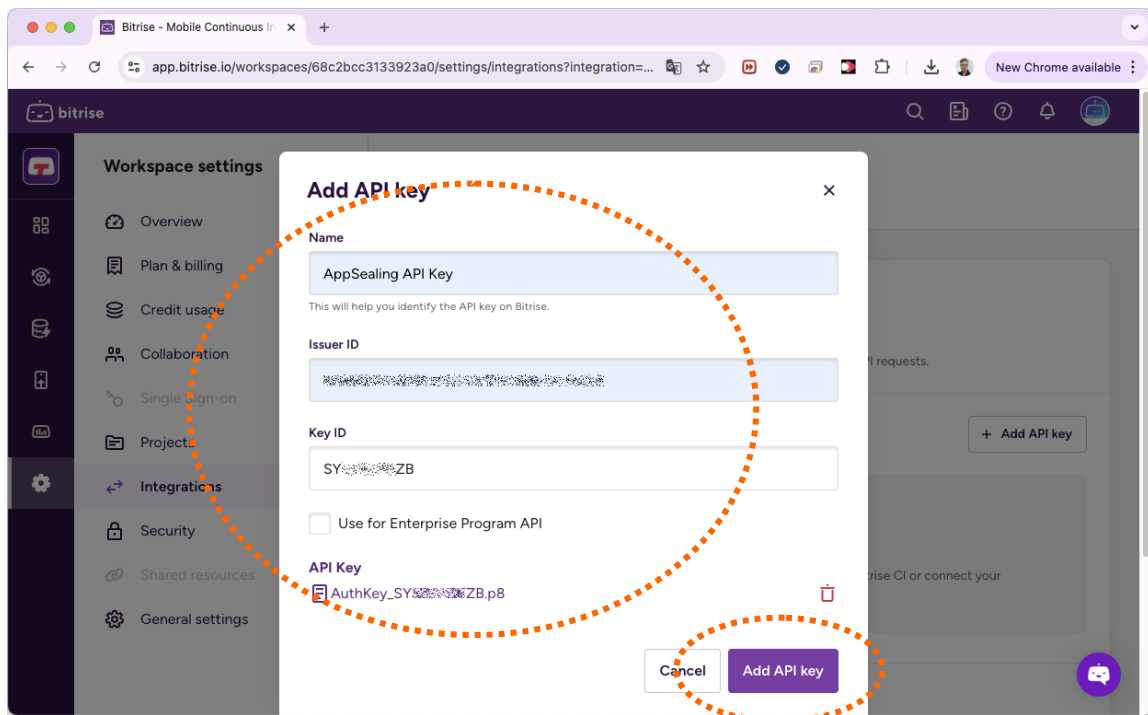
항목 클릭 시 "App Store Connect" 상세 페이지가 표시됩니다. 여기서 우측의 "Add API Key" 버튼을 클릭합니다.



버튼을 클릭하면 아래와 같이 API Key를 추가할 수 있는 창이 나타납니다.

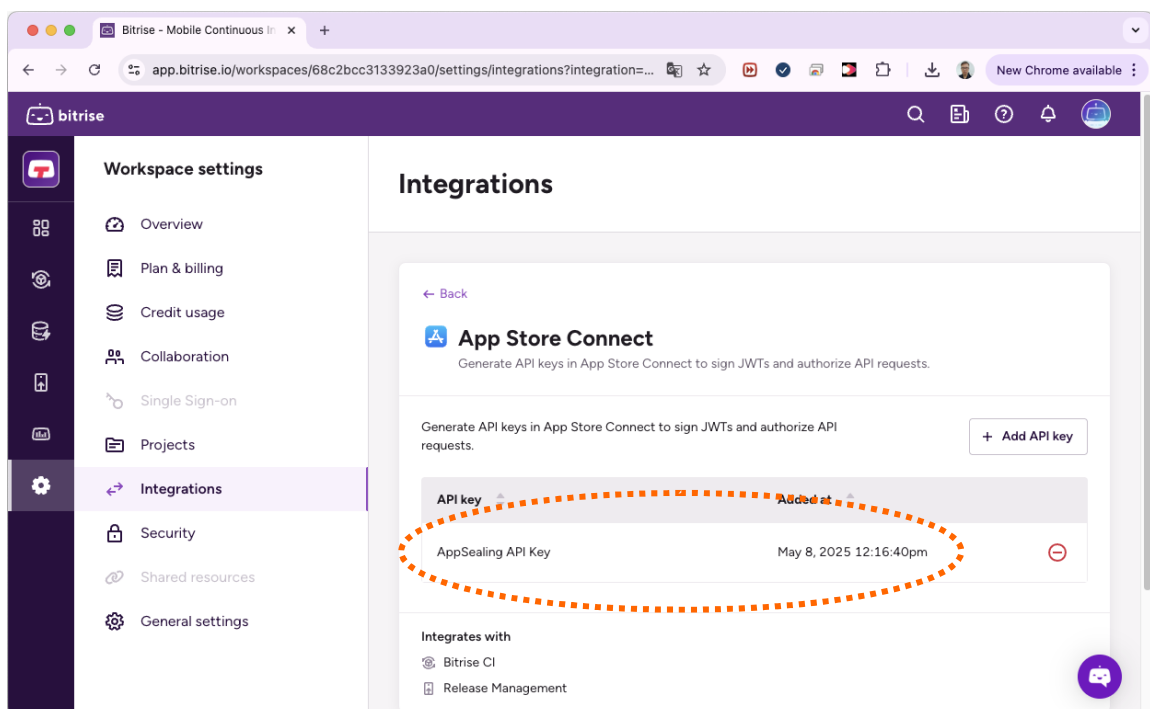


이 창에서 API Key 정보를 입력하고 키 파일(P8)을 업로드 한 뒤 "Add API Key" 버튼을 클릭합니다.



기존에 App Store Connect API Key가 발급되어 있지 않은 상태라면 다음 링크의 내용을 참조하여 API 키를 발급받은 후 위 과정을 진행합니다.

(<https://developer.apple.com/documentation/appstoreconnectapi/creating-api-keys-for-app-store-connect-api>)



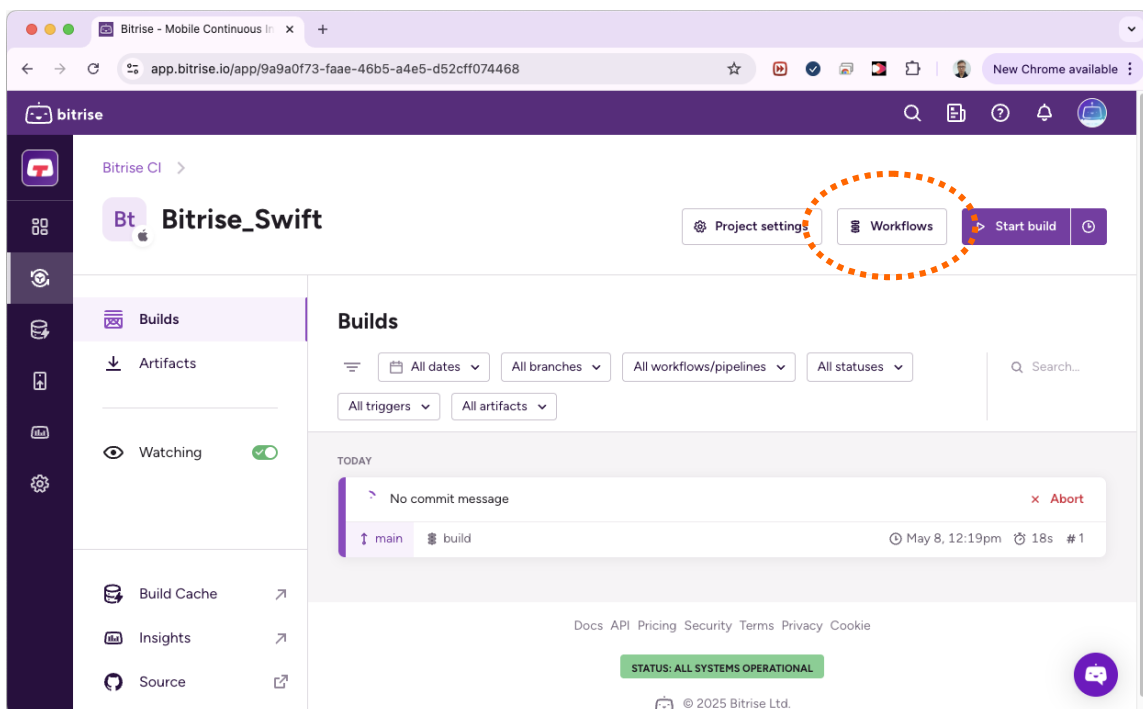
+

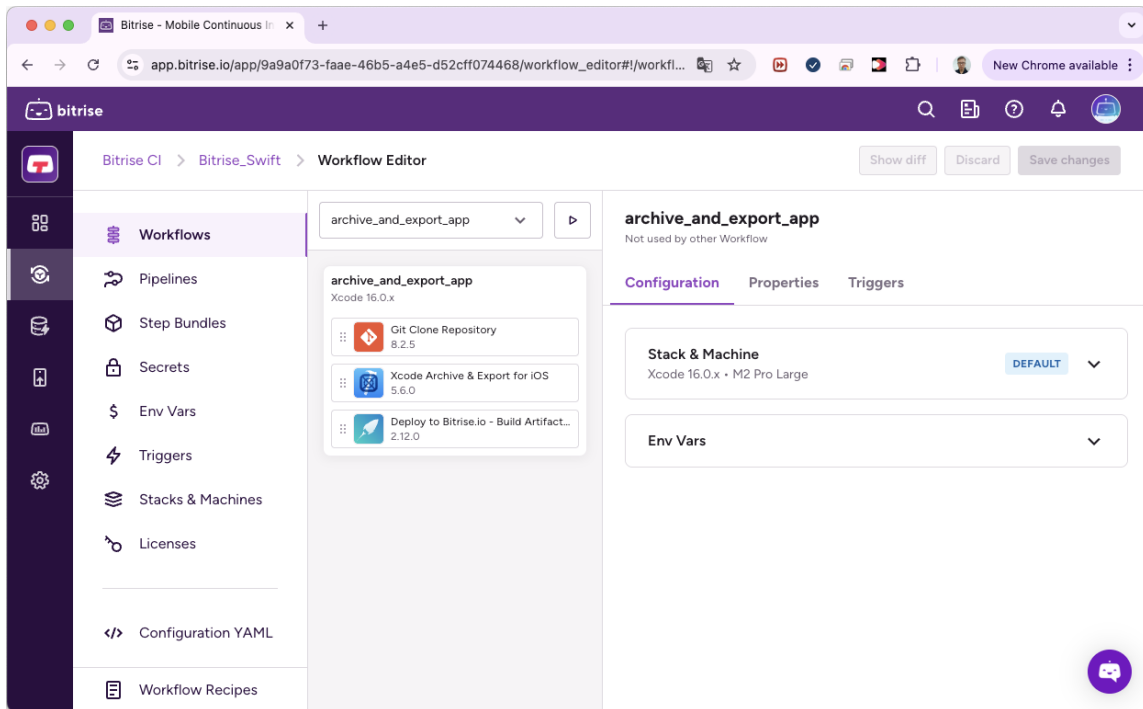
5-3 Bitrise CI 프로젝트 설정

이 절에서는 프로젝트의 설정을 진행합니다. 워크스페이스 설정은 1회만 진행하지만 프로젝트 설정은 모든 Bitrise CI 프로젝트마다 각각 진행해야 됩니다. 여기서는 Xcode의 기본 Swift 프로젝트와 Github 저장소를 연결한 Bitrise_Swift 라는 프로젝트를 예로 들어 설명합니다.

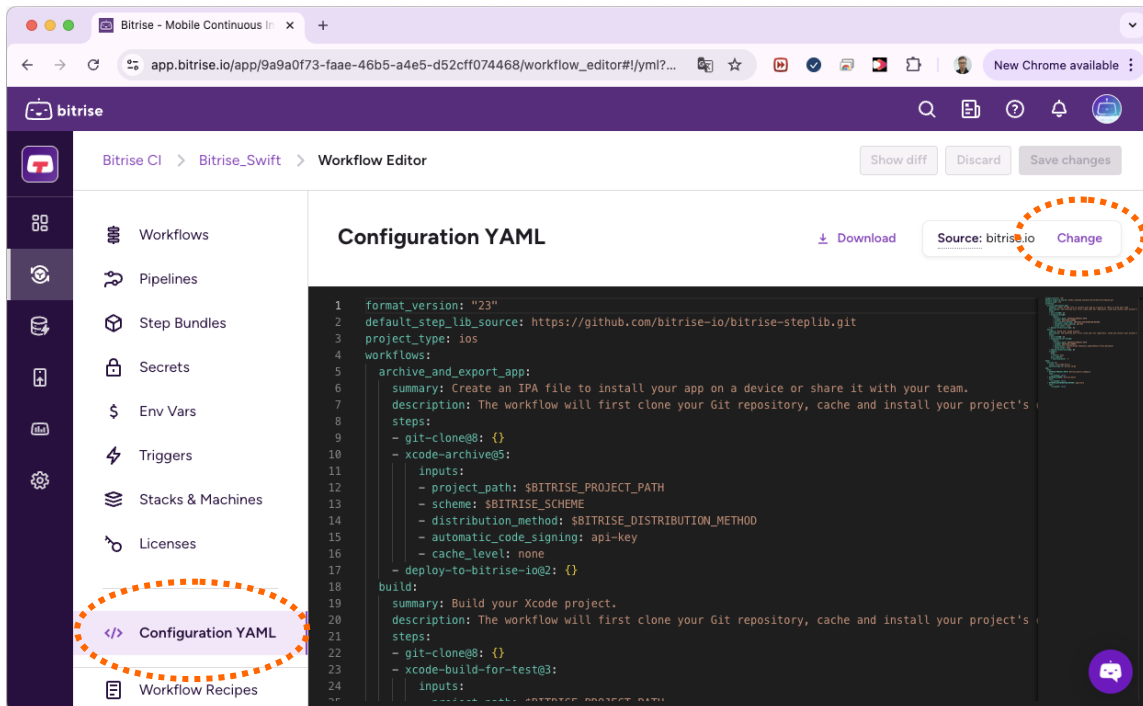
프로젝트 설정을 하기에 앞서 먼저 AppSealing SDK가 적용된 Xcode 프로젝트를 빌드하기 위한 YAML 스크립트 파일을 프로젝트 폴더에 추가하고 github 저장소에 추가합니다. 해당 스크립트 파일은 SDK의 "Bitrise Scripts/bitrise_swift.yml" 입니다. 이 파일의 이름을 "bitrise.yml"로 변경한 후 프로젝트 폴더에 복사합니다.

이제 스크립트 파일을 추가한 후 Bitrise_Swift 프로젝트 빌드에 이 스크립트를 사용하도록 수정해야 합니다. 먼저 프로젝트 화면에서 우측 상단의 "Workflows" 버튼을 클릭하여 워크플로우 설정 화면으로 이동합니다.

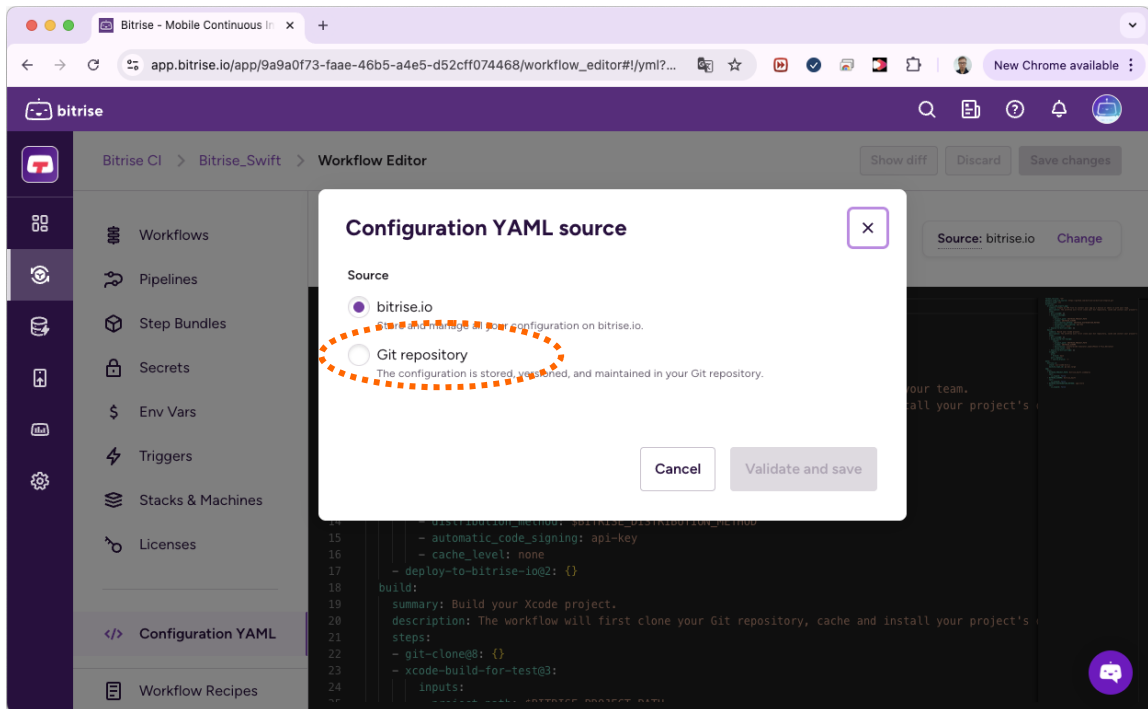




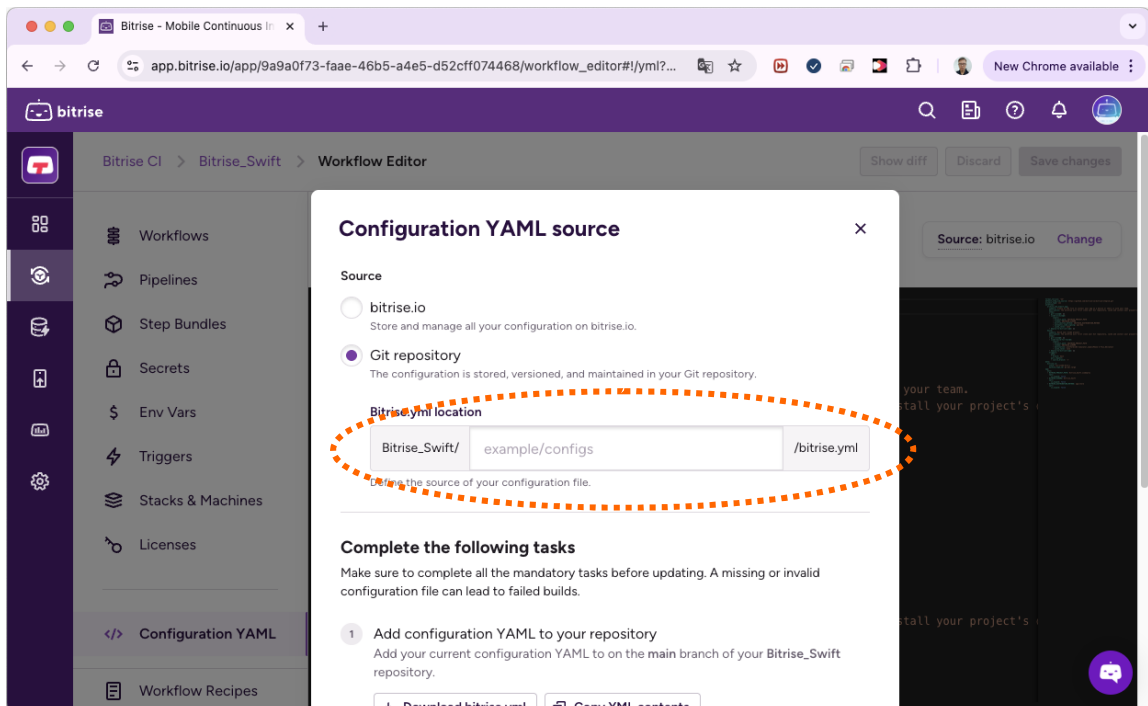
좌측의 메뉴에서 "Configuration YAML" 항목을 선택합니다. 우측에 기본 설정된 YAML 스크립트가 표시됩니다. 우측 상단에 "Source: Bitrise.io" 라고 표시되어 있는데 이 부분을 새로 수정한 스크립트로 바꾸기 위해 바로 오른쪽의 "Change" 버튼을 클릭합니다.



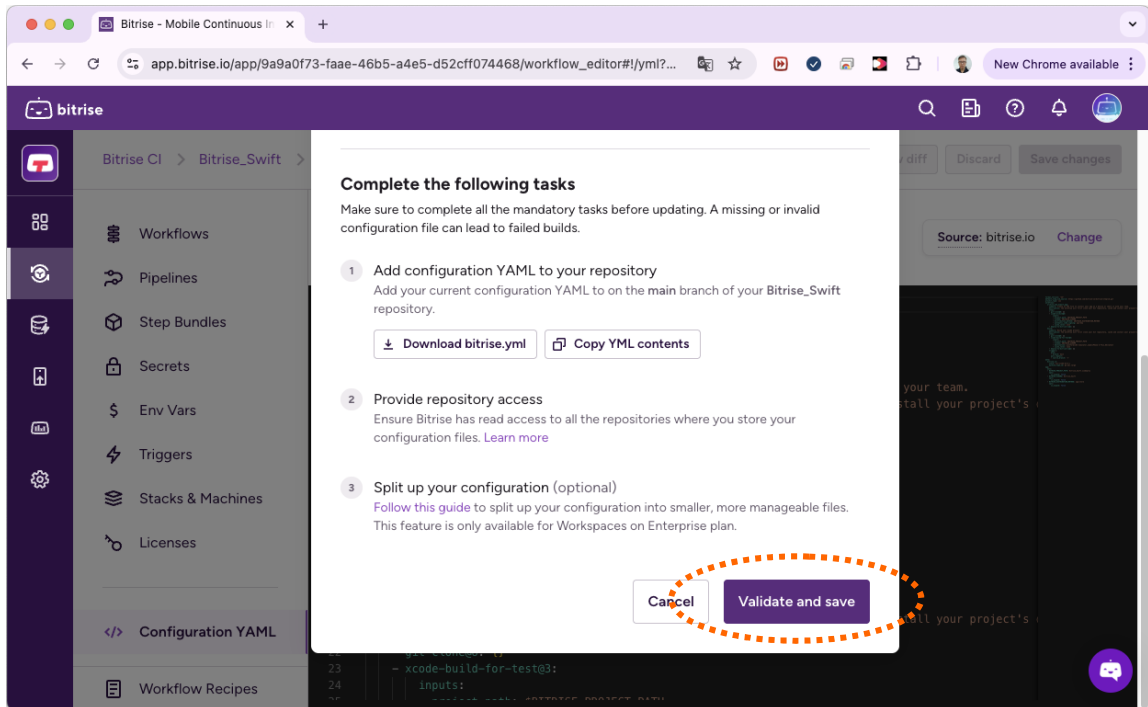
"Change" 버튼을 클릭하면 아래와 같이 YAML을 설정할 수 있는 창이 나타납니다. 이 창에서 아래쪽의 "Git repository"를 선택합니다.



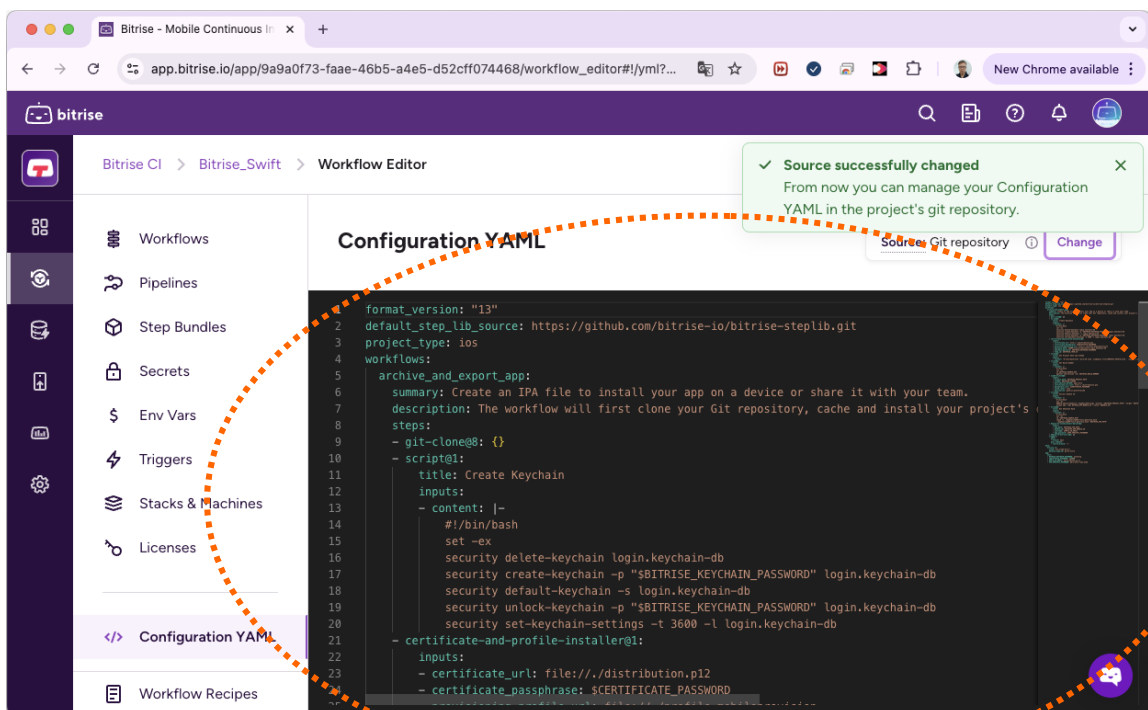
"Git repository"를 선택하면 아래 화면과 같이 bitrise.yml 파일의 경로를 선택하기 위한 창이 나타납니다. bitrise.yml 파일을 프로젝트 루트 폴더에 추가했기 때문에 따로 경로를 지정하지 않아도 됩니다.



경로 지정 없이 창을 아래로 스크롤 한 후 "Validate and save" 버튼을 클릭합니다.

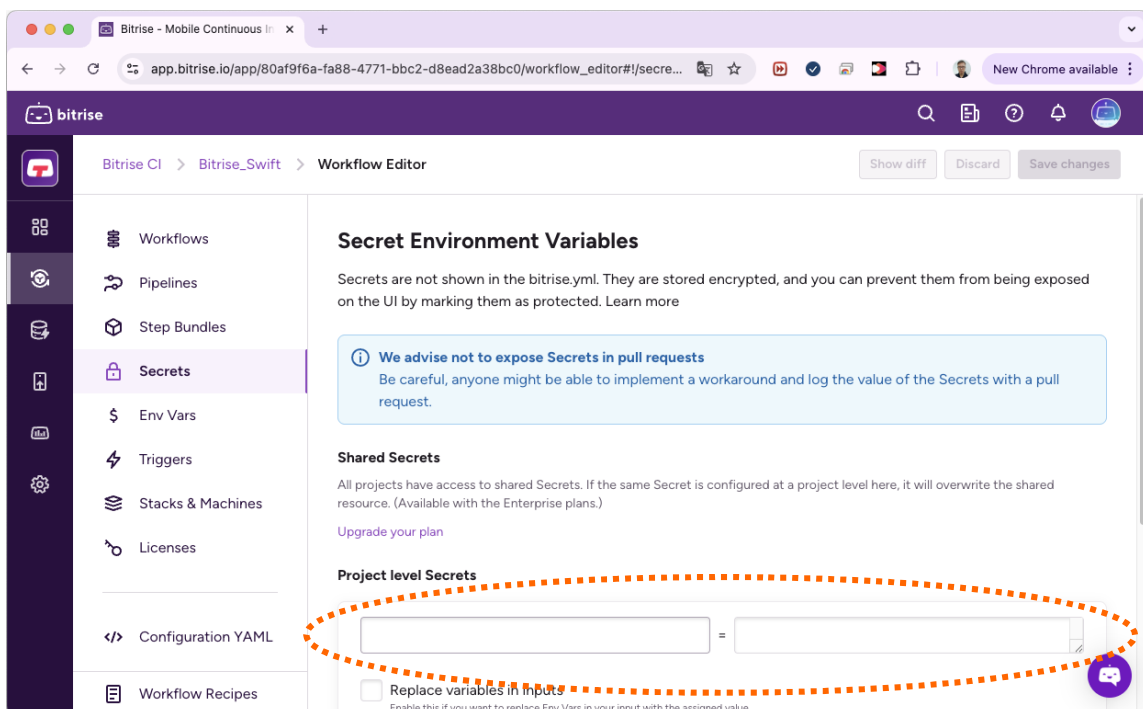
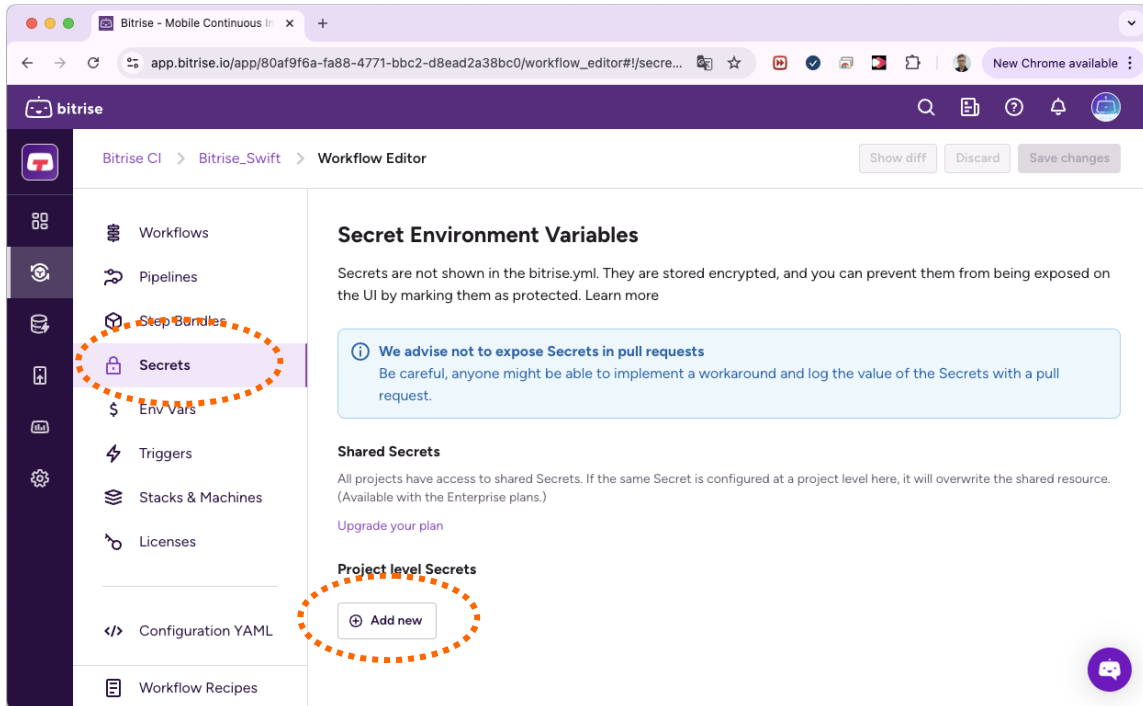


이제 아래와 같이 직접 추가한 bitrise.yml 파일의 내용이 표시됩니다.

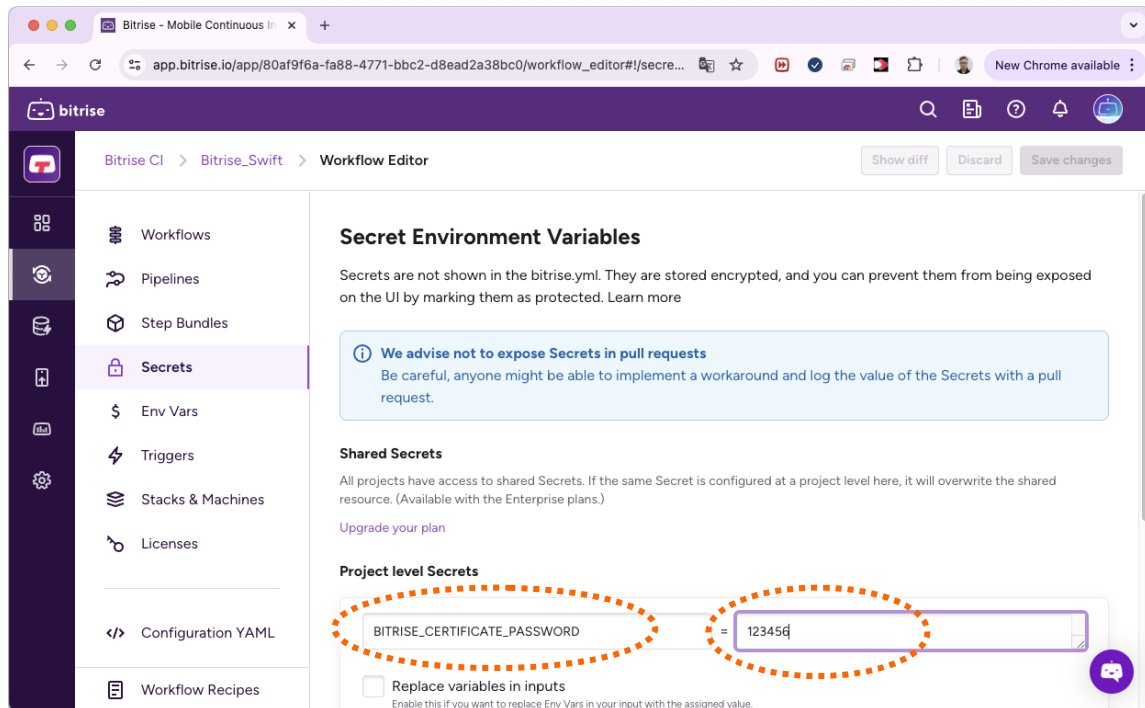


다음은 스크립트에서 사용하는 BITRISE_CERTIFICATE_PASSWORD 변수의 값을 설정해야 합니다. 이 비밀번호는 P12 인증서 파일을 생성할 때 지정한 비밀번호로 7-1에서 설명된 배포용 인증서 내보내기 과정에서 지정한 비밀번호입니다.

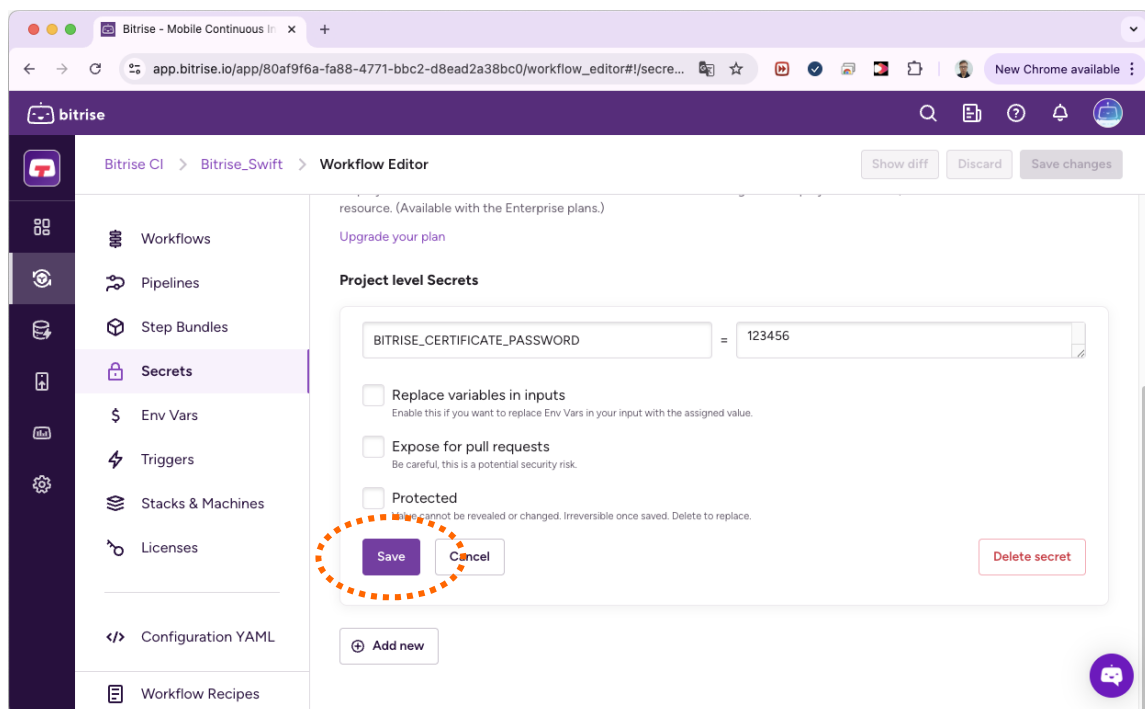
좌측의 메뉴 중 "Secrets"를 선택하고 오른쪽 아래의 "Add new" 버튼을 클릭합니다.



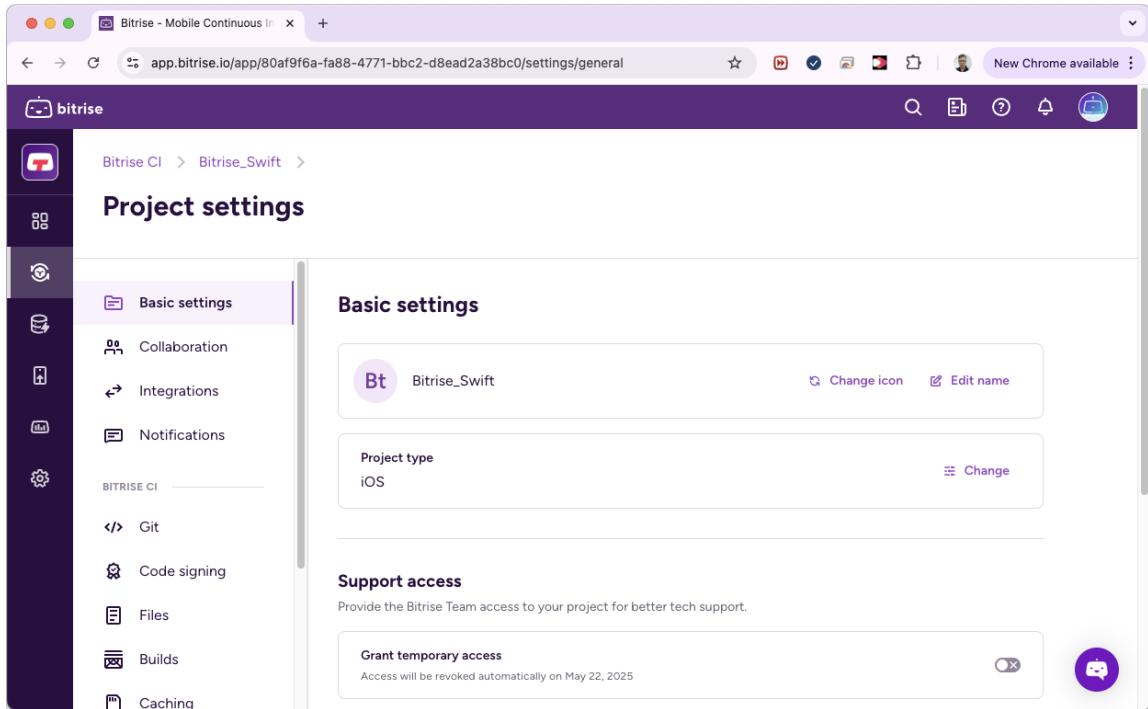
입력 창이 나타나면 좌측의 변수 이름은 "BITRISE_CERTIFICATE_PASSWORD" 로 입력하고 우측의 값은 P12 내보내기 시에 사용했던 실제 비밀번호를 입력합니다.



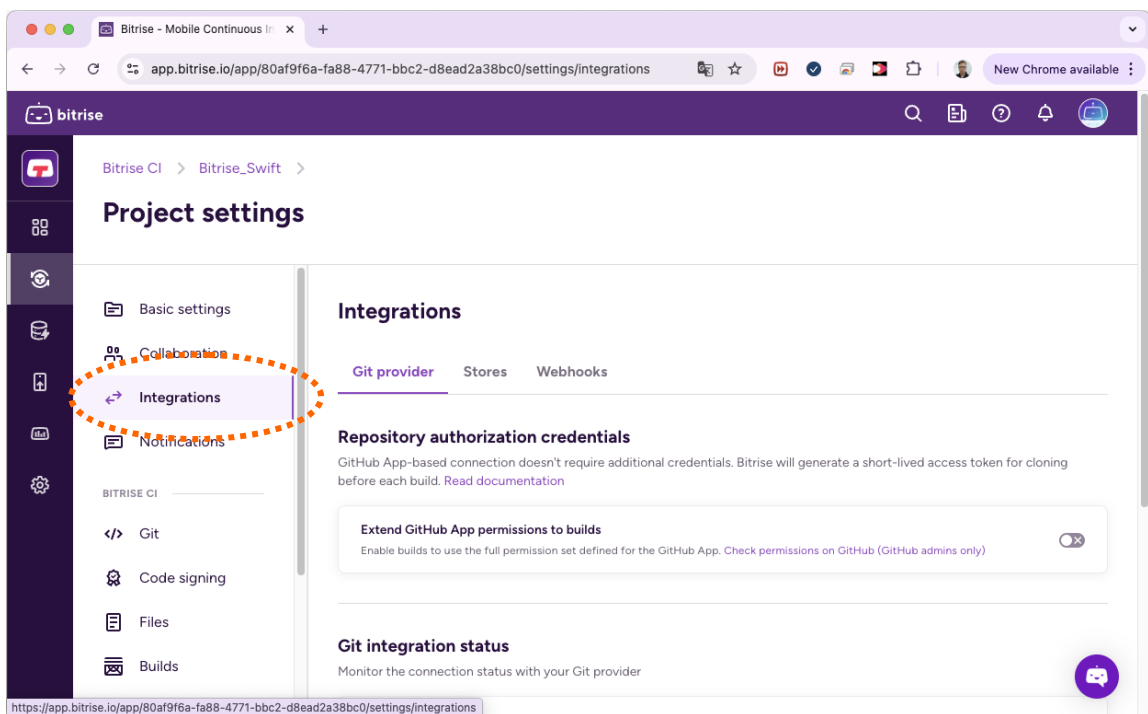
입력이 완료되면 화면을 아래로 스크롤 하여 "Save" 버튼을 클릭합니다.



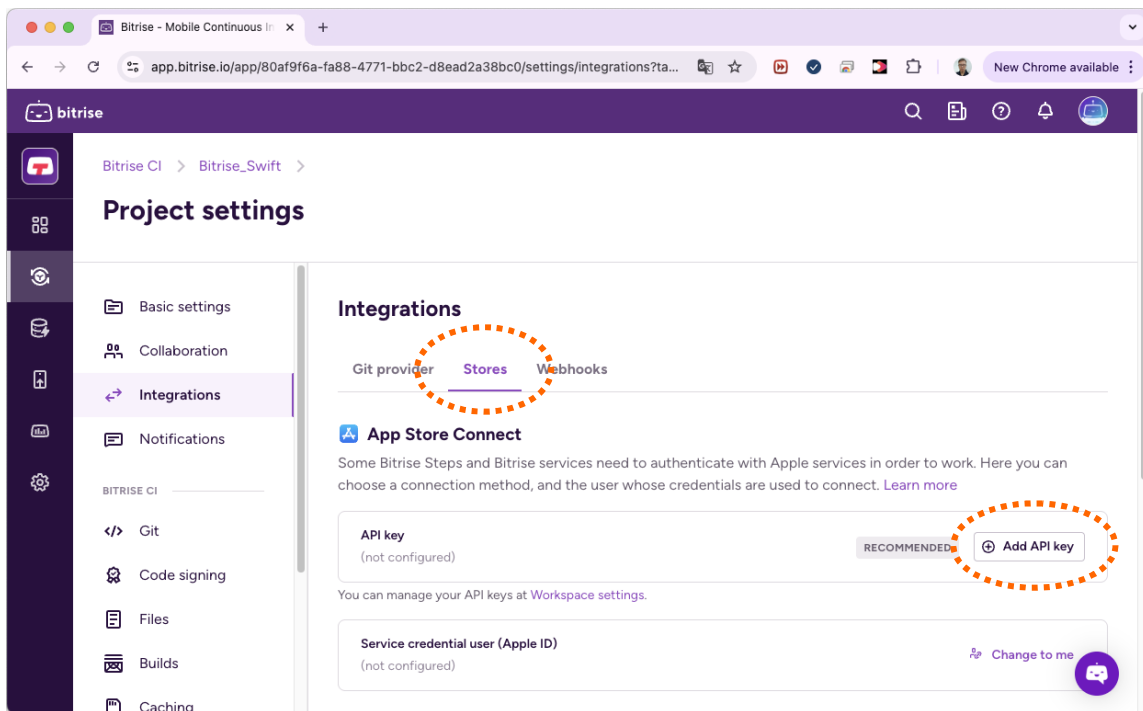
다음은 빌드 후 App Store Connect 업로드를 위한 API Key 설정을 진행합니다. 앞서 워크스페이스 설정에서 등록해 둔 API Key를 사용하기 위해 다시 프로젝트 설정 화면으로 이동합니다.



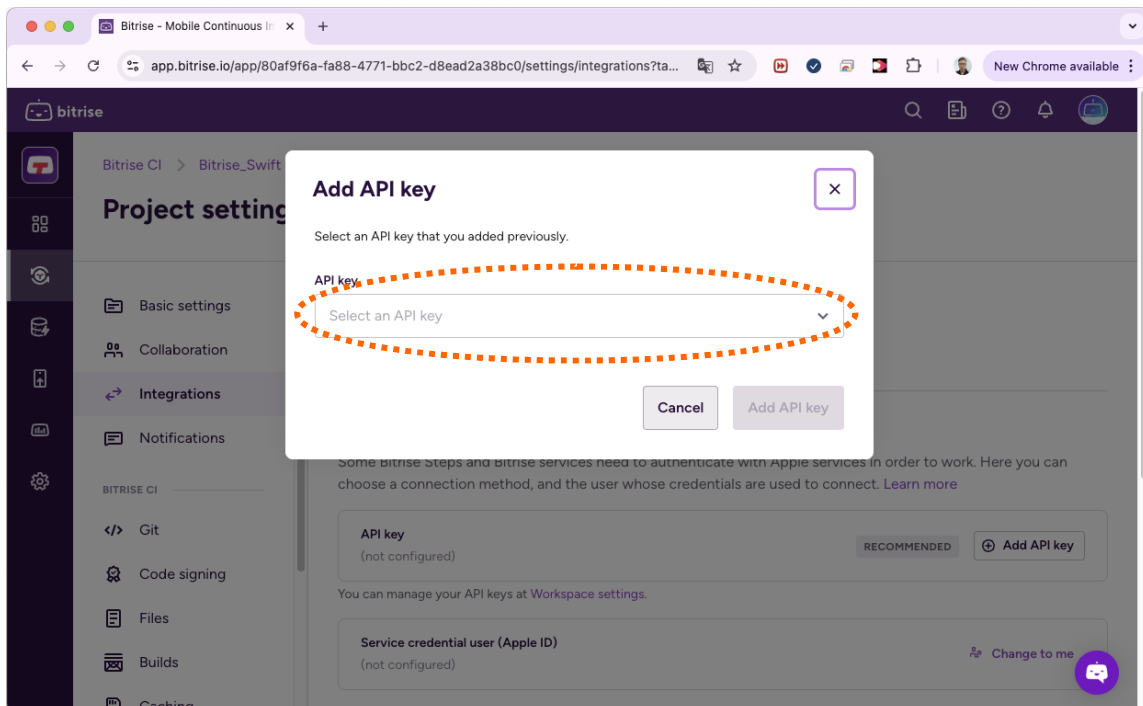
왼쪽 메뉴 중에 Integrations 항목을 선택합니다. 오른쪽에 Integrations 페이지가 표시 됩니다.



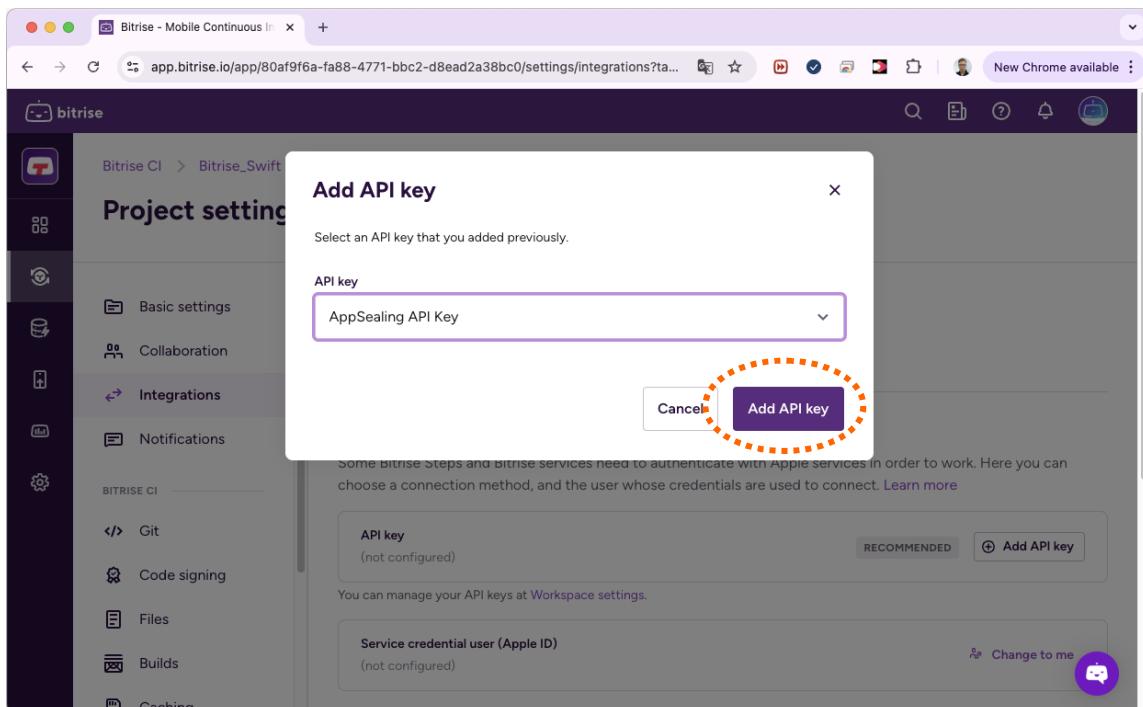
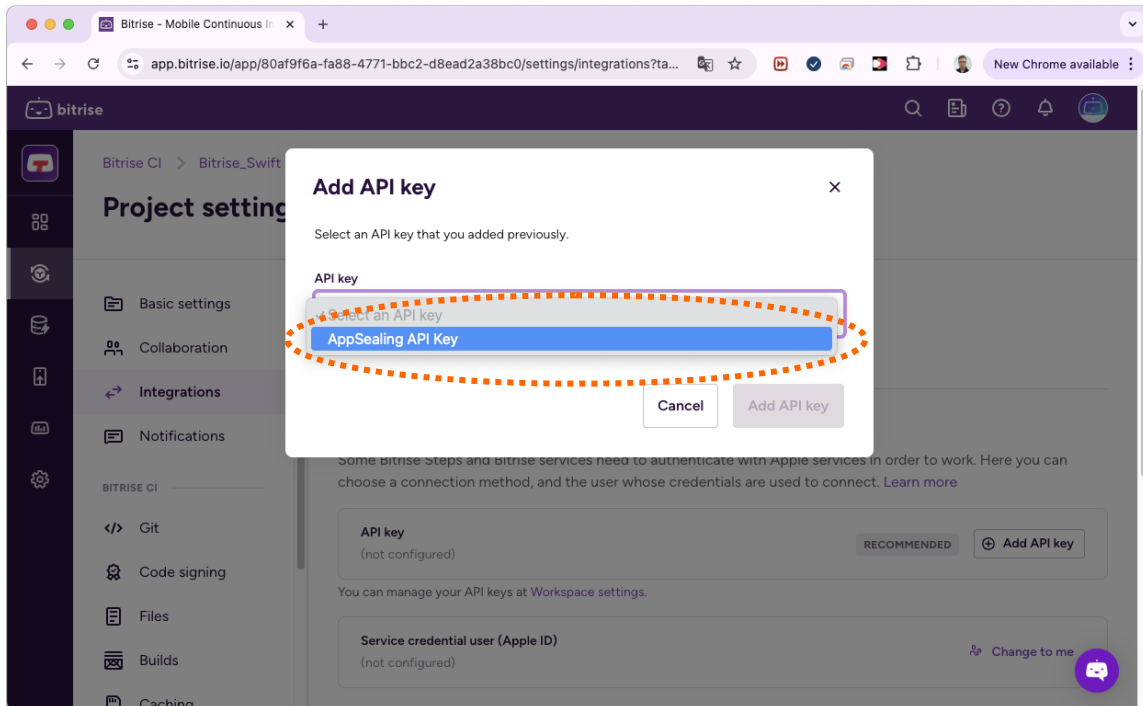
오른쪽에 표시되는 세 개의 탭 중 가운데의 "Stores"를 선택합니다. 아래 쪽에 "App Store Connect" 내용이 표시되면 우측 하단의 "Add API Key" 버튼을 클릭합니다.



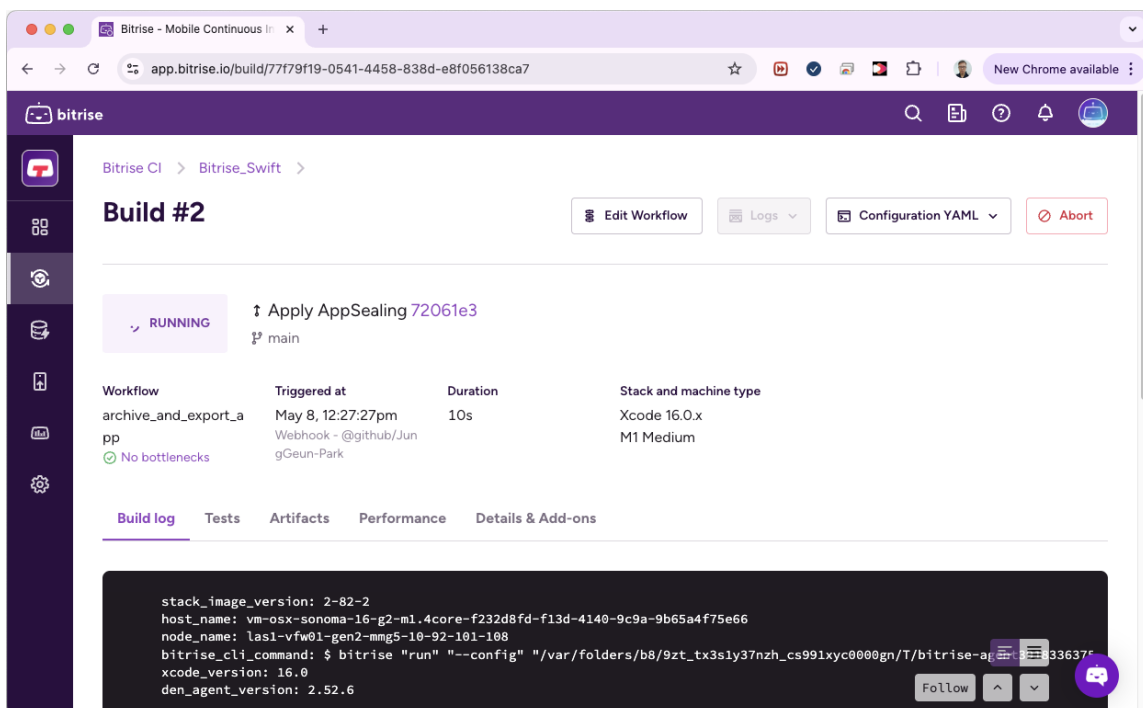
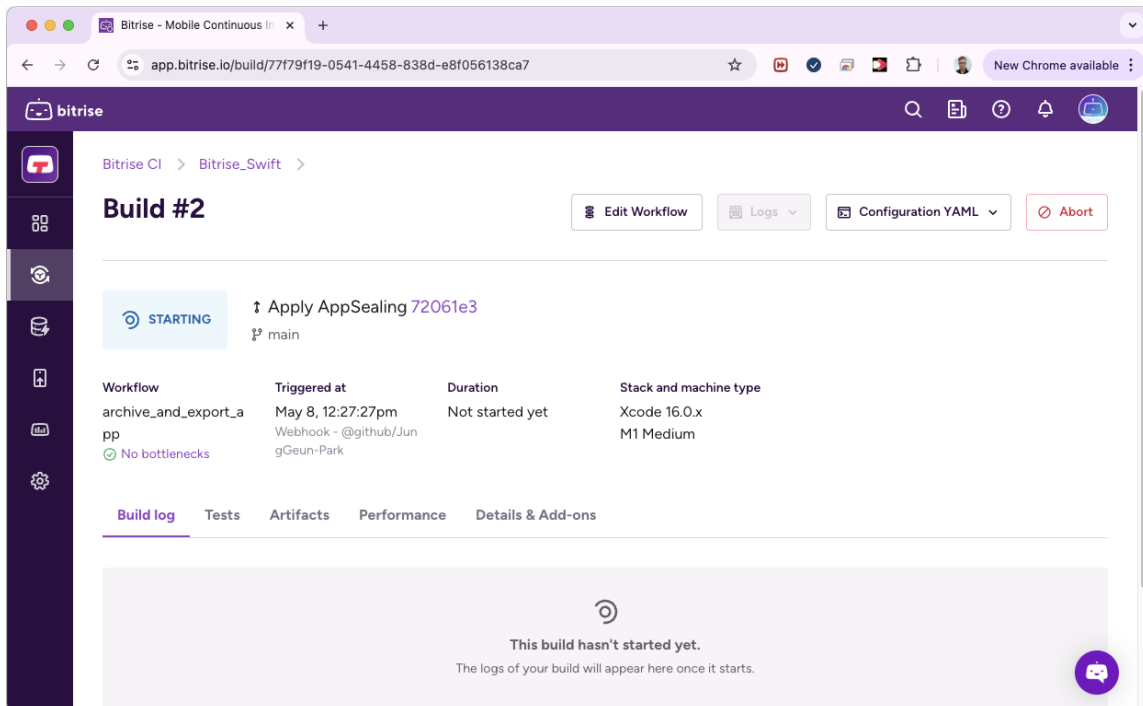
Add API Key 창이 나타납니다. 여기서 "API Key" 목록을 클릭합니다.



이전에 워크스페이스 설정에서 등록해 둔 API Key 항목이 표시되면 해당 항목을 선택하고 하단의 "Add API Key" 버튼을 클릭합니다.



이제 모든 준비와 설정 과정이 완료됐습니다. 수동으로 “Start Build” 버튼을 클릭하거나 새로운 코드를 푸시하여 자동으로 빌드를 시작합니다.



SDK에 제공되는 스크립트를 통해 IPA 빌드와 generate_hash 적용, App Store Connect 업로드까지 모두 한꺼번에 진행됩니다.

The screenshot shows the Bitrise CI interface for a build named 'Build #4'. The build status is 'SUCCESS'. The workflow is 'archive_and_export_app'. The build was triggered at May 8, 2:04:24pm by a Webhook from @github/JungGeun-Park. The duration was 1m 49s, and it used 4 credits. The stack and machine type are Xcode 16.0.x on an M1 Medium machine. The build log shows the following details:

```
host_name: vm-osx-sonoma-16-g2-m1.4core-8cb6b0a1-deff-477d-aec8-00375c240567
node_name: las1-vfw01-gen2-mm5-10-92-100-200
bitrise_cli_command: $ bitrise "run" "--config" "/var/folders/b8/9zt_tx3sly37nzh_cs991xyc0000gn/T/bitrise-agent3399448571
xcode_version: 16.0
den_agent_version: 2.52.6
stack_image_version: 2-82-2
```

The screenshot shows the Bitrise CI interface for a build named 'Build #4'. The build status is 'SUCCESS'. The workflow is 'archive_and_export_app'. The build was triggered at May 8, 2:04:24pm by a Webhook from @github/JungGeun-Park. The duration was 1m 49s, and it used 4 credits. The stack and machine type are Xcode 16.0.x on an M1 Medium machine. The build log shows the following details:

```
bitrise summary: archive_and_export_app
+-----+-----+
| title | time (s) |
+-----+-----+
| ✓ | Git Clone Repository | 12.44 sec |
| ✓ | Create Keychain | 0.75 sec |
| ✓ | Certificate and profile installer | 2.57 sec |
| ✓ | Set Project Path and Scheme | 2.43 sec |
| ✓ | Set Build Number | 2.12 sec |
| ✓ | Xcode Archive & Export for iOS | 41.48 sec |
| ✓ | Extract Bundle ID | 1.33 sec |
| ✓ | Run Generate Hash | 3.40 sec |
| ✓ | Deploy to App Store Connect with Deliver (formerly iTunes C...) | 35.50 sec |
| ✓ | Deploy to Bitrise.io - Build Artifacts, Test Reports, and P... | 4.46 sec |
+-----+-----+
| Total runtime: 1.8 min |
+-----+-----+
```

Part 6. Circle CI 파이프라인 적용

이 장에서는 Circle CI 툴에서 관리되는 프로젝트에 AppSealing SDK를 적용하여 빌드하기 위한 방법을 소개합니다. 적용 가능한 프로젝트의 종류는 Xcode 기본 프로젝트(Swift / Objective-C)와 Flutter 프로젝트, Ionic, Cordova, React Native 프로젝트 입니다. 모든 플랫폼의 프로젝트에 대해 동일한 준비 과정이 필요하며 빌드 스크립트는 각 플랫폼마다 다른 스크립트를 사용합니다.

6-1 인증서 및 프로파일 준비

AppSealing SDK를 적용한 프로젝트의 경우 IPA가 export 되고 난 후 generate_hash 스크립트를 실행하는 단계에서 재서명 과정을 거칩니다. 따라서 최초 IPA 서명에 사용된 배포용 인증서와 개인키, 배포용 프로비저닝 프로파일이 필요하며 이 파일들을 준비해서 소스 코드 저장소의 폴더에 추가해 주어야 합니다.

App Store 배포용 인증서

프로젝트 폴더에 반드시 배포용 인증서가 포함되어야 합니다. 이 인증서는 Apple Developer 사이트에서 다운로드 받은 배포용 인증서를 개인키와 함께 PKCS#12 형식으로 내보내기 한 P12 파일이어야 합니다. 파일명은 반드시 "distribution.p12"로 고정되어 있어야 하고 P12 파일을 생성할 때 설정한 비밀번호는 4-2의 Azure 프로젝트 설정 단계에서 variables 에 추가되어야 합니다. (4-2 참조)

App Store 배포용 프로비저닝 프로파일

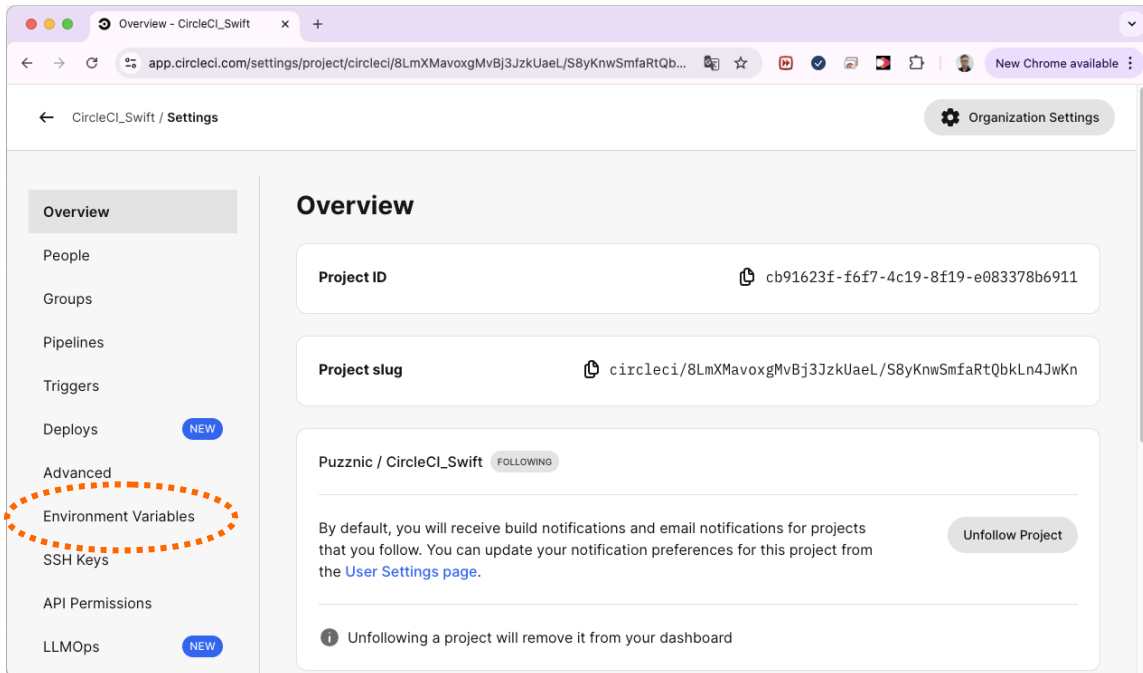
프로젝트 폴더에 App store 배포를 위한 프로비저닝 프로파일이 포함되어야 합니다. 이 파일은 Apple Developer 사이트에서 다운로드 한 프로파일을 "profile.mobileprovision"이라는 이름으로 포함시켜야 합니다. 파일명이 맞지 않거나 올바른 프로파일이 아닐 경우 재서명에 실패하거나 앱이 정상적으로 설치/실행이 되지 않을 수 있습니다.

인증서와 프로파일을 프로젝트에 추가하고 소스코드 저장소에 푸시하기 전에 다음 단계를 먼저 수행하십시오. 최종 단계까지 모두 완료한 후 저장소에 푸시해야 Circle CI 파이프라인에서 정상적으로 빌드가 수행됩니다.

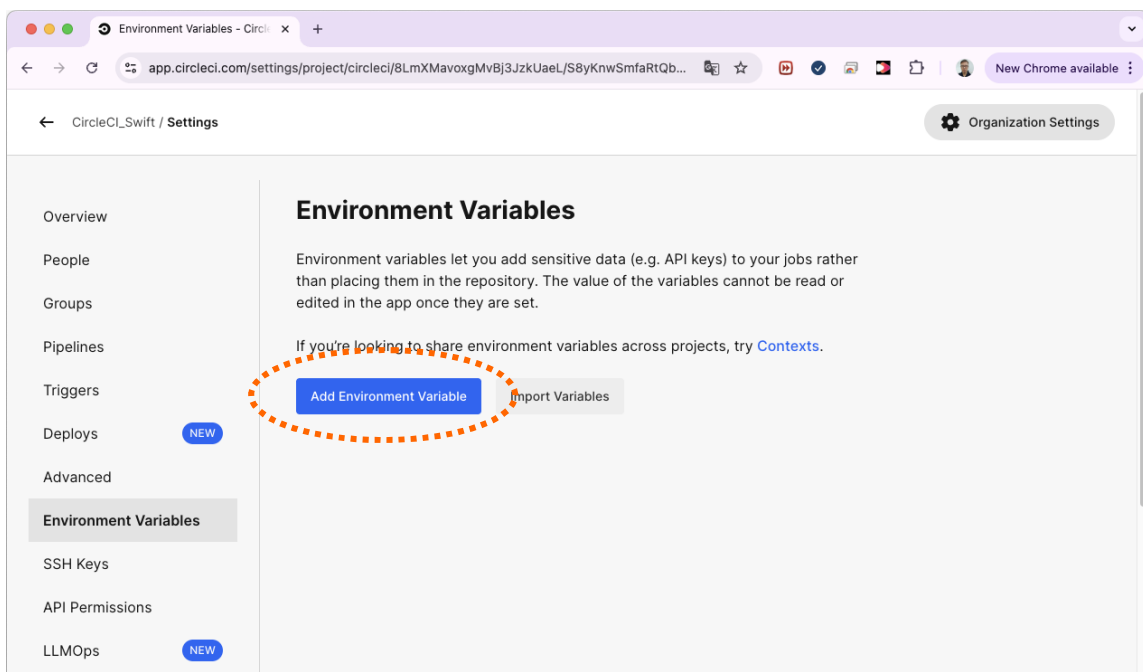
6-2 Circle CI 프로젝트 설정

이 단계에서는 Circle CI 프로젝트와 관련된 설정을 진행합니다. 프로젝트의 플랫폼과 무관하게 모든 프로젝트에 동일한 과정을 수행하면 됩니다.

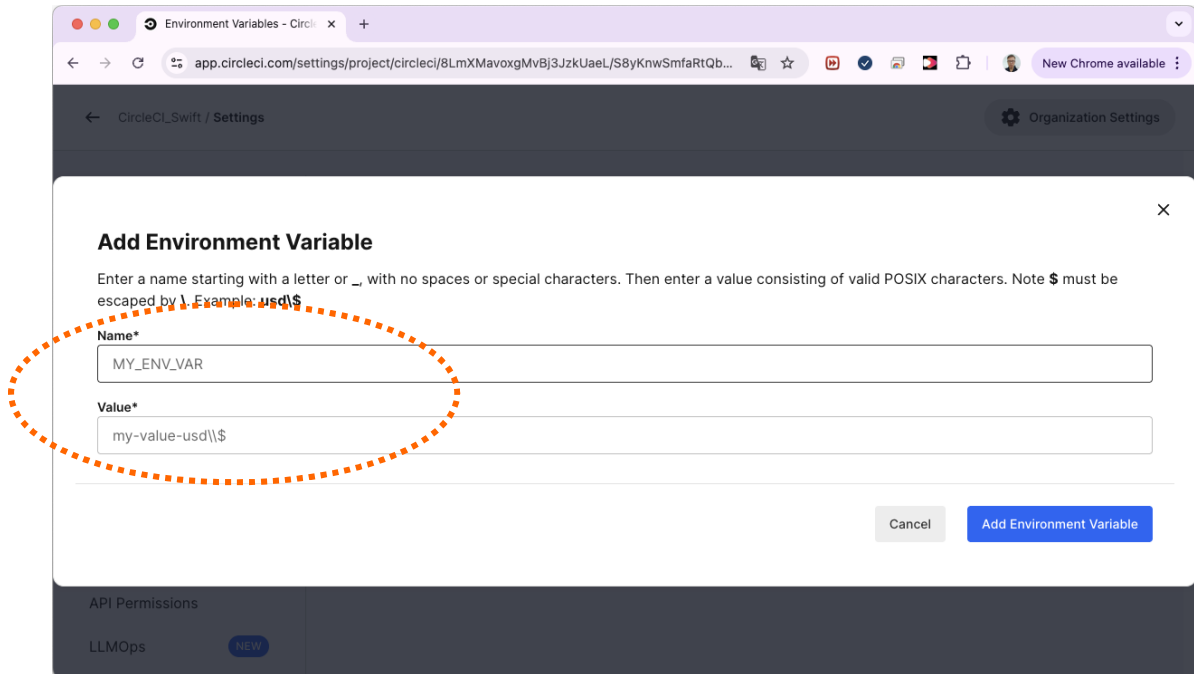
대상 프로젝트의 Overview 화면으로 이동하고 좌측 메뉴 중 "Environment Variables" 항목을 선택합니다.



"Environment Variables" 화면이 표시되면 "Add Environment Variable" 버튼을 클릭합니다.



아래와 같이 variable 등록 화면이 나타나면 Name과 Value 칸에 각각 "TEAM_ID" 문자열과 실제 사용 중인 팀 ID 값을 입력합니다.



Environment Variables - CircleCI

app.circleci.com/settings/project/circleci/8LmXMavoxgMvBj3JzkUaeL/S8yKnwSmfaRtQb...

CircleCI_Swift / Settings

Organization Settings

Add Environment Variable

Enter a name starting with a letter or _ with no spaces or special characters. Then enter a value consisting of valid POSIX characters. Note \$ must be escaped by \. Example: `usd\\$`

Name*

MY_ENV_VAR

Value*

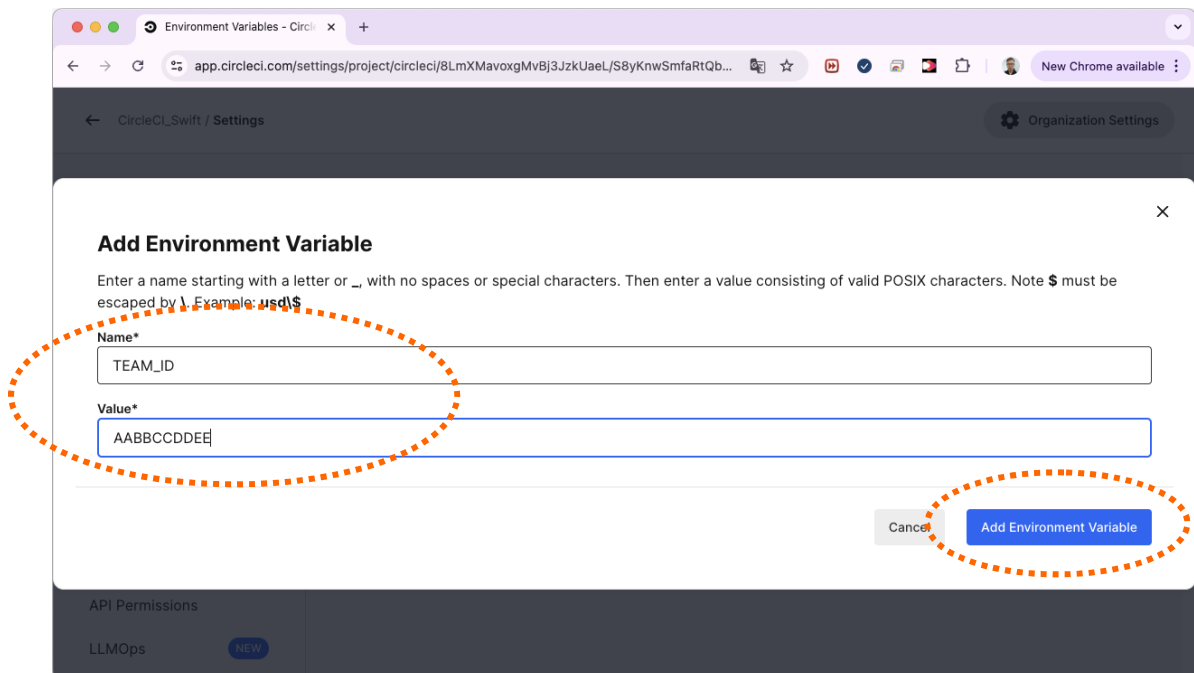
my-value-usd\\\$

Cancel Add Environment Variable

API Permissions

LLMOps NEW

아래와 같이 variable의 이름과 실제 값을 입력한 후 하단의 "Add Environment Variable" 버튼을 클릭합니다.



Environment Variables - CircleCI

app.circleci.com/settings/project/circleci/8LmXMavoxgMvBj3JzkUaeL/S8yKnwSmfaRtQb...

CircleCI_Swift / Settings

Organization Settings

Add Environment Variable

Enter a name starting with a letter or _ with no spaces or special characters. Then enter a value consisting of valid POSIX characters. Note \$ must be escaped by \. Example: `usd\\$`

Name*

TEAM_ID

Value*

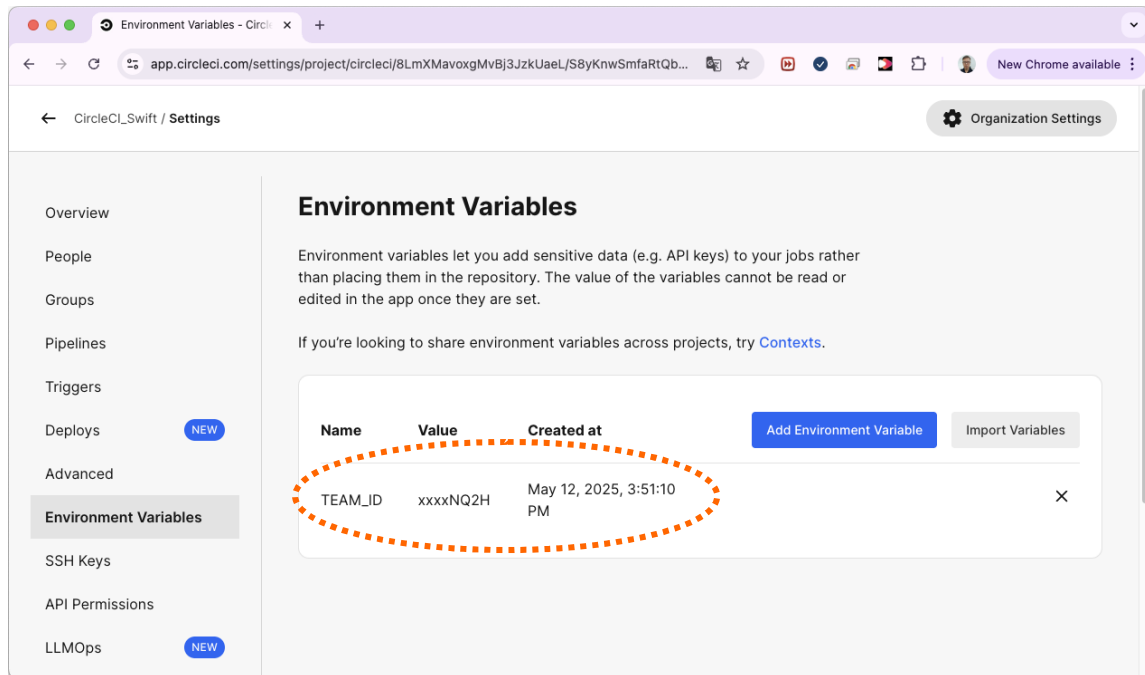
AABBCCDDEE

Cancel Add Environment Variable

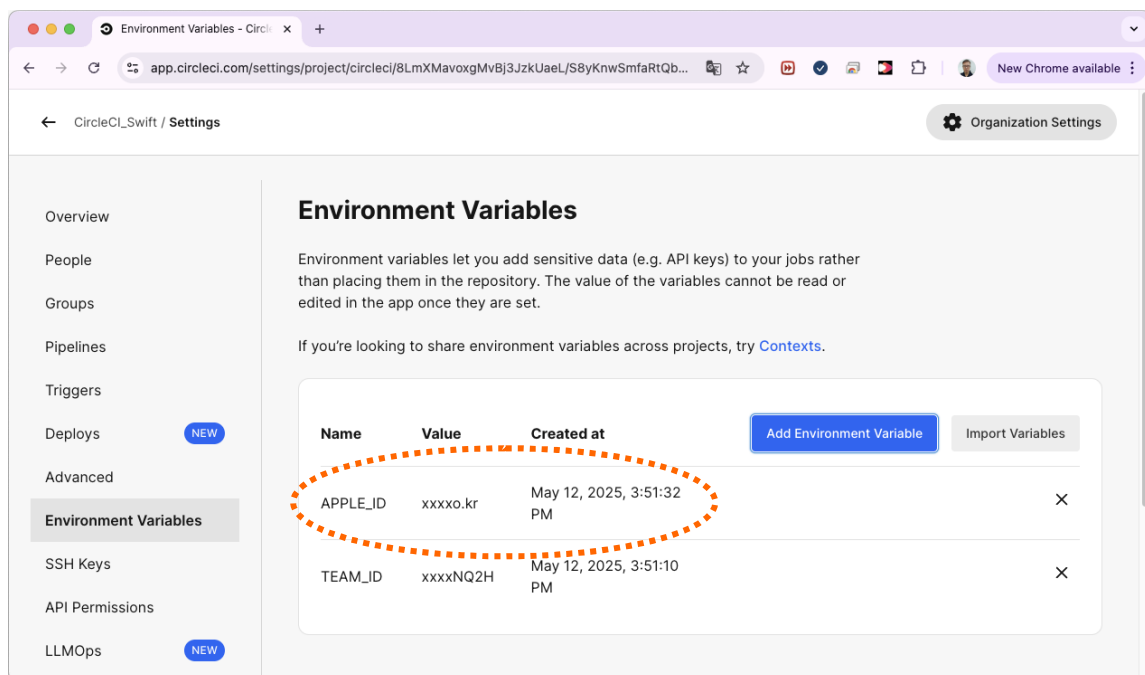
API Permissions

LLMOps NEW

버튼을 클릭하면 아래 화면과 같이 TEAM_ID 변수가 추가되어 있는 것을 확인할 수 있습니다.



같은 방법으로 "APPLE_ID" 이름과 실제 애플 계정의 ID를 값으로 하여 변수를 추가합니다.

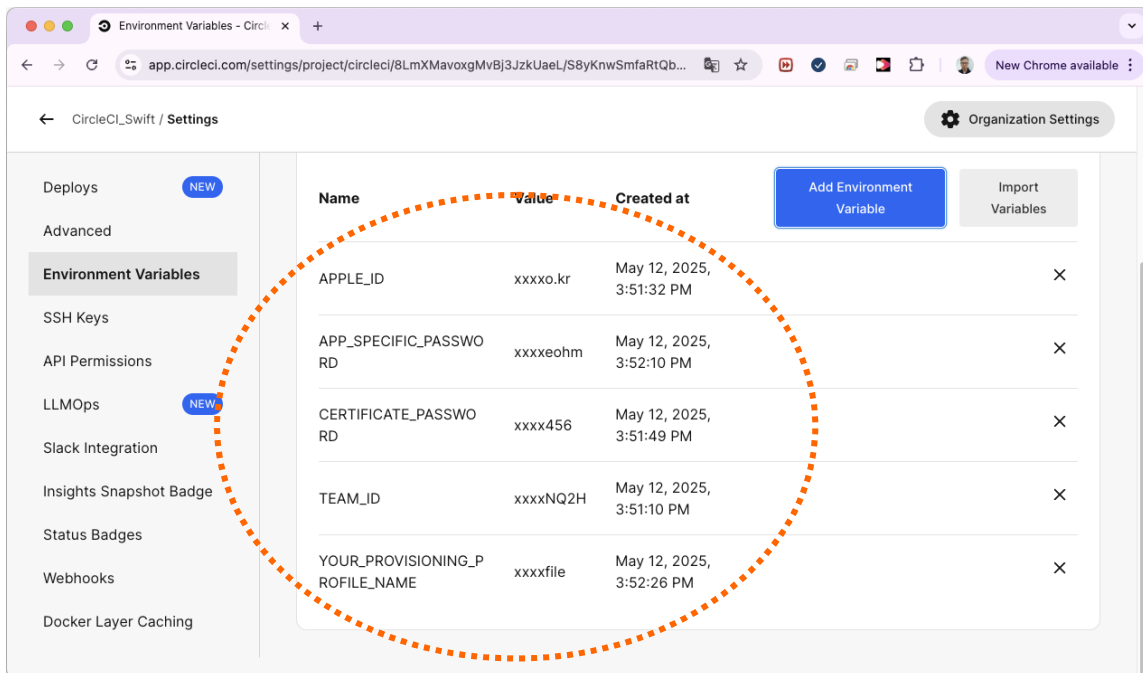


같은 방법으로 나머지 변수들을 모두 추가합니다.

APPLE_ID 외에 App Store Connect에서 사용하는 앱 전용 비밀번호와 배포용 인증서 P12 파일의 비밀번호, 배포용 프로비저닝 프로파일의 이름을 각각 입력합니다.

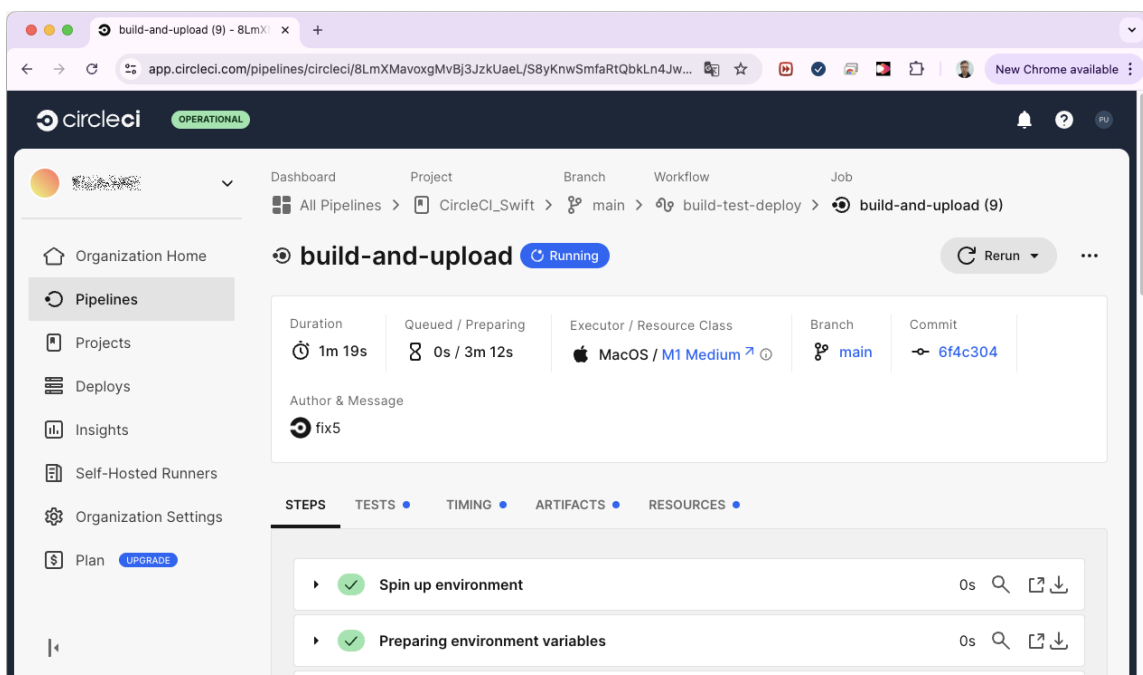
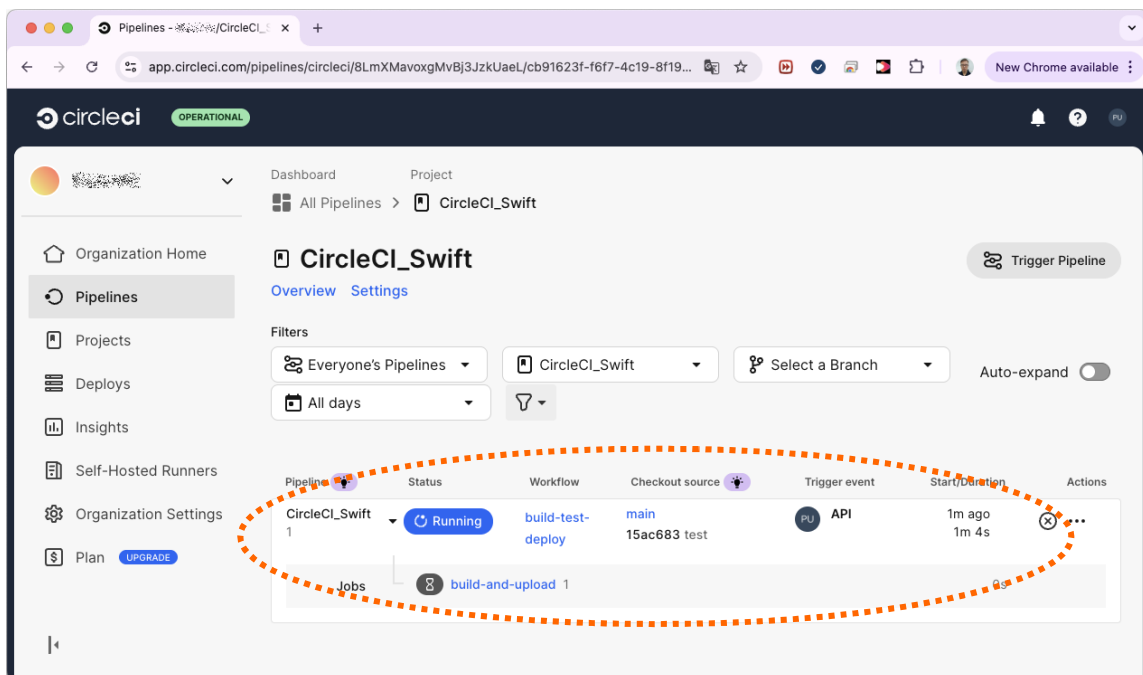
"APP_SPECIFIC_PASSWORD" 변수에 앱 전용 비밀번호 값, "CERTIFICATE_PASSWORD" 변수에 P12 비밀번호 값, 그리고 "YOUR_PROVISIONING_PROFILE_NAME" 변수에 배포용 프로파일 이름을 입력하면 됩니다.

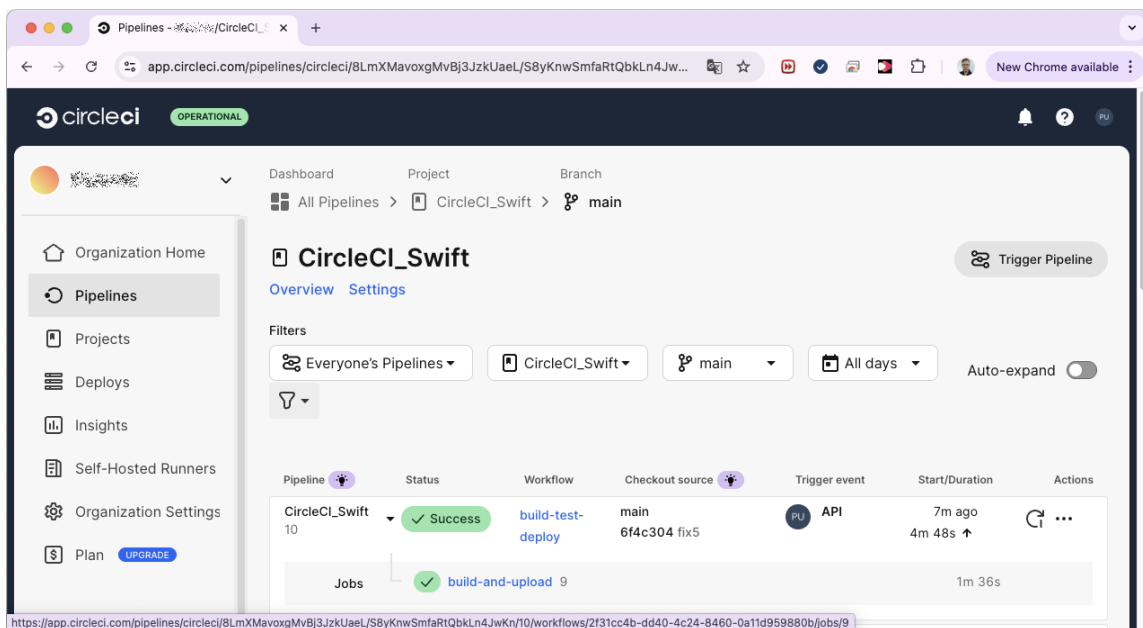
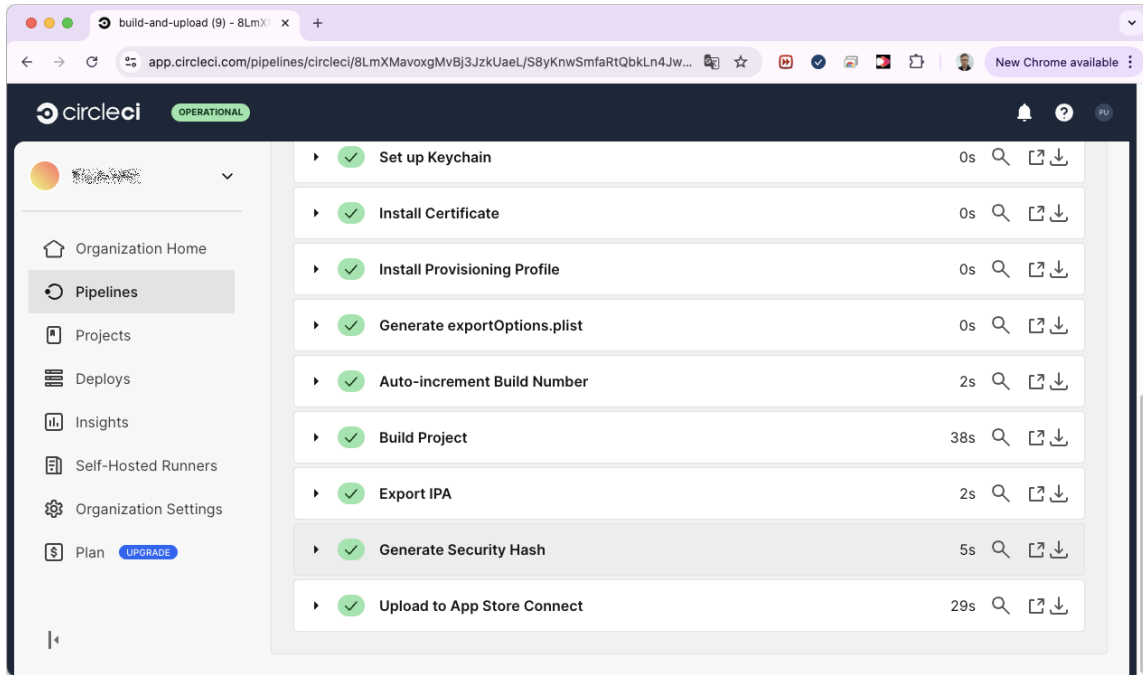
만약 앱 전용 비밀번호가 없다면 7-2절의 내용을 참고하여 새로 발급받아 사용하도록 합니다.



6-3 Circle CI 프로젝트 빌드

환경변수 설정이 모두 마무리 되면 빌드 스크립트를 교체합니다. Circle CI 파이프라인을 위한 빌드 스크립트는 SDK의 "ci_scripts/Circle CI" 안에 들어있는 config_####.yaml 파일을 사용합니다. Swift 또는 Objective-C 기본 프로젝트인 경우 config_swift.yaml 파일을 사용하여 파일명을 config.yaml로 변경하고 프로젝트 폴더의 .circleci 하위 폴더에 복사한 후 코드 저장소에 푸시하여 빌드가 시작되도록 합니다.





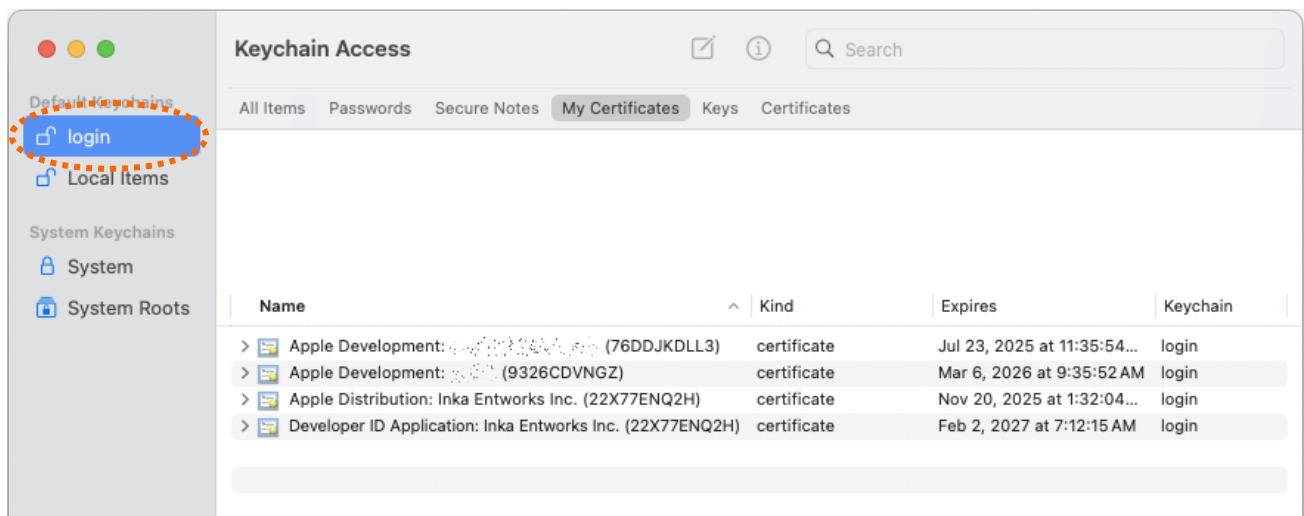
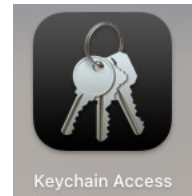
수정한 스크립트의 단계대로 빌드가 진행되면 IPA export와 generate_hash 적용, App Store Connect 업로드까지 모두 이루어 집니다.

프로젝트가 Flutter, Cordova, Ionic, React Native 기반일 경우 각각에 해당하는 "ci_scripts/config_PLATFORM.yml" 스크립트 파일을 대신 사용하도록 합니다.

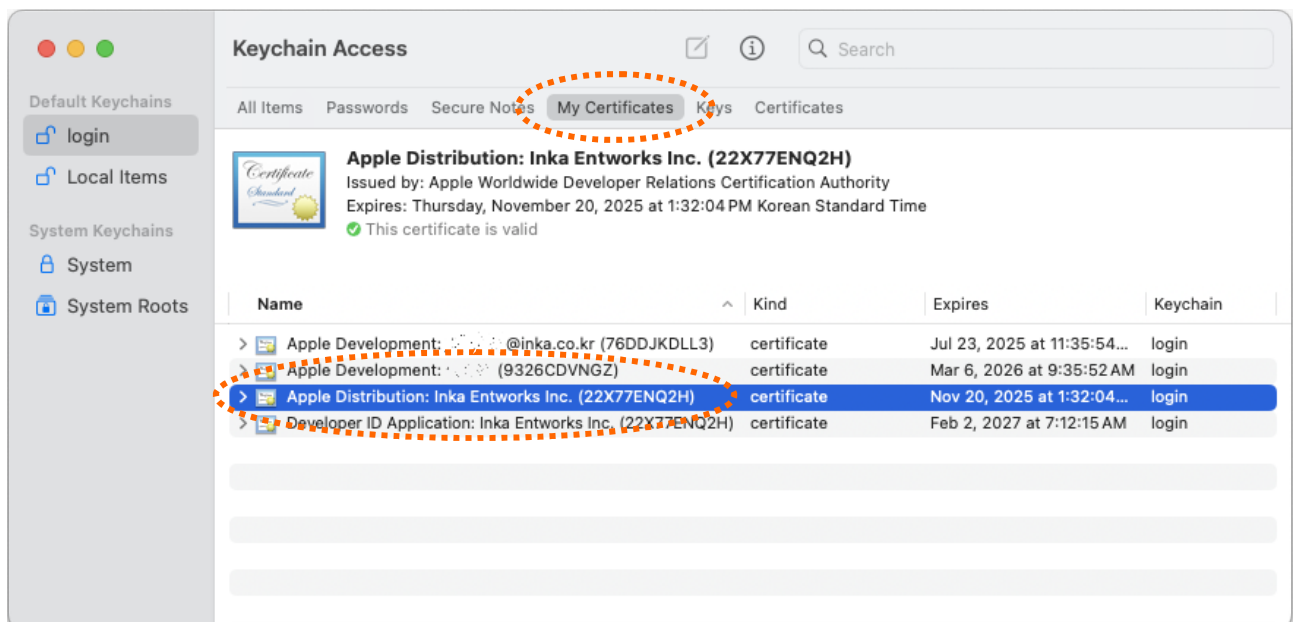
Part 7. 기타 / 공통 작업

7-1 배포용 인증서 PKCS#12 파일로 내보내기

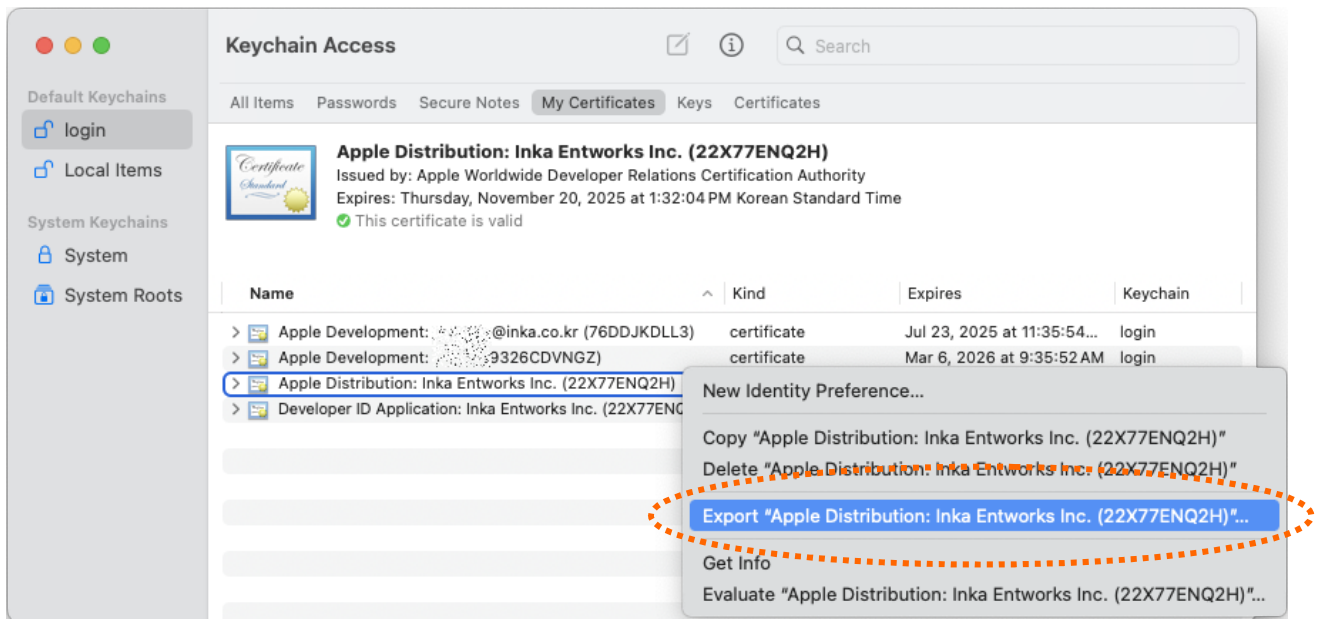
맥의 launchpad에서 기타 항목으로 들어가면 "Keychain Access" 앱이 포함되어 있습니다. 이 앱을 실행하고 아래와 같은 화면이 표시되면 좌측 탭에서 "login" 항목을 선택합니다.



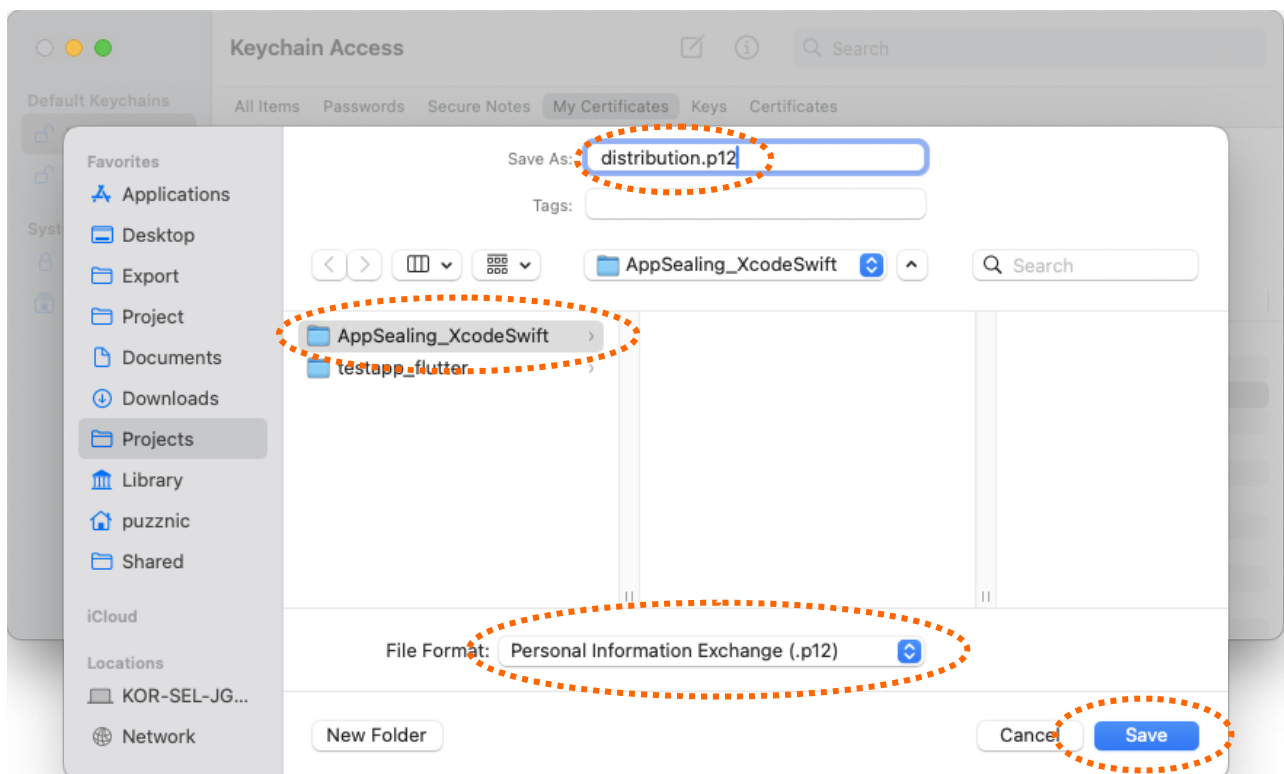
오른 쪽 창의 상단 탭 메뉴에서 "My Certificates"를 선택하고 키체인에 등록된 배포용 인증서를 선택합니다.



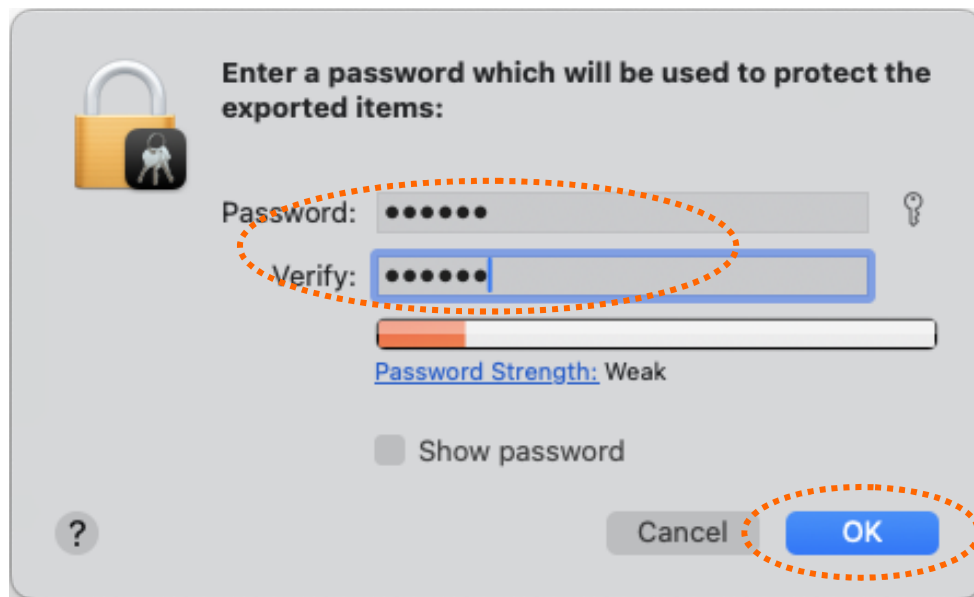
인증서를 선택한 후 마우스 우측 버튼을 클릭하여 컨텍스트 메뉴가 표시되면 "Export ..." 항목을 선택합니다.



저장 위치는 프로젝트 폴더, 파일 이름은 "distribution.p12", 파일 형식은 "Personal Information Exchange" 로 선택하세요.



Save 버튼을 클릭하고 아래 패스워드 입력창이 나타나면 CI 툴에 인증서를 등록하기 위한 패스워드를 입력합니다.

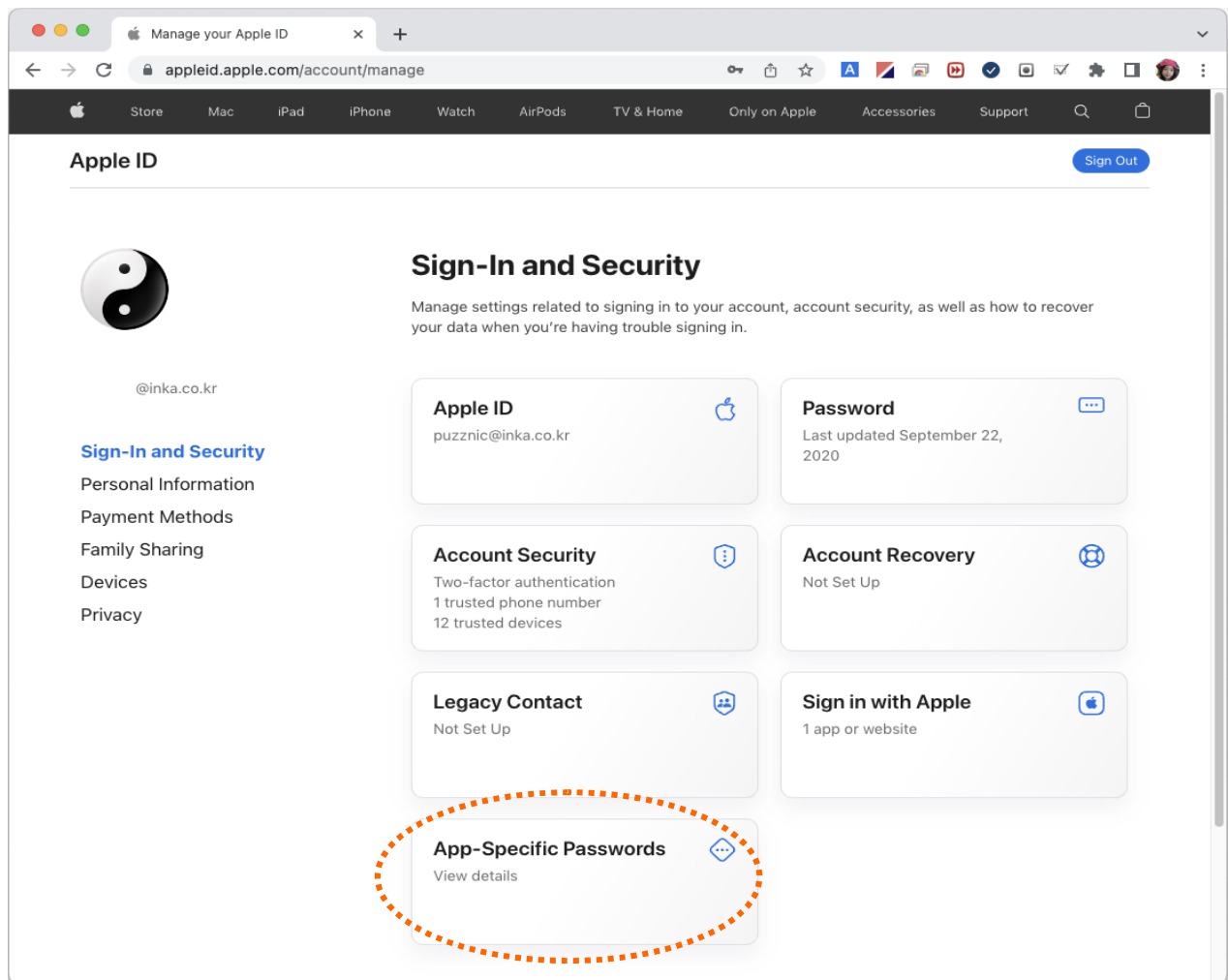


여기서 입력한 패스워드는 각 CI 툴 스크립트에 CERTIFICATE_PASSWORD 변수 값으로 직접 기록하거나 CI 툴 환경 변수로 등록하여 사용합니다.

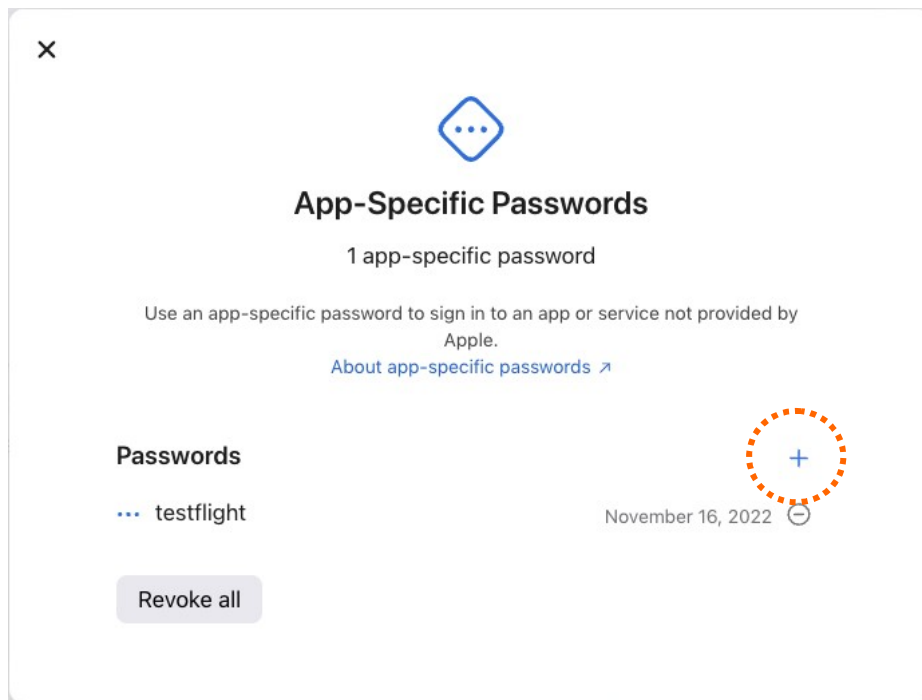
7-2 앱 전용 비밀번호 생성하기

App Store Connect에 IPA를 업로드 하기 위해 사용되는 앱 전용 비밀번호를 생성하기 위해 먼저 아래 페이지로 이동합니다.

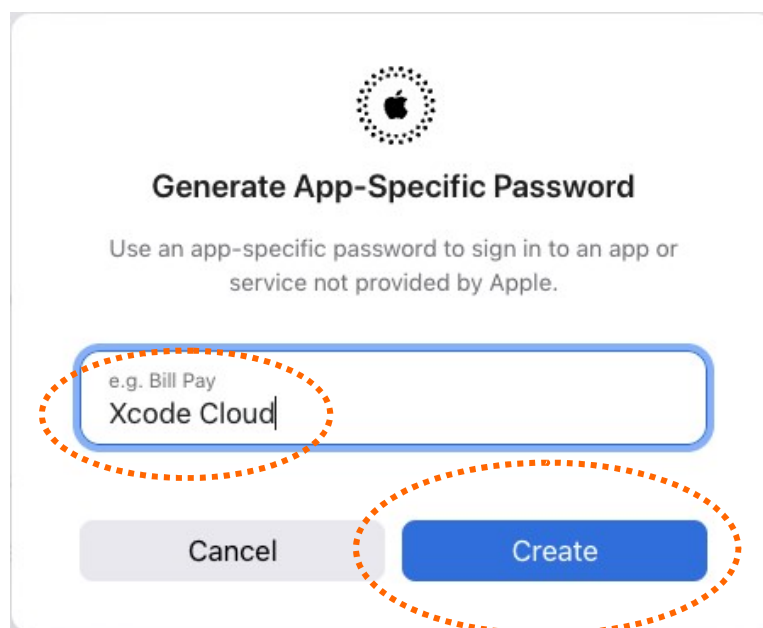
<https://appleid.apple.com/account/manage>



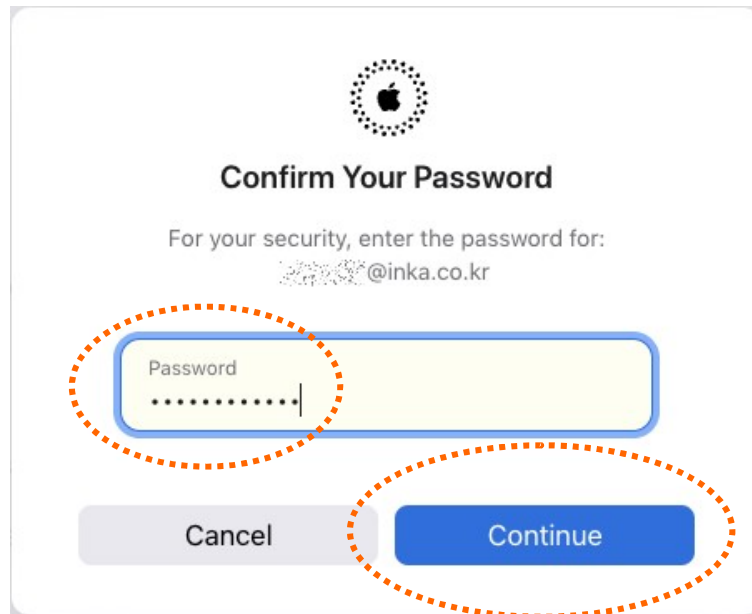
“Sign-In and Security” 페이지에서 “App-Specific Passwords” 항목을 선택하면 다음과 같이 “App-Specific Passwords” 창이 표시됩니다. 여기서 + 버튼을 클릭하여 새로운 앱 전용 비밀번호 생성을 시작합니다.



+ 버튼을 클릭하면 앱 전용 비밀번호의 이름을 입력하는 창이 나타납니다. 이름은 용도에 맞게 알아보기 편한 이름을 적당히 입력하면 됩니다. 여기서는 임의로 "Xcode Cloud"라는 이름을 입력했습니다. 이제 "Create" 버튼을 클릭합니다.



이제 Apple 계정 패스워드를 입력하라는 창이 나타나면 Apple ID의 계정 패스워드를 입력하고 "Continue" 버튼을 클릭합니다.



이제 아래 창과 같이 새로 생성된 16자리의 앱 전용 비밀번호가 표시됩니다. 이렇게 생성된 앱 전용 비밀번호는 이후 **값을 확인할 수 없고 폐기하는 것만 가능하므로 잘 기록해 두어야 합니다.**

